Last updated on ***Mar 6, 2026***

Lakeflow Spark Declarative Pipelines

Lakeflow Spark Declarative Pipelines (SDP) is a framework for creating batch and streaming data pipelines in SQL and Python. Lakeflow SDP extends and is interoperable with Apache Spark Declarative Pipelines, while running on the performance-optimized Databricks Runtime. Common use cases for pipelines include data ingestion from sources such as cloud storage (such as Amazon S3, Azure ADLS Gen2, and Google Cloud Storage) and message buses (such as Apache Kafka, Amazon Kinesis, Google Pub/Sub, Azure EventHub, and Apache Pulsar), and incremental batch and streaming transformations.

This section provides detailed information about using pipelines. The following topics will help you to get started.

Topic	Description
Lakeflow Spark Declarative Pipelines concepts	Learn about the high-level concepts of SDP, including pipelines, flows, streaming tables, and materialized views.
Tutorials	Follow tutorials to give you hands-on experience with using pipelines.
Develop pipelines	Learn how to develop and test pipelines that create flows for ingesting and transforming data.
Configure pipelines	Learn how to schedule and configure pipelines.
Monitor pipelines	Learn how to monitor your pipelines and troubleshoot pipeline queries.

Topic	Description
Developers	Learn how to use Python and SQL when developing pipelines.
Pipelines in Databricks SQL	Learn about using streaming tables and materialized views in Databricks SQL.
Best practices	Learn recommended patterns for building reliable, efficient, and maintainable pipelines.

More information

- Pipeline Limitations
- Pipeline developer reference

Last updated on ***Mar 4, 2026***

Lakeflow Spark Declarative Pipelines concepts

Learn what Lakeflow Spark Declarative Pipelines (SDP) is, the core concepts (such as pipelines, streaming tables, and materialized views) that define it, the relationships between those concepts, and the benefits of using it in your data processing workflows.

What is SDP?

Lakeflow Spark Declarative Pipelines is a declarative framework for developing and running batch and streaming data pipelines in SQL and Python. Lakeflow SDP extends and is interoperable with Apache Spark Declarative Pipelines, while running on the performance-optimized Databricks Runtime, and the Lakeflow Spark Declarative Pipelines `flows` API uses the same DataFrame API as Apache Spark and Structured Streaming. Common use cases for SDP include incremental data ingestion from sources such as cloud storage (including Amazon S3, Azure ADLS Gen2, and Google Cloud Storage) and message buses (such as Apache Kafka, Amazon Kinesis, Google Pub/Sub, Azure EventHub, and Apache Pulsar), incremental batch and streaming transformations with stateless and stateful operators, and real-time stream processing between transactional stores like message buses and databases.

For more details on declarative data processing, see Procedural vs. declarative data processing in Databricks.

What are the benefits of SDP?

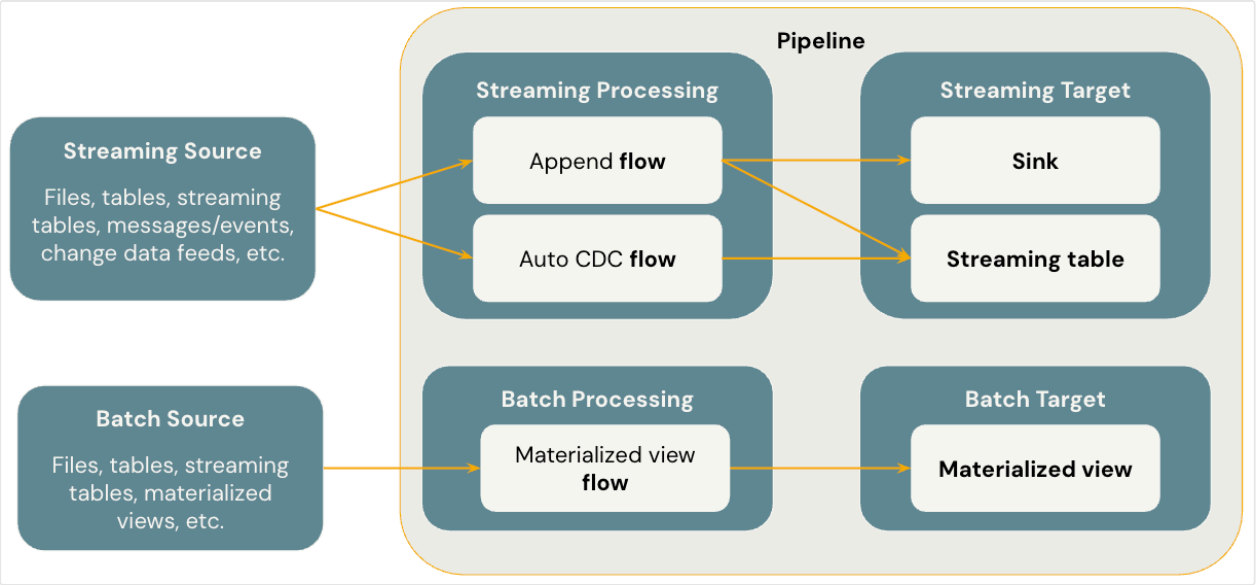
The declarative nature of SDP provides the following benefits compared to developing data processes with the Apache Spark and Spark Structured Streaming

running them with the Databricks Runtime using manual orchestration via Lakeflow Jobs.

- **Automatic orchestration**: SDP orchestrates processing steps (called "flows") automatically to ensure the correct order of execution and the maximum level of parallelism for optimal performance. Additionally, pipelines automatically and efficiently retry transient failures. The retry process begins with the most granular and cost-effective unit: the Spark task. If the task-level retry fails, SDP proceeds to retry the flow, and then finally the entire pipeline if necessary.
- **Declarative processing**: SDP provides declarative functions that can reduce hundreds or even thousands of lines of manual Spark and Structured Streaming code to only a few lines. The SDP AUTO CDC API simplifies processing of Change Data Capture (CDC) events with support for both SCD Type 1 and SCD Type 2. It eliminates the need for manual code to handle out-of-order events, and it does not require an understanding of streaming semantics or concepts like watermarks.
- **Incremental processing**: SDP provides an incremental processing engine for materialized views. To use it, you write your transformation logic with batch semantics, and the engine will only process new data and changes in the data sources whenever possible. Incremental processing reduces inefficient reprocessing when new data or changes occur in the sources and eliminates the need for manual code to handle incremental processing.

Key Concepts

The diagram below illustrates the most important concepts of Lakeflow Spark Declarative Pipelines.



Flows

A flow is the foundational data processing concept in SDP which supports both streaming and batch semantics. A flow reads data from a source, applies user-defined processing logic, and writes the result into a target. SDP shares the same streaming flow type (*Append*, *Update*, *Complete*) as Spark Structured Streaming. (Currently, only the *Append* flow is exposed.) For more details, see output modes in Structured Streaming.

Lakeflow Spark Declarative Pipelines also provides additional flow types:

- *AUTO CDC* is a unique streaming flow in Lakeflow SDP that handles out of order CDC events and supports both SCD Type 1 and SCD Type 2. Auto CDC is not available in Apache Spark Declarative Pipelines.
- *Materialized view* is a batch flow in SDP that only processes new data and changes in the source tables whenever possible.

For more details, see:

- Load and process data incrementally with SDP flows

Streaming tables

A *streaming table* is a form of Unity Catalog managed table that is also a streaming target for Lakeflow SDP. A streaming table can have one or more streaming flows

(*Append*, *AUTO CDC*) written into it. *AUTO CDC* is a unique streaming flow that's only available to streaming tables in Databricks. You can define streaming flows explicitly and separately from their target streaming table. You can also define streaming flows implicitly as part of a streaming table definition.

For more details, see:

- How streaming tables work

Materialized views

A *materialized view* is also a form of Unity Catalog managed table and is a batch target. A materialized view can have one or more materialized view flows written into it. Materialized views differ from streaming tables in that you always define the flows implicitly as part of the materialized view definition.

For more details, see:

- How materialized views work

Sinks

A *sink* is a streaming target for a pipeline and supports Delta tables, Apache Kafka topics, Azure EventHubs topics, and custom Python data sources. A sink can have one or more streaming flows (*Append*) written into it.

For more details, see:

- Stream records to external services with sinks

Pipelines

A *pipeline* is the unit of development and execution in Lakeflow Spark Declarative Pipelines. A pipeline can contain one or more flows, streaming tables, materialized views, and sinks. You use SDP by defining flows, streaming tables, materialized views, and sinks in your pipeline source code and then running the pipeline. While your pipeline runs, it analyzes the dependencies of your defined flows, streaming tables,

materialized views, and sinks, and orchestrates their order of execution and parallelization automatically.

For more details, see:

- Configure a pipeline

Databricks SQL pipelines

Streaming tables and materialized views are two foundational capabilities in Databricks SQL. You can use standard SQL to create and refresh streaming tables and materialized views in Databricks SQL. Streaming tables and materialized views in Databricks SQL run on the same Databricks infrastructure and have the same processing semantics as they do in Lakeflow Spark Declarative Pipelines. When you use streaming tables and materialized views in Databricks SQL, flows are defined implicitly as part of the streaming tables and materialized views definition.

For more details, see:

- Standalone pipelines

More information

- Tutorial: Build an ETL pipeline using change data capture
- Develop Lakeflow Spark Declarative Pipelines
- Pipeline Limitations
- Pipeline developer reference

Last updated on ***Apr 10, 2026***

Lakeflow Spark Declarative Pipelines tutorials

The tutorials in this section are designed to help you learn about SDP. The following tutorials are available:

Get started with Lakeflow Spark Declarative Pipelines

Topic	Description
Tutorial: Build an ETL pipeline with Lakeflow Spark Declarative Pipelines	This tutorial shows you how to create and deploy a simple ETL (extract, transform, and load) pipeline for data orchestration Auto Loader.

Tutorials

Topic	Description
Tutorial: Create your first pipeline using the Lakeflow Pipelines Editor	This tutorial walks you through creating your first pipeline by extending the sample code that is created with a new pipeline.
Tutorial: Build an ETL pipeline using change data capture	This tutorial walks you through the steps to create and deploy an ETL pipeline with change data capture (CDC) using SDP for data orchestration andAuto l

Topic	Description
ETL in Databricks SQL	This tutorial walks you through building an incremental ETL pipeline in Databricks SQL using streaming tables, `AUTO CDC`, and materialized views.
Tutorial: Build a geospatial pipeline with native spatial types	This tutorial walks you through building a geospatial pipeline that ingests GPS data, converts coordinates to native spatial types, and joins against warehouse geofences using SDP and Auto Loader.
Create a source-controlled pipeline	This tutorial shows you how to create a source-controlled pipeline using Declarative Automation Bundles and Git folders.
Convert a pipeline into a bundle project	This tutorial shows you how to convert an existing pipeline into a Declarative Automation Bundles project, to provide easier maintenance and automated deployment to target environments.

*Last updated on **May 12, 2026***

Tutorial: Create your first pipeline using the Lakeflow Pipelines Editor

Learn how to create a new pipeline using Lakeflow Spark Declarative Pipelines (SDP) for data orchestration and Auto Loader. This tutorial extends the sample pipeline by cleaning the data and creating a query to find the top 100 users.

In this tutorial, you learn how to use the Lakeflow Pipelines Editor to:

- Create a new pipeline with the default folder structure and start with a set of sample files.
- Define data quality constraints using expectations.
- Use the editor features to extend the pipeline with a new transformation to perform analysis on your data.

Requirements

Before you start this tutorial, you must:

- Be logged into a Databricks workspace.
- Have Unity Catalog enabled for your workspace.
- Have permission to create a compute resource or access to a compute resource.
- Have permissions to create a new schema in a catalog. The required permissions are `ALL PRIVILEGES` or `USE CATALOG` and `CREATE SCHEMA`.

Step 1: Create a pipeline

In this step, you create a pipeline using the default folder structure and code samples. The code samples reference the `users` table in the `wanderbricks` sample data source.

1. In your Databricks workspace, click + **New**, then **ETL pipeline**. This opens the pipeline editor with a default pipeline name like `New Pipeline <date> <time>`.

2. (Optional) Select the name and enter a descriptive name for the pipeline.

3. (Optional) To the right of the name, click the catalog and schema to set different defaults.

4. (Optional) In the `my_transformation` source file created for you, select **Python** or **SQL** from the language drop-down list to set the language for the file.

5. Click **Use sample code**.

   Sample code in your selected language appears in the `my_transformation` source file in the `transformations` folder. The output datasets have not yet been created, and the **Pipeline graph** on the right side of the screen is empty.

6. To run the pipeline code (the code in the `transformations` folder), click **Run pipeline** in the upper right part of the screen.

   After the run completes, the bottom part of the workspace shows the two new tables that were created, `sample_users_<date_time>` and `sample_aggregation_<date_time>`. The **Pipeline graph** on the right side of the workspace now shows the two tables, including that `sample_users` is the source for `sample_aggregation`. Make a note of the full `sample_users_<date_time>` table name — you reference it in the next step.

Step 2: Apply data quality checks

In this step, you add a data quality check to the `sample_users` table. You use pipeline expectations to constrain the data. In this case, you delete any user records that do not have a valid email address, and output the cleaned table as `users_cleaned`.

1. In the pipeline asset browser on the left, click +, and select **Transformation**.

2. In the **Create new transformation file** dialog, make the following selections:

- Choose either **Python** or **SQL** for the **Language**. This does not have to match your previous selection.
- Give the file a name. In this case, choose `users_cleaned`.
- For **Destination path**, leave the default.
- For **Dataset type**, either leave it as **None selected** or choose **Materialized view**. If you select **Materialized view**, it generates sample code for you.

3. Click **Create** to create the transformation code file.

4. In your new code file, edit the code to match the following (use SQL or Python, based on your selection on the previous screen). Replace `sample_users_<date_time>` with the full name of your `sample_users` table from the previous section.

SQL Python

SQL

```
-- Drop all rows that do not have an email address

CREATE MATERIALIZED VIEW users_cleaned
(
  CONSTRAINT non_null_email EXPECT (email IS NOT NULL) ON VIOLATION
DROP ROW
) AS
SELECT *
FROM sample_users_<date_time>;
```

5. Click **Run pipeline** to update the pipeline. It should now have three tables.

Step 3: Analyze top users

Next get the top 100 users by the number of bookings they have created. Join the `wanderbricks.bookings` table to the `users_cleaned` materialized view.

1. In the pipeline asset browser on the left, click +, and select **Transformation**.

2. In the **Create new transformation file** dialog, make the following selections:

- Choose either **Python** or **SQL** for the **Language**. This does not have to match your previous selections.
- Give the file a name. In this case, choose `users_and_bookings`.
- For **Destination path**, leave the default.
- For **Dataset type**, leave it as **None selected**.

3. Click **Create** to create the transformation code file.

4. In your new code file, edit the code to match the following (use SQL or Python, based on your selection on the previous screen).

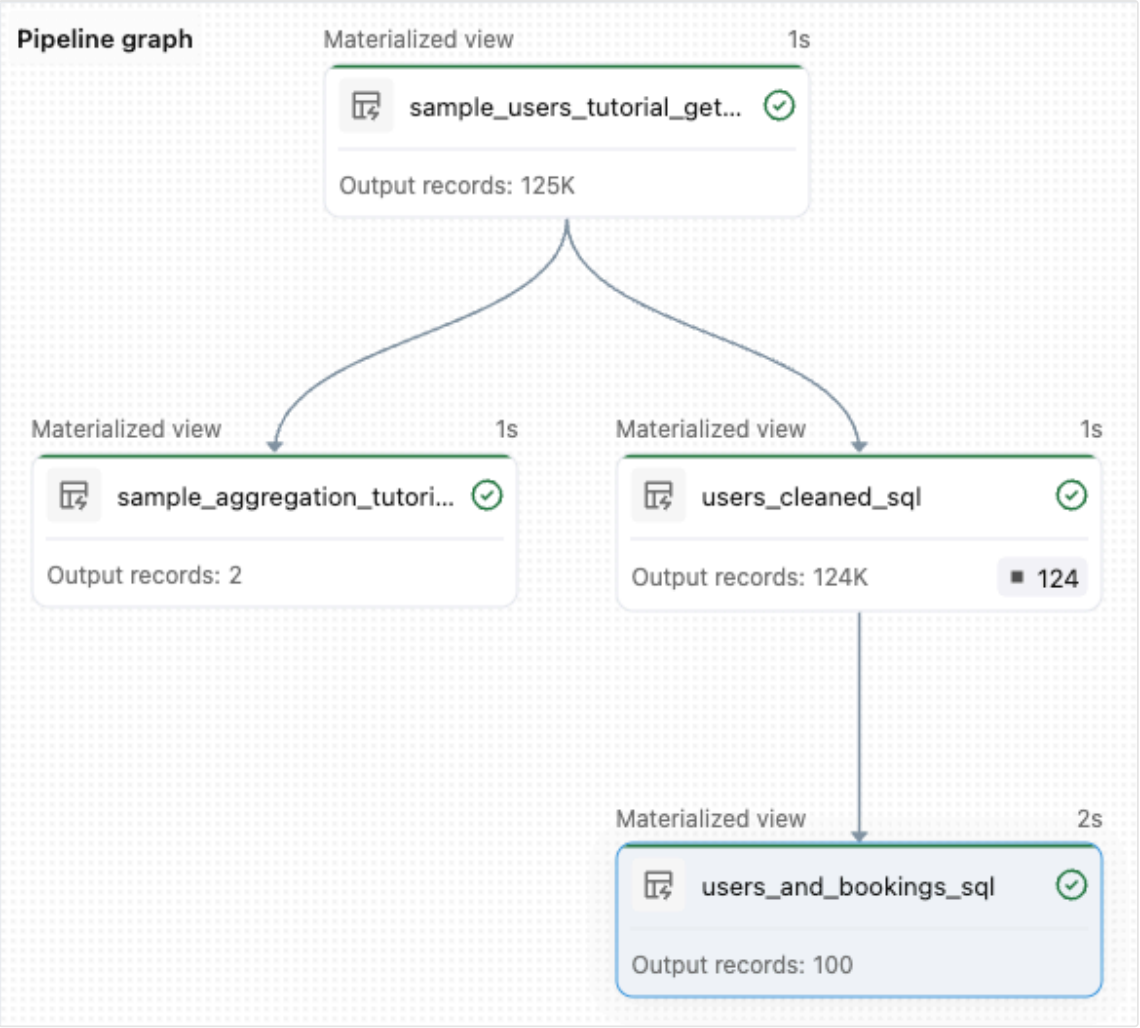
SQL Python

SQL

```sql
-- Get the top 100 users by number of bookings

CREATE OR REFRESH MATERIALIZED VIEW users_and_bookings AS
SELECT u.name AS name, COUNT(b.booking_id) AS booking_count
FROM users_cleaned u
JOIN samples.wanderbricks.bookings b ON u.user_id = b.user_id
GROUP BY u.name
ORDER BY booking_count DESC
LIMIT 100;
```

5. Click **Run pipeline** to update the datasets. When the run is complete, you can see in the **Pipeline Graph** that there are four tables, including the new `users_and_bookings` table.



Next steps

Now that you have learned how to use some of the features of the Lakeflow pipelines editor and created a pipeline, here are some other features to learn more about:

- Tools for working with and debugging transformations while creating pipelines:
  - Selective execution
  - Data previews
  - Interactive pipeline graph (graph of the datasets in your pipeline)
- Built-in Declarative Automation Bundles integration for efficient collaboration, version control, and CI/CD integration directly from the editor:
  - Create a source-controlled pipeline
  - Convert to Declarative Automation Bundles

this page

as this page helpful?

Yes | No

Send feedback

Ask Genie

*Last updated on **May 12, 2026***

Tutorial: Build an ETL pipeline using change data capture

Learn how to create and deploy an ETL (extract, transform, and load) pipeline with change data capture (CDC) using Lakeflow Spark Declarative Pipelines (SDP) for data orchestration and Auto Loader. An ETL pipeline implements the steps to read data from source systems, transform that data based on requirements, such as data quality checks and record de-duplication, and write the data to a target system, such as a data warehouse or a data lake.

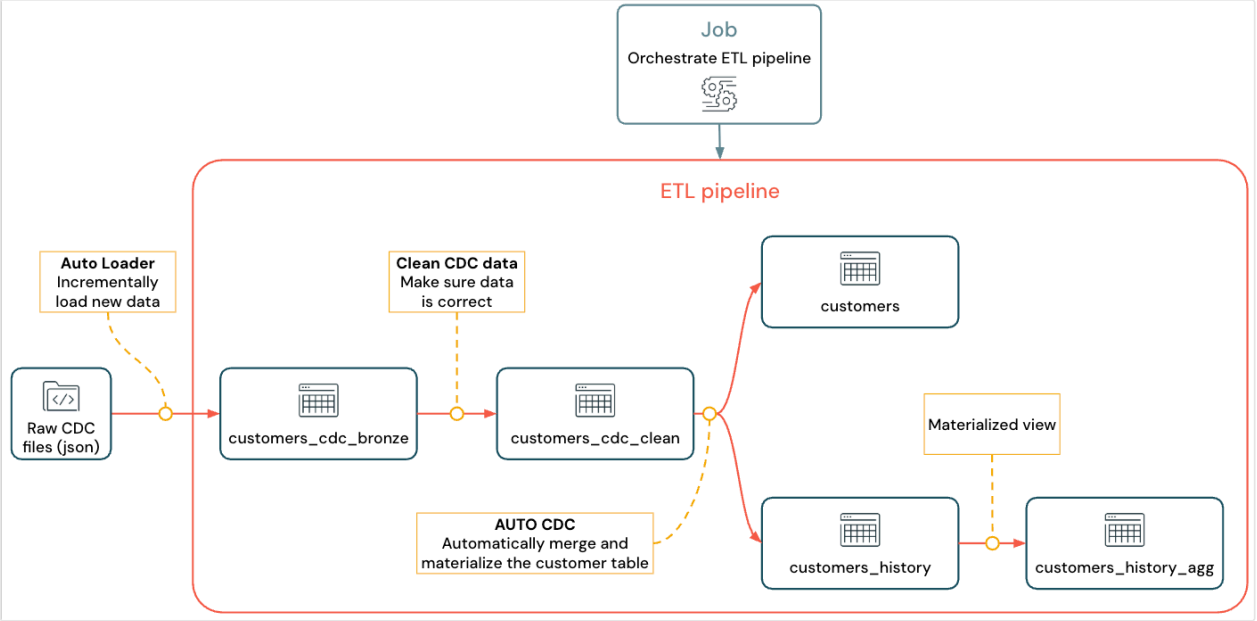
In this tutorial, you'll use data from a `customers` table in a MySQL database to:

- Extract the changes from a transactional database using Debezium or another tool and save them to cloud object storage (S3, ADLS, or GCS). In this tutorial, you skip setting up an external CDC system and instead generate fake data to simplify the tutorial.
- Use Auto Loader to incrementally load the messages from cloud object storage, and store the raw messages in the `customers_cdc` table. Auto Loader infers the schema and handles schema evolution.
- Create the `customers_cdc_clean` table to check data quality using expectations. For example, the `id` should never be `null` because it's used to run upsert operations.
- Perform `AUTO CDC ... INTO` on the cleaned CDC data to upsert changes into the final `customers` table.
- Show how a pipeline can create a type 2 slowly changing dimension (SCD2) table to track all changes.

The goal is to ingest the raw data in near real time and build a table for your analyst team while ensuring data quality.

The tutorial uses the medallion Lakehouse architecture, where it ingests raw data through the bronze layer, cleans and validates data with the silver layer, and applies dimensional modeling and aggregation using the gold layer. See What is the medallion lakehouse architecture? for more information.

The implemented flow looks like this:



For more information about pipeline, Auto Loader, and CDC see Lakeflow Spark Declarative Pipelines, What is Auto Loader?, and Change data capture and snapshots

Requirements

To complete this tutorial, you must meet the following requirements:

- Be logged in to a Databricks workspace.
- Have Unity Catalog enabled for your workspace.
- Have serverless compute available in your workspace (enabled by default in workspaces with Unity Catalog). Serverless Lakeflow Spark Declarative Pipelines is not available in all workspace regions. See Features with limited regional availability for available regions. If serverless compute is not available, the steps should work with the default compute for your workspace.
- Have permission to create a compute resource or access to a compute resource.
- Have permissions to create a new schema in a catalog. The required permissions are `USE CATALOG` and `CREATE SCHEMA`.

- Have permissions to create a new volume in an existing schema. The required permissions are `USE SCHEMA` and `CREATE VOLUME`.

Change data capture in an ETL pipeline

Change data capture (CDC) is the process that captures changes in records made to a transactional database (for example, MySQL or PostgreSQL) or a data warehouse. CDC captures operations like data deletes, appends, and updates, typically as a stream to re-materialize tables in external systems. CDC enables incremental loading while eliminating the need for bulk-load updates.

> **NOTE**
>
> To simplify this tutorial, skip setting up an external CDC system. Assume it's running and saving CDC data as JSON files in cloud object storage (S3, ADLS, or GCS). This tutorial uses the `Faker` library to generate the data used in the tutorial.

Capturing CDC

A variety of CDC tools are available. One of the leading open source solutions is Debezium, but other implementations that simplify data sources exist, such as Fivetran, Qlik Replicate, StreamSets, Talend, Oracle GoldenGate, and AWS DMS.

In this tutorial, you use CDC data from an external system like Debezium or DMS. Debezium captures every changed row. It typically sends the history of data changes to Kafka topics or saves them as files.

You must ingest the CDC information from the `customers` table (JSON format), check that it is correct, and then materialize the customers table in the Lakehouse.

CDC input from Debezium

For each change, you receive a JSON message containing all the fields of the row being updated (`id`, `firstname`, `lastname`, `email`, `address`). The message also includes

additional metadata:

- `operation`: An operation code, typically (`DELETE`, `APPEND`, `UPDATE`).
- `operation_date`: The date and timestamp for the record for each operation action.

Tools like Debezium can produce more advanced output, such as the row value before the change, but this tutorial omits them for simplicity.

Step 1: Create a pipeline

Create a new ETL pipeline to query your CDC data source and generate tables in your workspace.

1. In your workspace, click + **New** in the sidebar, then select **ETL Pipeline**. This opens the pipeline editor with a default pipeline name like `New Pipeline <date> <time>`.

2. Select the name and enter a descriptive name, such as `Pipelines with CDC tutorial`.

3. To the right of the name, click the catalog and schema to choose defaults for which you have write permissions.

   This catalog and schema are used by default, if you do not specify a catalog or schema in your code. Your code can write to any catalog or schema by specifying the full path. This tutorial uses the defaults that you specify here.

4. (Optional) In the `my_transformation` source file created for you, select **Python** or **SQL** from the language drop-down list to set the language for the file.

5. Click **Use sample code**.

The Lakeflow Pipelines Editor opens with sample files in your pipeline. You will add your own transformation files in later steps. Next, set up the sample data to import in the tutorial.

Step 2: Create the sample data to import in this tutorial

This step is not needed if you are importing your own data from an existing source. For this tutorial, generate fake data as an example for the tutorial. Create a notebook to run the Python data generation script. This code only needs to be run once to generate the sample data, so create it within the pipeline's `explorations` folder, which is not run as part of a pipeline update.

> **NOTE**
>
> This code uses Faker to generate the sample CDC data. Faker is available to install automatically, so the tutorial uses `%pip install faker`. You can also set a dependency on faker for the notebook. See Add dependencies to the notebook.

1. From within the Lakeflow Pipelines Editor, in the asset browser sidebar to the left of the editor, click + **Add**, then choose **Exploration**.

2. Give it a **Name**, such as `Setup data`, select **Python**. You can leave the default destination folder, which is a new `explorations` folder.

3. Click **Create**. This creates a notebook in the new folder.

4. Enter the following code in the first cell. You must change the definition of `<my_catalog>` and `<my_schema>` to match the default catalog and schema that you selected in the previous procedure:

Python

```
%pip install faker
# Update these to match the catalog and schema
# that you used for the pipeline in step 1.
catalog = "<my_catalog>"
schema = dbName = db = "<my_schema>"

spark.sql(f'USE CATALOG `{catalog}`')
spark.sql(f'USE SCHEMA `{schema}`')
spark.sql(f'CREATE VOLUME IF NOT EXISTS `{catalog}`.`{db}`.`raw_data`')
volume_folder =  f"/Volumes/{catalog}/{db}/raw_data"
```

```
try:
  dbutils.fs.ls(volume_folder+"/customers")
except:
  print(f"folder doesn't exist, generating the data under {volume_folder
  from pyspark.sql import functions as F
  from faker import Faker
  from collections import OrderedDict
  import uuid
  fake = Faker()
  import random

  fake_firstname = F.udf(fake.first_name)
  fake_lastname = F.udf(fake.last_name)
  fake_email = F.udf(fake.ascii_company_email)
  fake_date = F.udf(lambda:fake.date_time_this_month().strftime("%m-%d-%
  fake_address = F.udf(fake.address)
  operations = OrderedDict([("APPEND", 0.5),("DELETE", 0.1),("UPDATE", 0
  fake_operation = F.udf(lambda:fake.random_elements(elements=operations
  fake_id = F.udf(lambda: str(uuid.uuid4()) if random.uniform(0, 1) < 0.

  df = spark.range(0, 100000).repartition(100)
  df = df.withColumn("id", fake_id())
  df = df.withColumn("firstname", fake_firstname())
  df = df.withColumn("lastname", fake_lastname())
  df = df.withColumn("email", fake_email())
  df = df.withColumn("address", fake_address())
  df = df.withColumn("operation", fake_operation())
  df_customers = df.withColumn("operation_date", fake_date())

df_customers.repartition(100).write.format("json").mode("overwrite").sav
```

5. To generate the dataset used in the tutorial, type **Shift** + **Enter** to run the code:

6. Optional. To preview the data used in this tutorial, enter the following code in the next cell and run the code. Update the catalog and schema to match the path from the previous code.

Python

```
# Update these to match the catalog and schema
# that you used for the pipeline in step 1.
catalog = "<my_catalog>"
schema = "<my_schema>"

display(spark.read.json(f"/Volumes/{catalog}/{schema}/raw_data/customers
```

This generates a large data set (with fake CDC data) that you can use in the rest of the tutorial. In the next step, ingest the data using Auto Loader.

Step 3: Incrementally ingest data with Auto Loader

The next step is to ingest the raw data from the (faked) cloud storage into a bronze layer.

This can be challenging for multiple reasons, as you must:

- Operate at scale, potentially ingesting millions of small files.
- Infer schema and JSON type.
- Handle bad records with incorrect JSON schema.
- Take care of schema evolution (for example, a new column in the customer table).

Auto Loader simplifies this ingestion, including schema inference and schema evolution, while scaling to millions of incoming files. Auto Loader is available in Python using `cloudFiles` and in SQL using the `SELECT * FROM STREAM read_files(...)` and can be used with a variety of formats (JSON, CSV, Apache Avro, etc.):

Defining the table as a streaming table guarantees that you only consume new incoming data. If you do not define it as a streaming table, it scans and ingests all the available data. See Streaming tables for more information.

1. To ingest the incoming CDC data using Auto Loader, copy and paste the following code into the code file that was created with your pipeline (called `my_transformation.py` or `my_transformation.sql`). You can use Python or SQL, based on the language you chose when creating the pipeline. Be sure to replace the `<catalog>` and `<schema>` with the ones that you set up for the default for the pipeline.

Python SQL

Python Ask Genie

```
from pyspark import pipelines as dp
from pyspark.sql.functions import *

# Replace with the catalog and schema name that
# you are using:
path = "/Volumes/<catalog>/<schema>/raw_data/customers"


# Create the target bronze table
dp.create_streaming_table("customers_cdc_bronze", comment="New
customer data incrementally ingested from cloud object storage
landing zone")

# Create an Append Flow to ingest the raw data into the bronze table
@dp.append_flow(
  target = "customers_cdc_bronze",
  name = "customers_bronze_ingest_flow"
)
def customers_bronze_ingest_flow():
  return (
      spark.readStream
          .format("cloudFiles")
          .option("cloudFiles.format", "json")
          .option("cloudFiles.inferColumnTypes", "true")
          .load(f"{path}")
  )
```

2. Click ▶ **Run file** or **Run pipeline** to start an update for the connected pipeline. With only one source file in your pipeline, these are functionally equivalent.

When the update completes, the editor is updated with information about your pipeline.

- The pipeline graph, also called the directed acyclic graph (DAG), in the sidebar to the right of your code, shows a single table, `customers_cdc_bronze`.
- A summary of the update is shown at the top of the pipeline asset browser.
- Details of the table that was generated are shown in the bottom pane, and you can browse data from the table by selecting it.

This is the raw bronze layer data imported from cloud storage. In the next step, clean the data to create a silver layer table.

Step 4: Cleanup and expectations to track data quality

After the bronze layer is defined, create the silver layer by adding expectations to control data quality. Check the following conditions:

- ID must never be `null`.
- The CDC operation type must be valid.
- JSON must be read correctly by Auto Loader.

Rows that don't meet these conditions are dropped.

See Manage data quality with pipeline expectations for more information.

1. From the pipeline asset browser sidebar, click + **Add**, then **Transformation**.
2. Enter a **Name** (for example, `customers_silver`) and choose a language (Python or SQL) for the source code file. The `.py` or `.sql` extension is appended based on your language choice. You can mix and match languages within a pipeline, so you can choose either one for this step.
3. To create a silver layer with a cleansed table and impose constraints, copy and paste the following code into the new file (choose Python or SQL based on the language of the file).

Python SQL

Python

```
from pyspark import pipelines as dp
from pyspark.sql.functions import *

dp.create_streaming_table(
  name = "customers_cdc_clean",
  expect_all_or_drop = {"no_rescued_data": "_rescued_data IS
NULL","valid_id": "id IS NOT NULL","valid_operation": "operation IN
('APPEND', 'DELETE', 'UPDATE')"}
  )
```

Ask Genie

```
@dp.append_flow(
  target = "customers_cdc_clean",
  name = "customers_cdc_clean_flow"
)
def customers_cdc_clean_flow():
  return (
      spark.readStream.table("customers_cdc_bronze")
          .select("address", "email", "id", "firstname", "lastname",
"operation", "operation_date", "_rescued_data")
  )
```

4. Click ▶ **Run file** or **Run pipeline** to start an update for the connected pipeline.

   Because there are now two source files, these do not do the same thing, but in this case, the output is the same.

   - **Run pipeline** runs your entire pipeline, including the code from step 3. If your input data were being updated, this would pull in any changes from that source to your bronze layer. This does not run the code from the data setup step, because that is in the explorations folder, and not part of the source for your pipeline.
   - **Run file** runs only the current source file. In this case, without your input data being updated, this generates the silver data from the cached bronze table. It would be useful to run just this file for faster iteration when creating or editing your pipeline code.

When the update completes, you can see that the pipeline graph now shows two tables (with the silver layer depending on the bronze layer), and the bottom panel shows details for both tables. The top of the pipeline asset browser now shows multiple runs' times, but only details for the most recent run.

Next, create your final gold layer version of the `customers` table.

Step 5: Materialize the customers table with an AUTO CDC flow

Up to this point, the tables have just passed the CDC data along in each step. Now, create the `customers` table to both contain the most up-to-date view and to be a replica of the original table, not the list of CDC operations that created it.

This is nontrivial to implement manually. You must consider things like data deduplication to keep the most recent row.

However, Lakeflow Spark Declarative Pipelines solves these challenges with the `AUTO CDC` operation.

1. From the pipeline asset browser sidebar, click + **Add** and **Transformation**.
2. Enter a **Name** and choose a language (Python or SQL) for the new source code file. You can again choose either language for this step, but use the correct code, below.
3. To process the CDC data using `AUTO CDC` in Lakeflow Spark Declarative Pipelines, copy and paste the following code into the new file.

Python SQL

Python

```
from pyspark import pipelines as dp
from pyspark.sql.functions import *

dp.create_streaming_table(name="customers", comment="Clean,
materialized customers")

dp.create_auto_cdc_flow(
  target="customers",  # The customer table being materialized
  source="customers_cdc_clean",  # the incoming CDC
  keys=["id"],  # what we'll be using to match the rows to upsert
  sequence_by=col("operation_date"),  # de-duplicate by operation
date, getting the most recent value
  ignore_null_updates=False,
  apply_as_deletes=expr("operation = 'DELETE'"),  # DELETE condition
  except_column_list=["operation", "operation_date",
"_rescued_data"],
)
```

4. Click ▶ **Run file** to start an update for the connected pipeline.

When the update is complete, you can see that your pipeline graph shows 3 tables, progressing from bronze to silver to gold.

Step 6: Track update history with slowly changing dimension type 2 (SCD2)

It's often required to create a table tracking all the changes resulting from `APPEND`, `UPDATE`, and `DELETE`:

- History: You want to keep a history of all the changes to your table.
- Traceability: You want to see which operation occurred.

SCD2 with Lakeflow SDP

Delta supports change data flow (CDF), and `table_change` can query table modifications in SQL and Python. However, CDF's main use case is to capture changes in a pipeline, not to create a full view of table changes from the beginning.

Things get especially complex to implement if you have out-of-order events. If you must sequence your changes by a timestamp and receive a modification that happened in the past, you must append a new entry in your SCD table and update the previous entries.

Lakeflow SDP removes this complexity. You can create a separate table that contains all modifications from the beginning of time, which Lakeflow SDP maintains automatically. This table supports data layout optimizations such as liquid clustering and handles out-of-order records automatically based on `_sequence_by`. See Use liquid clustering for tables.

To create an SCD2 table, use the option `STORED AS SCD TYPE 2` in SQL or `stored_as_scd_type="2"` in Python.

> **NOTE**
>
> You can also limit which columns the feature tracks using the option: `TRACK HISTORY ON {columnList | EXCEPT(exceptColumnList)}`

1. From the pipeline asset browser sidebar, click + **Add** and **Transformation**.
2. Enter a **Name** and choose a language (Python or SQL) for the new source code file.

3. Copy and paste the following code into the new file.

Python SQL

Python

```python
from pyspark import pipelines as dp
from pyspark.sql.functions import *

# create the table
dp.create_streaming_table(
    name="customers_history", comment="Slowly Changing Dimension Type
2 for customers"
)

# store all changes as SCD2
dp.create_auto_cdc_flow(
    target="customers_history",
    source="customers_cdc_clean",
    keys=["id"],
    sequence_by=col("operation_date"),
    ignore_null_updates=False,
    apply_as_deletes=expr("operation = 'DELETE'"),
    except_column_list=["operation", "operation_date",
"_rescued_data"],
    stored_as_scd_type="2",
)  # Enable SCD2 and store individual updates
```

4. Click ▶ **Run file** to start an update for the connected pipeline.

When the update is complete, the pipeline graph includes the new `customers_history` table, also dependent on the silver layer table, and the bottom panel shows the details for all 4 tables.

Step 7: Create a materialized view that tracks who has changed their information the most

The table `customers_history` contains all historical changes a user has made to their information. Create a simple materialized view in the gold layer that keeps track of who has changed their information the most. This could be used for fraud detection analysis or user recommendations in a real-world scenario. Additionally, applying changes with SCD2 has already removed duplicates, so you can directly count the rows per user ID.

1. From the pipeline asset browser sidebar, click + **Add** and **Transformation**.
2. Enter a **Name** and choose a language (Python or SQL) for the new source code file.
3. Copy and paste the following code into the new source file.

Python SQL

Python

```python
from pyspark import pipelines as dp
from pyspark.sql.functions import *

@dp.table(
  name = "customers_history_agg",
  comment = "Aggregated customer history"
)
def customers_history_agg():
  return (
    spark.read.table("customers_history")
      .groupBy("id")
      .agg(
          count_distinct("address").alias("address_count"),
          count_distinct("email").alias("email_count"),
          count_distinct("firstname").alias("firstname_count"),
          count_distinct("lastname").alias("lastname_count")
      )
  )
```

4. Click ▶ **Run file** to start an update for the connected pipeline.

After the update is complete, there is a new table in the pipeline graph that depends on the `customers_history` table, and you can view it in the bottom panel. Your pipeline

is now complete. You can test it by performing a full **Run pipeline**. The only steps left are to schedule the pipeline to update regularly.

Step 8: Create a job to run the ETL pipeline

Next, create a workflow to automate the data ingestion, processing, and analysis steps in your pipeline using a Databricks job.

1. At the top of the editor, choose the **Schedule** button.
2. If the **Schedules** dialog appears, choose **Add schedule**.
3. This opens the **New schedule** dialog, where you can create a job to run your pipeline on a schedule.
4. Optionally, give the job a name.
5. By default, the schedule is set to run once per day. You can accept this default, or set your own schedule. Choosing **Advanced** gives you the option to set a specific time that the job will run. Selecting **More options** allows you to create notifications when the job runs.
6. Select **Create** to apply the changes and create the job.

Now the job will run daily to keep your pipeline up to date. You can choose **Schedule** again to view the list of schedules. You can manage schedules for your pipeline from that dialog, including adding, editing, or removing schedules.

Clicking the name of the schedule (or job) takes you to the job's page in the **Jobs & pipelines** list. From there you can view details about job runs, including the history of runs, or run the job immediately with the **Run now** button.

See Monitoring and observability for Lakeflow Jobs for more information about job runs.

Additional resources

- Lakeflow Spark Declarative Pipelines
- Tutorial: Build an ETL pipeline with Lakeflow Spark Declarative Pipelines

- Change data capture and snapshots
- The AUTO CDC APIs: Simplify change data capture with pipelines
- Convert a pipeline into a bundle project
- What is Auto Loader?

ı this page

ıs this page helpful?

Yes | No

Send feedback

Ask Genie

*Last updated on **May 4, 2026***

Tutorial: Build a geospatial pipeline with native spatial types

Learn how to create and deploy a pipeline that ingests GPS data, converts coordinates to native spatial types, and joins against warehouse geofences to track arrivals using Lakeflow Spark Declarative Pipelines (SDP) for data orchestration and Auto Loader. This tutorial uses Databricks native spatial types (`GEOMETRY`, `GEOGRAPHY`) and built-in spatial functions such as `ST_Point`, `ST_GeomFromWKT`, and `ST_Contains`, so you can run geospatial workflows at scale without external libraries.

In this tutorial, you will:

- Create a pipeline and generate sample GPS and geofence data in a Unity Catalog volume.
- Ingest raw GPS pings incrementally with Auto Loader into a bronze streaming table.
- Build a silver streaming table that converts latitude and longitude to a native `GEOMETRY` point.
- Create a materialized view of warehouse geofences from WKT polygons.
- Run a spatial join to produce a table of warehouse arrivals (which device entered which geofence).

The result is a medallion-style pipeline: bronze (raw GPS), silver (points as geometry), and gold (geofences and arrival events). See What is the medallion lakehouse architecture? for more information.

Requirements

To complete this tutorial, you must meet the following requirements:

- Be logged in to a Databricks workspace.
- Have Unity Catalog enabled for your workspace.
- Have serverless compute available in your workspace if you want to use Serverless Lakeflow Spark Declarative Pipelines (enabled by default in workspaces with Unity Catalog). If serverless compute is not available, the steps work with the default compute for your workspace.
- Have permission to create a compute resource or access to a compute resource.
- Have permissions to create a new schema in a catalog. The required permissions are `USE CATALOG` and `CREATE SCHEMA`.
- Have permissions to create a new volume in an existing schema. The required permissions are `USE SCHEMA` and `CREATE VOLUME`.
- Use a runtime that supports native spatial types and spatial functions.

Step 1: Create a pipeline

Create a new ETL pipeline and set the default catalog and schema for your tables.

1. In your workspace, click + **New** in the sidebar, then select **ETL Pipeline**. This opens the pipeline editor with a default pipeline name like `New Pipeline <date> <time>`.

2. Select the name and enter a descriptive name, such as `Spatial pipeline tutorial`.

3. To the right of the name, click the catalog and schema to choose defaults for which you have write permissions.

   This catalog and schema are used by default when you do not specify a catalog or schema in your code. Replace `<catalog>` and `<schema>` in the following steps with the values you choose here.

4. (Optional) In the `my_transformation` source file created for you, select **Python** or **SQL** from the language drop-down list to set the language for the file.

5. Click ‹› **Use sample code**.

The Lakeflow Pipelines Editor opens with sample files in your pipeline. Next, create the sample GPS and geofence data.

Step 2: Create the sample GPS and geofence data

This step generates sample data in a volume: raw GPS pings (JSON) and warehouse geofences (JSON with WKT polygons). The GPS points are generated in a bounding box that overlaps the two warehouse polygons, so the spatial join in a later step will return arrival rows. You can skip this step if you already have your own data in a volume or table.

1. In the Lakeflow Pipelines Editor, in the asset browser, click + **Add**, then **Exploration**.
2. Set **Name** to `Setup spatial data`, choose **Python**, and leave the default destination folder.
3. Click **Create**.
4. In the new notebook, paste the following code. Replace `<catalog>` and `<schema>` with the default catalog and schema you set in Step 1.

   Use the following code in the notebook to generate GPS and geofence data.

   Python

```python
from pyspark.sql import functions as F

catalog = "<catalog>"   # for example, "main"
schema = "<schema>"    # for example, "default"

spark.sql(f"USE CATALOG `{catalog}`")
spark.sql(f"USE SCHEMA `{schema}`")
spark.sql(f"CREATE VOLUME IF NOT EXISTS
`{catalog}`.`{schema}`.`raw_data`")
volume_base = f"/Volumes/{catalog}/{schema}/raw_data"

# GPS: 5000 rows in a box that overlaps both warehouse geofences (LA
area)
```

```
gps_path = f"{volume_base}/gps"
df_gps = (
    spark.range(0, 5000)
    .repartition(10)
    .select(
        F.format_string("device_%d",
F.col("id").cast("long")).alias("device_id"),
        F.current_timestamp().alias("timestamp"),
        (-118.3 + F.rand() * 0.2).alias("longitude"),   # -118.3 to
-118.1
        (34.0 + F.rand() * 0.2).alias("latitude"),     # 34.0 to 34.2
    )
)
df_gps.write.format("json").mode("overwrite").save(gps_path)
print(f"Wrote 5000 GPS rows to {gps_path}")

# Geofences: two warehouse polygons (WKT) in the same region
geofences_path = f"{volume_base}/geofences"
geofences_data = [
    ("Warehouse_A", "POLYGON ((-118.35 34.02, -118.25 34.02, -118.25
34.08, -118.35 34.08, -118.35 34.02))"),
    ("Warehouse_B", "POLYGON ((-118.20 34.05, -118.12 34.05, -118.12
34.12, -118.20 34.12, -118.20 34.05))"),
]
df_geo = spark.createDataFrame(geofences_data, ["warehouse_name",
"boundary_wkt"])
df_geo.write.format("json").mode("overwrite").save(geofences_path)
print(f"Wrote {len(geofences_data)} geofences to {geofences_path}")
```

5. Run the notebook cell (Shift + Enter).

After the run completes, the volume contains `gps` (raw pings) and `geofences` (polygons in WKT). In the next step, you ingest the GPS data into a bronze table.

Step 3: Ingest GPS data into a bronze streaming table

Ingest the raw GPS JSON from the volume incrementally using Auto Loader and write into a bronze streaming table.

1. In the asset browser, click + **Add**, then **Transformation**.
2. Set **Name** to `gps_bronze`, choose **SQL** or **Python**, and click **Create**.

3. Replace the file contents with the following (use the tab that matches your language). Replace `<catalog>` and `<schema>` with your default catalog and schema.

SQL Python

SQL

```
CREATE OR REFRESH STREAMING TABLE gps_bronze
COMMENT "Raw GPS pings ingested from volume using Auto Loader";

CREATE FLOW gps_bronze_ingest_flow AS
INSERT INTO gps_bronze BY NAME
SELECT *
FROM STREAM read_files(
  "/Volumes/<catalog>/<schema>/raw_data/gps",
  format => "json",
  inferColumnTypes => "true"
)
```

4. Click ▶ **Run file** or **Run pipeline** to run an update.

When the update completes, the pipeline graph shows the `gps_bronze` table. Next, add a silver table that converts coordinates to a native geometry point.

Step 4: Add a silver streaming table with geometry points

Create a streaming table that reads from the bronze table and adds a `GEOMETRY` column using `ST_Point(longitude, latitude)`.

1. In the asset browser, click + **Add**, then **Transformation**.
2. Set **Name** to `raw_gps_silver`, choose **SQL** or **Python**, and click **Create**.
3. Paste the following code into the new file.

SQL Python

SQL

```
CREATE OR REFRESH STREAMING TABLE raw_gps_silver
COMMENT "GPS pings with native geometry point for spatial joins";

CREATE FLOW raw_gps_silver_flow AS
INSERT INTO raw_gps_silver BY NAME
SELECT
  device_id,
  timestamp,
  longitude,
  latitude,
  ST_Point(longitude, latitude) AS point_geom
FROM STREAM(gps_bronze)
```

4. Click ▶ **Run file** or **Run pipeline**.

The pipeline graph now shows `gps_bronze` and `raw_gps_silver`. Next, add the warehouse geofences as a materialized view.

Step 5: Create the warehouse geofences gold table

Create a materialized view that reads the geofences from the volume and converts the WKT column to a `GEOMETRY` column using `ST_GeomFromWKT`.

1. In the asset browser, click + **Add**, then **Transformation**.
2. Set **Name** to `warehouse_geofences_gold`, choose **SQL** or **Python**, and click **Create**.
3. Paste the following code. Replace `<catalog>` and `<schema>` with your default catalog and schema.

SQL Python

Ask Genie

SQL

```
CREATE OR REPLACE MATERIALIZED VIEW warehouse_geofences_gold AS
SELECT
  warehouse_name,
  ST_GeomFromWKT(boundary_wkt) AS boundary_geom
FROM read_files(
  "/Volumes/<catalog>/<schema>/raw_data/geofences",
  format => "json"
)
```

4. Click ▶ **Run file** or **Run pipeline**.

The pipeline now includes the geofences table. Next, add the spatial join to compute warehouse arrivals.

Step 6: Create the warehouse arrivals table with a spatial join

Add a materialized view that joins the silver GPS points to the geofences using `ST_Contains(boundary_geom, point_geom)` to determine when a device is inside a warehouse polygon.

1. In the asset browser, click + **Add**, then **Transformation**.
2. Set **Name** to `warehouse_arrivals`, choose **SQL** or **Python**, and click **Create**.
3. Paste the following code.

SQL Python

SQL

```
CREATE OR REPLACE MATERIALIZED VIEW warehouse_arrivals AS
SELECT
  g.device_id,
  g.timestamp,
  w.warehouse_name
```

```
FROM raw_gps_silver g
JOIN warehouse_geofences_gold w
  ON ST_Contains(w.boundary_geom, g.point_geom)
```

4. Click ▶ **Run file** or **Run pipeline**.

When the update completes, the pipeline graph shows all four datasets: `gps_bronze`, `raw_gps_silver`, `warehouse_geofences_gold`, and `warehouse_arrivals`.

Verify the spatial join

Confirm that the spatial join produced rows: points from the silver table that fall inside a geofence appear in `warehouse_arrivals`. Run one of the following in a notebook or SQL editor (use the same catalog and schema as your pipeline target).

Count arrivals by warehouse (SQL):

SQL

```
SELECT warehouse_name, COUNT(*) AS arrival_count
FROM warehouse_arrivals
GROUP BY warehouse_name
ORDER BY warehouse_name;
```

You should see non-zero counts for `Warehouse_A` and `Warehouse_B` (the sample GPS data overlaps both polygons). To inspect sample rows:

SQL

```
SELECT device_id, timestamp, warehouse_name
FROM warehouse_arrivals
ORDER BY timestamp DESC
LIMIT 10;
```

Same checks in Python (notebook):

Python

```
# Count by warehouse
display(spark.table("warehouse_arrivals").groupBy("warehouse_name").count().

# Sample rows
display(spark.table("warehouse_arrivals").orderBy("timestamp", ascending=Fal
```

If you see rows in `warehouse_arrivals`, the `ST_Contains(boundary_geom, point_geom)` join is working correctly.

Step 7: Schedule the pipeline (optional)

To keep the pipeline up to date as new GPS data lands in the volume, create a job to run the pipeline on a schedule.

1. At the top of the editor, choose the **Schedule** button.
2. If the **Schedules** dialog appears, choose **Add schedule**.
3. Optionally, give the job a name.
4. By default, the schedule runs once per day. You can accept this or set your own. Choosing **Advanced** lets you set a specific time; **More options** lets you add run notifications.
5. Select **Create** to apply the schedule.

See Monitoring and observability for Lakeflow Jobs for more information about job runs.

Additional resources

- Lakeflow Spark Declarative Pipelines
- Lakeflow Spark Declarative Pipelines concepts
- Streaming tables
- Load data in pipelines
- `GEOMETRY` type
- ST geospatial functions

- What is Auto Loader?

this page

as this page helpful?

Yes | No

Send feedback

Ask Genie

*Last updated on **May 12, 2026***

Create a source-controlled pipeline

In Databricks, you can source control a pipeline and all of the code associated with it. By source controlling all files associated with your pipeline, changes to your transformation code, exploration code, and pipeline configuration are all versioned in Git and can be tested in development and confidently deployed to production.

A source-controlled pipeline offers the following advantages:

- **Traceability**: Capture every change in Git history.
- **Testing**: Validate pipeline changes in a development workspace before promoting to a shared production workspace. Every developer has their own development pipeline on their own code branch in a Git folder and in their own schema.
- **Collaboration**: When individual development and testing is finished, code changes are pushed to the main production pipeline.
- **Governance**: Align with enterprise CI/CD and deployment standards.

Databricks allows for pipelines and their source files to be source-controlled together using Declarative Automation Bundles. With bundles, pipeline configuration is source-controlled in the form of YAML configuration files alongside the Python or SQL source files of a pipeline. One bundle might have one or many pipelines, as well as other resource types, such as jobs.

This page shows how to set up a source-controlled pipeline using Declarative Automation Bundles (formerly known as Databricks Asset Bundles). For more information about bundles, see What are Declarative Automation Bundles?.

Requirements

To create a source-controlled pipeline, you must already have:

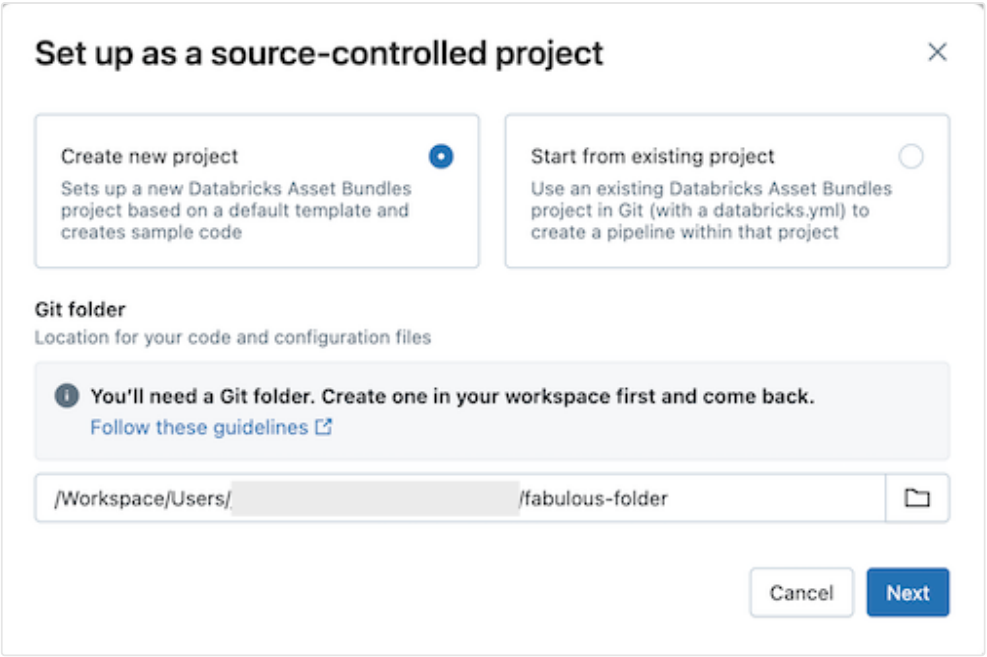
- A Git folder created in your workspace and configured. A Git folder allows individual users to author and test changes before committing them to a Git repository. See Databricks Git folders.
- The Lakeflow Pipelines Editor. See Develop and debug ETL pipelines with the Lakeflow Pipelines Editor for more information.

Create a new pipeline in a bundle

> **NOTE**
>
> Databricks recommends creating a pipeline that is source-controlled from the start. Alternatively, you can add an existing pipeline to a bundle that is already source-controlled. See Migrate existing resources to a bundle.

To create a new source-controlled pipeline:

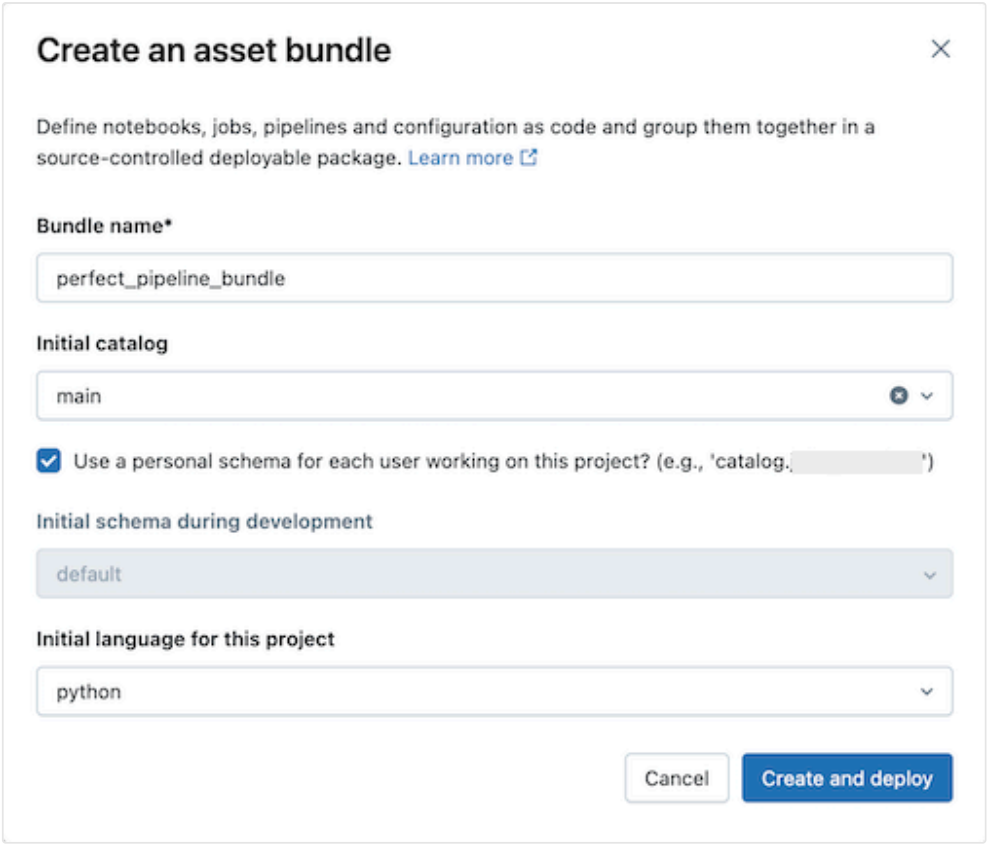
1. At the top of the sidebar, click + **New** and then select **ETL pipeline**.
2. Make any changes that you want to the pipeline name or schema. See Create a new ETL pipeline.
3. Click the ⋮ menu (to the right of the **Use sample code** button) and select **Set up as source-controlled**.
4. Click **Create new project**, then select a Git folder where you want to put your code and configuration:



5. Click **Next**.

6. Enter the following in the **Create an asset bundle** dialog:

   - **Bundle name**: The name of the bundle.
   - **Initial catalog**: The name of the catalog that contains the schema to use.
   - **Use a personal schema**: Leave this box checked if you want to isolate edits to a personal schema, so that when users in your organization collaborate on the same project you don't overwrite each other's changes in dev.
   - **Initial language**: The initial language to use for the project's sample pipeline files, either Python or SQL.

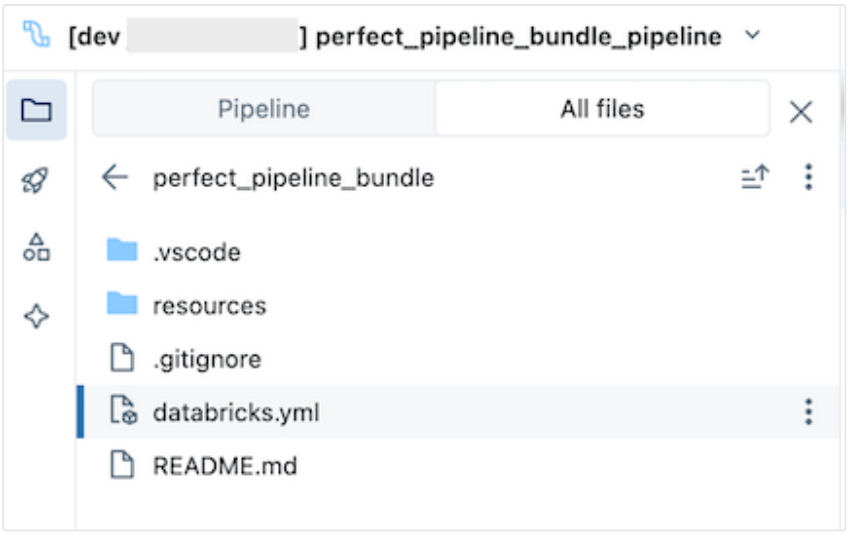


7. Click **Create and deploy**. A bundle with a pipeline is created in the Git folder.

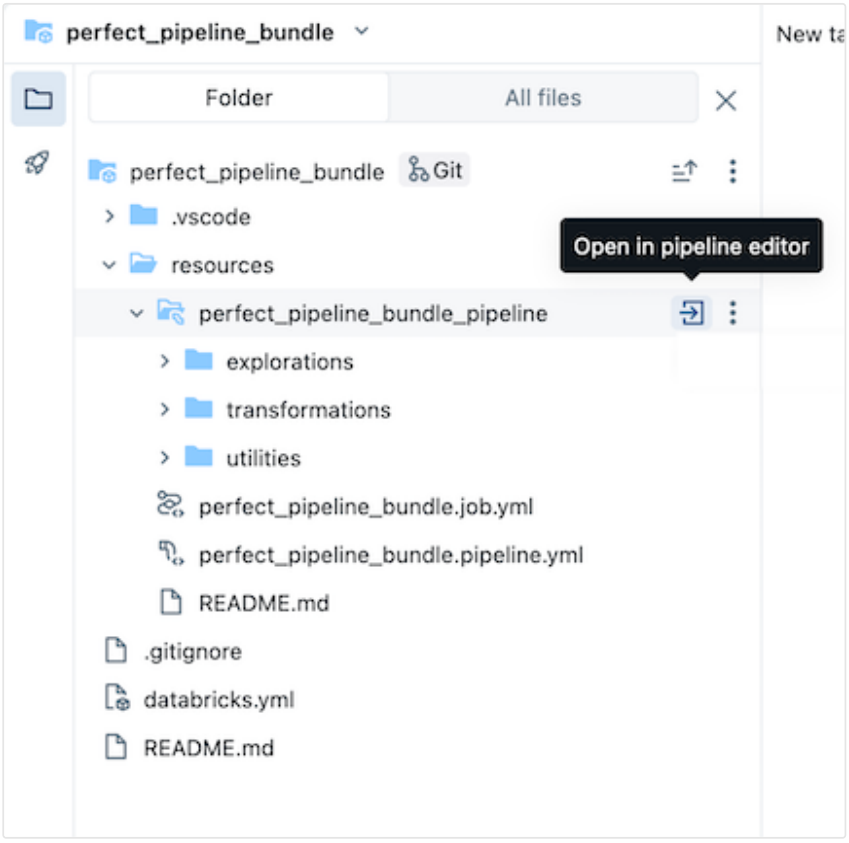
Explore the pipeline bundle

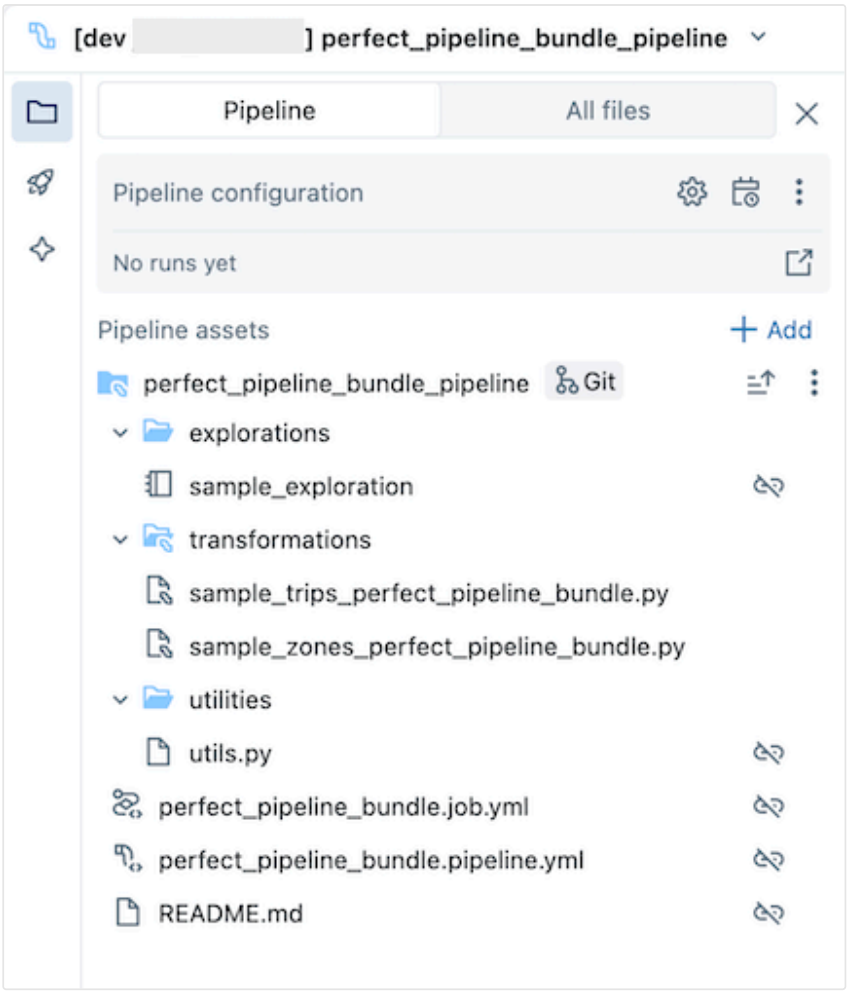
Next, explore the pipeline bundle that was created.

The bundle, which is in the Git folder, contains bundle system files and the `databricks.yml` file, which defines variables, target workspace URLs and permissions, and other settings for the bundle. Because `databricks.yml` lives in the bundle root (the parent of the pipeline root), switch to the **All files** tab in the pipeline asset browser to see it. The `resources` folder of a bundle is where definitions for resources such as pipelines and jobs are contained.



Open the `resources` folder, then click the pipeline editor button to view the source-controlled pipeline:





The sample pipeline bundle includes the following files:

- A sample exploration notebook
- Two sample code files that do transformations on tables
- A sample code file that contains a utility function
- A job configuration YAML file that defines the job in the bundle that runs the pipeline
- A pipeline configuration YAML file that defines the pipeline

> **IMPORTANT**
>
> You must edit this file to permanently persist any configuration changes to the pipeline, including changes made through the UI, otherwise UI changes are overridden when the bundle is redeployed. For example, to set a different

> default catalog for the pipeline, edit the `catalog` field in this configuration file.

- A README file with additional details about the sample pipeline bundle and instructions on how to run the pipeline

For information about pipeline files, see Pipeline asset browser.

For more information about authoring and deploying changes to the pipeline bundle, see Author bundles in the workspace and Deploy bundles and run workflows from the workspace.

Run the pipeline

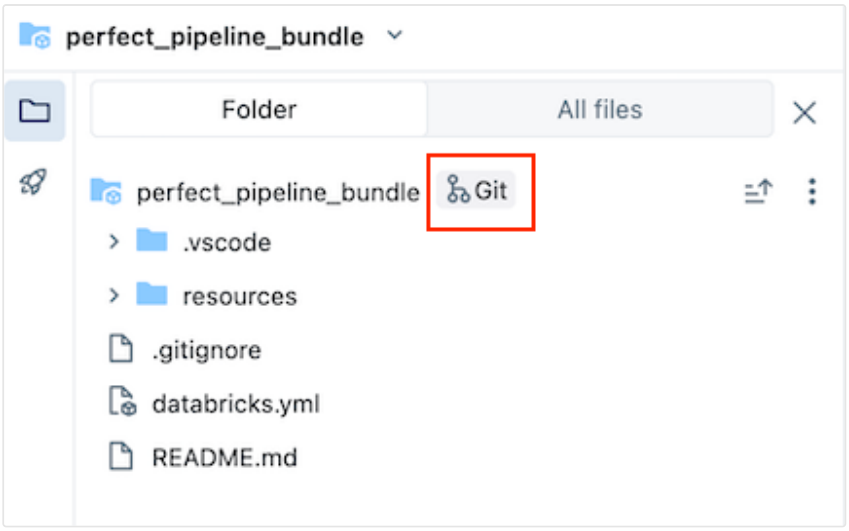
You can run either individual transformations or the entire source-controlled pipeline:

- To run and preview a single transformation in the pipeline, select the transformation file in the workspace browser tree to open it in the file editor. At the top of the file in the editor, click the **Run file** play button.
- To run all transformations in the pipeline, click the **Run pipeline** button in the upper right of the Databricks workspace.

For more information about running pipelines, see Run pipeline code.

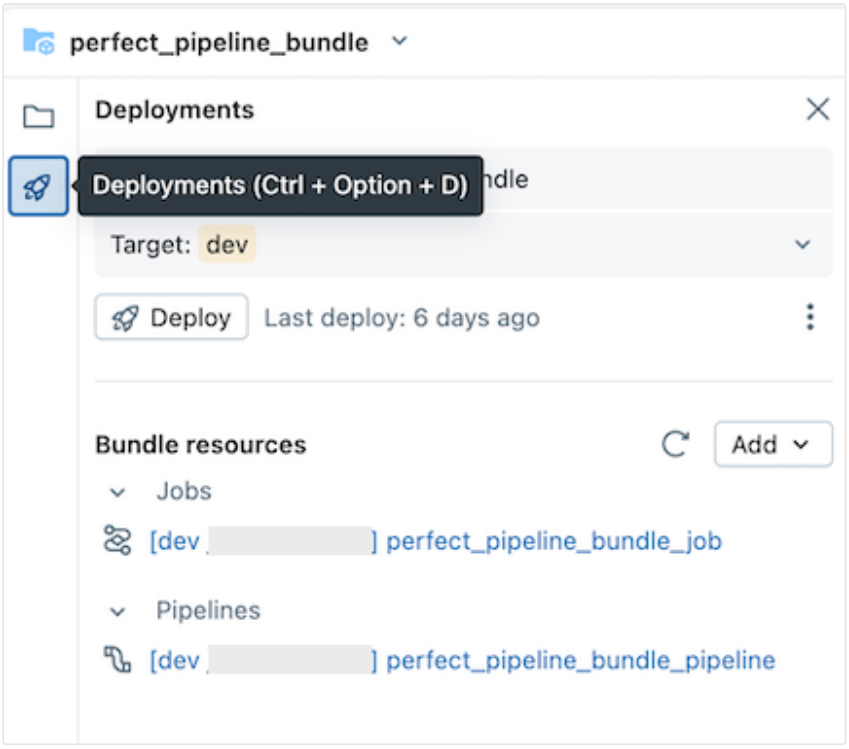
Update the pipeline

You can update artifacts in your pipeline or add additional explorations and transformations, but then you will want to push those changes to GitHub. Click the **Git** icon associated with the pipeline bundle or click the kebab for the folder then **Git...** to select which changes to push. See Commit and push changes.



In addition, when you update pipeline configuration files or add or remove files from the bundle, these changes are not propagated to the target workspace until you explicitly deploy the bundle. See Deploy bundles and run workflows from the workspace.

> **NOTE**
>
> Databricks recommends that you keep the default setup for source-controlled pipelines. The default setup is configured so that you do not need to edit the pipeline bundle YAML configuration when additional files are added through the UI.



Add an existing pipeline to a bundle

To add an existing pipeline to a bundle, first create a bundle in the workspace, then add the pipeline YAML definition to the bundle, as described in the following pages:

- Tutorial: Create and deploy a bundle in the workspace
- Add an existing resource to a bundle

For information on how to migrate resources to a bundle using the Databricks CLI, see Migrate existing resources to a bundle.

Additional resources

For additional tutorials and reference material for pipelines, see Lakeflow Spark Declarative Pipelines.

*Last updated on **Mar 16, 2026***

Convert a pipeline into a bundle project

This article shows how to convert an existing pipeline into a Declarative Automation Bundles project. Bundles enable you to define and manage your Databricks data processing configuration in a single, source-controlled YAML file that provides easier maintenance and enables automated deployment to target environments.

For a tutorial that uses `databricks pipelines` commands to create a pipelines project, then deploys and runs a pipeline, see Develop pipelines with Declarative Automation Bundles.

Conversion process overview

The steps you take to convert an existing pipeline to a bundle are:

1. Make sure you have access to a previously configured pipeline you want to convert to a bundle.
2. Create or prepare a folder (preferably in a source-controlled hierarchy) to store the bundle.
3. Generate a configuration for the bundle from the existing pipeline, using the Databricks CLI.
4. Review the generated bundle configuration to ensure it is complete.
5. Link the bundle to the original pipeline.
6. Deploy the pipeline to a target workspace using the bundle config.

Requirements

Before you start, you must have:

- The Databricks CLI installed on your local development machine. Databricks CLI version 0.218.0 or above is required to use Declarative Automation Bundles.
- The ID of an existing declarative pipeline you will manage with a bundle. To learn how you obtain this ID, see Get an existing pipeline definition using the UI.
- Authorization for the Databricks workspace where the existing pipeline runs. To configure authentication and authorization for your Databricks CLI calls, see Authorize access to Databricks resources.

Step 1: Set up a folder for your bundle project

You must have access to a Git repository that is configured in Databricks as a Git folder. You will create your bundle project in this repository, which will apply source control and make it available to other collaborators through a Git folder in the corresponding Databricks workspace. (For more details on Git folders, see Databricks Git folders.)

1. Go to the root of the cloned Git repository on your local machine.
2. At an appropriate place in the folder hierarchy, create a folder specifically for your bundle project. For example:

   Bash

   ```
   mkdir -p ~/source/my-pipelines/ingestion/events/my-bundle
   ```

3. Change your current working directory to this new folder. For example:

   Bash

   ```
   cd ~/source/my-pipelines/ingestion/events/my-bundle
   ```

   Ask Genie

4. Initialize a new bundle by running:

```
Bash

databricks bundle init
```

Answer the prompts. Once it completes, you will have a project configuration file named `databricks.yml` in the new home folder for your project. This file is required for deploying your pipeline from the command line. For more details on this configuration file, see Declarative Automation Bundles configuration.

Step 2: Generate the pipeline configuration

From this new directory in your cloned Git repository's folder tree, run the Databricks CLI bundle generate command, providing the ID of your pipeline as `<pipeline-id>`:

```
Bash

databricks bundle generate pipeline --existing-pipeline-id <pipeline-id>
--profile <profile-name>
```

When you run the `generate` command, it creates a bundle configuration file for your pipeline in the bundle's `resources` folder and downloads any referenced artifacts to the `src` folder. The `--profile` (or `-p` flag) is optional, but if you have a specific Databricks configuration profile (defined in your `.databrickscfg` file created when you installed the Databricks CLI) that you'd rather use instead of the default profile, provide it in this command. For information about Databricks configuration profiles, see Databricks configuration profiles.

> **TIP**
>
> If you have an existing Spark Declarative Pipelines (SDP) project (it has a `spark-pipeline.yml` file), you can copy that pipeline project to the `src` folder of the bundle, then use the `databricks pipelines generate` command to generate bundle configuration for it. See databricks pipelines generate.

Step 3: Review the bundle project files

When the `bundle generate` command completes, it will have created two new folders:

- `resources` is the project subdirectory that contains project configuration files.
- `src` is the project folder where source files, such as queries and notebooks, are stored.

The command also creates some additional files:

- `*.pipeline.yml` under the `resources` subdirectory. This file contains the specific configuration and settings for your pipeline.
- Source files such as SQL queries under the `src` subdirectory, copied from your existing pipeline.

```
├── databricks.yml                          # Project configuration
file created with the bundle init command
├── resources/
│   └── {your-pipeline-name.pipeline}.yml    # Pipeline configuration
└── src/
    └── {source folders and files...}        # Your pipeline's
declarative queries
```

Step 4: Bind the bundle pipeline to your existing pipeline

You must link, or *bind*, the pipeline definition in the bundle to your existing pipeline in order to keep it up to date as you make changes. To do this, run the Databricks CLI bundle deployment bind command:

Bash

```
databricks bundle deployment bind <pipeline-name> <pipeline-ID> --profile
<profile-name>
```

`<pipeline-name>` is the name of the pipeline. This name should be the same as the prefixed string value of the file name for the pipeline configuration in your new

`resources` directory. For example, if you have a pipeline configuration file named `ingestion_data_pipeline.pipeline.yml` in your `resources` folder, then you must provide `ingestion_data_pipeline` as your pipeline name.

`<pipeline-ID>` is the ID for your pipeline. It is the same as the one you copied as part of the requirements for these instructions.

Step 5: Deploy your pipeline using your new bundle

Now, deploy your pipeline bundle to your target workspace using the Databricks CLI bundle deploy command:

Bash

```
databricks bundle deploy --target <target-name> --profile <profile-name>
```

The `--target` flag is required and must be set to a string that matches a configured target workspace name, such as `development` or `production`.

If this command is successful, you now have your pipeline configuration in an external project that can be loaded into other workspaces and run, and easily shared with other Databricks users in your account.

Troubleshooting

Issue	Solution
"`databricks.yml` not found" error when running `bundle generate`	Currently, the `bundle generate` command doesn't create the bundle configuration file (`databricks.yml`) automatically. You must create the file using `databricks bundle init` or manually.

Issue	Solution
Existing pipeline settings don't match the values in the generated pipeline YAML configuration	The pipeline ID does not appear in the bundle configuration YML file. If you notice any other missing settings, you can manually apply them.

Tips for success

- Always use version control. If you aren't using Databricks Git folders, store your project subdirectories and files in a Git or other version-controlled repository or file system.
- Test your pipeline in a non-production environment (such as a “development” or “test” environment) before deploying it to a production environment. It's easy to introduce a misconfiguration by accident.

Additional resources

For more information about using bundles to define and manage data processing, see:

- What are Declarative Automation Bundles?
- Develop pipelines with Declarative Automation Bundles. This topic covers creating a bundle for a new pipeline rather than an existing one, with version-controlled source files for processing that you provide.

Last updated on ***Apr 29, 2026***

Develop Lakeflow Spark Declarative Pipelines

Developing and testing pipeline code differs from other Apache Spark workloads. This article provides an overview of supported functionality, best practices, and considerations when developing pipeline code. For more recommendations and best practices, see Applying software development & DevOps best practices to pipelines.

> **NOTE**
>
> You must add source code to a pipeline configuration to validate code or run an update. See Configure Pipelines.

What files are valid for pipeline source code?

Pipeline code can be Python or SQL. You can have a mix of Python and SQL source code files backing a single pipeline, but each file can only contain one language. See Develop pipeline code with Python and Develop Lakeflow Spark Declarative Pipelines code with SQL.

Source files for pipelines are stored in your workspace. Workspace files represent Python or SQL scripts authored in the Lakeflow Pipelines Editor. You can also edit the files locally in your preferred IDE, and sync to the workspace. For information about files in the workspace, see What are workspace files?. For information about editing with the Lakeflow Pipelines Editor, see Develop and debug ETL pipelines with the Lakeflow Pipelines Editor. For information about authoring code in a local IDE, see Develop pipeline code in your local development environment.

If you develop Python code as modules or libraries, you must install and import the code and then call methods from a Python file configured as source code. See Manage Python dependencies for pipelines.

> **NOTE**
>
> If you need to use arbitrary SQL commands in a Python notebook, you can use the syntax pattern `spark.sql("<QUERY>")` to run SQL as Python code.
>
> Unity Catalog functions allow you to register arbitrary Python user-defined functions for use in SQL. See User-defined functions (UDFs) in Unity Catalog.

Overview of pipeline development features

Pipelines extend and leverages many Databricks development features, and introduce new features and concepts. The following table provides a brief overview of concepts and features that support pipeline code development:

Feature	Description
Dry run	A **Dry run** update verifies the correctness of pipeline source code without running an update on any tables. See Check a pipeline for errors without waiting for tables to update.
Lakeflow Pipelines Editor	Python and SQL files configured as source code for pipelines provide interactive options for validating code and running updates. See Develop and debug ETL pipelines with the Lakeflow Pipelines Editor.
Parameters	Leverage parameters in source code and pipeline configurations to simplify testing and extensibility. See Use parameters with pipelines.
Declarative Automation	Declarative Automation Bundles allow you to move pipeline configurations and source code between workspaces. See

Feature	Description
Bundles	Convert a pipeline into a bundle project.

Create sample datasets for development and testing

Databricks recommends creating development and test datasets to test pipeline logic with expected data and potentially malformed or corrupt records. There are multiple ways to create datasets that can be useful for development and testing, including the following:

- Select a subset of data from a production dataset.
- Use anonymized or artificially generated data for sources containing PII. To see a tutorial that uses the `faker` library to generate data for testing, see Tutorial: Build an ETL pipeline using change data capture.
- Create test data with well-defined outcomes based on downstream transformation logic.
- Anticipate potential data corruption, malformed records, and upstream data changes by creating records that break data schema expectations.

For example, if you have a file that defines a dataset using the following code:

SQL

```
CREATE OR REFRESH STREAMING TABLE input_data
AS SELECT * FROM STREAM read_files(
  "/production/data",
  format => "json")
```

You could create a sample dataset containing a subset of the records using a query like the following:

SQL

```
CREATE OR REFRESH MATERIALIZED VIEW input_data AS
SELECT "2021/09/04" AS date, 22.4 as sensor_reading UNION ALL
SELECT "2021/09/05" AS date, 21.5 as sensor_reading
```

The following example demonstrates filtering published data to create a subset of the production data for development or testing:

SQL

```
CREATE OR REFRESH MATERIALIZED VIEW input_data AS SELECT * FROM
prod.input_data WHERE date > current_date() - INTERVAL 1 DAY
```

To use these different datasets, create multiple pipelines with the source code implementing the transformation logic. Each pipeline can read data from the `input_data` dataset but is configured to include the file that creates the dataset specific to the environment.

How do pipeline datasets process data?

The following table describes how materialized views, streaming tables, and views process data:

Dataset type	How are records processed through defined queries?
Streaming table	Each record is processed exactly once. This assumes an append-only source.
Materialized view	Records are processed as required to return accurate results for the current data state. Materialized views should be used for data processing tasks such as transformations, aggregations, or pre-computing slow queries and frequently used computations. Results are cached between updates.

Dataset type	How are records processed through defined queries?
View	Records are processed each time the view is queried. Use views for intermediate transformations and data quality checks that should not be published to public datasets.

Declare your first datasets in pipelines

Pipelines introduce new syntax for Python and SQL. To learn the basics of pipeline syntax, see Develop pipeline code with Python and Develop Lakeflow Spark Declarative Pipelines code with SQL.

> **NOTE**
> Pipelines separate dataset definitions from update processing, and pipeline source is not intended for interactive execution.

How do you configure pipelines?

The settings for a pipeline fall into two broad categories:

1. Configurations that define a collection of files (known as *source code*) that use pipeline syntax to declare datasets.
2. Configurations that control pipeline infrastructure, dependency management, how updates are processed, and how tables are saved in the workspace.

Most configurations are optional, but some require careful attention, especially when configuring production pipelines. These include the following:

- To make data available outside the pipeline, you must declare a **target schema** to publish to the Hive metastore or a **target catalog** and **target schema** to publish to Unity Catalog.
- Data access permissions are configured through the cluster used for execution. Ensure your cluster has appropriate permissions configured for data sources and

the target **storage location**, if specified.

For details on using Python and SQL to write source code for pipelines, see Pipeline SQL language reference and Lakeflow Spark Declarative Pipelines Python language reference.

For more on pipeline settings and configurations, see Configure Pipelines.

Deploy your first pipeline and trigger updates

To process data with SDP, configure a pipeline. After a pipeline is configured, you can trigger an update to calculate results for each dataset in your pipeline. To get started using pipelines, see Tutorial: Build an ETL pipeline using change data capture.

What is a pipeline update?

Pipelines deploy infrastructure and recompute data state when you start an *update*. An update does the following:

- Starts a cluster with the correct configuration.
- Discovers all the tables and views defined and checks for any analysis errors such as invalid column names, missing dependencies, and syntax errors.
- Creates or updates tables and views with the most recent data available.

Pipelines can be run continuously or on a schedule depending on your use case's cost and latency requirements. See Run a pipeline update.

Ingest data with pipelines

Pipelines support all data sources available in Databricks.

Databricks recommends using streaming tables for most ingestion use cases. For files arriving in cloud object storage, Databricks recommends Auto Loader. You can directly ingest data with a pipeline from most message buses.

For more information about configuring access to cloud storage, see Cloud storage configuration.

For formats not supported by Auto Loader, you can use Python or SQL to query any format supported by Apache Spark. See Load data in pipelines.

Monitor and enforce data quality

You can use *expectations* to specify data quality controls on the contents of a dataset. Unlike a `CHECK` constraint in a traditional database which prevents adding any records that fail the constraint, expectations provide flexibility when processing data that fails data quality requirements. This flexibility allows you to process and store data that you expect to be messy and data that must meet strict quality requirements. See Manage data quality with pipeline expectations.

How are Lakeflow Spark Declarative Pipelines and Delta Lake related?

SDP extends the functionality of Delta Lake. Because tables created and managed by pipelines are Delta tables, they have the same guarantees and features provided by Delta Lake. See What is Delta Lake in Databricks?.

Pipelines add several table properties in addition to the many table properties that can be set in Delta Lake. See Pipeline properties reference and Table properties reference.

How tables are created and managed by pipelines

Databricks automatically manages tables created by pipelines, determining how updates need to be processed to correctly compute the current state of a table and performing a number of maintenance and optimization tasks.

For most operations, you should allow the pipeline to process all updates, inserts, and deletes to a target table. For details and limitations, see Retain manual deletes or updates.

Maintenance tasks performed by pipelines

Databricks performs maintenance tasks on tables managed by pipelines on an optimal cadence using predictive optimization. Maintenance can improve query performance and reduce cost by removing old versions of tables. This includes full OPTIMIZE and VACUUM operations. Maintenance tasks are performed on a schedule decided by predictive optimization, and only if a pipeline update has run since the previous maintenance.

To understand how often predictive optimization runs, and to understand maintenance costs, see Predictive optimization system table reference.

Limitations

For a list of limitations, see Pipeline Limitations.

For a list of requirements and limitations that are specific to using pipelines with Unity Catalog, see Use Unity Catalog with pipelines

Additional resources

- Pipelines have full support in the Databricks REST API. See Pipelines REST API.
- For pipeline and table settings, see Pipeline properties reference.
- Pipeline SQL language reference.
- Lakeflow Spark Declarative Pipelines Python language reference.

*Last updated on **May 12, 2026***

Develop and debug ETL pipelines with the Lakeflow Pipelines Editor

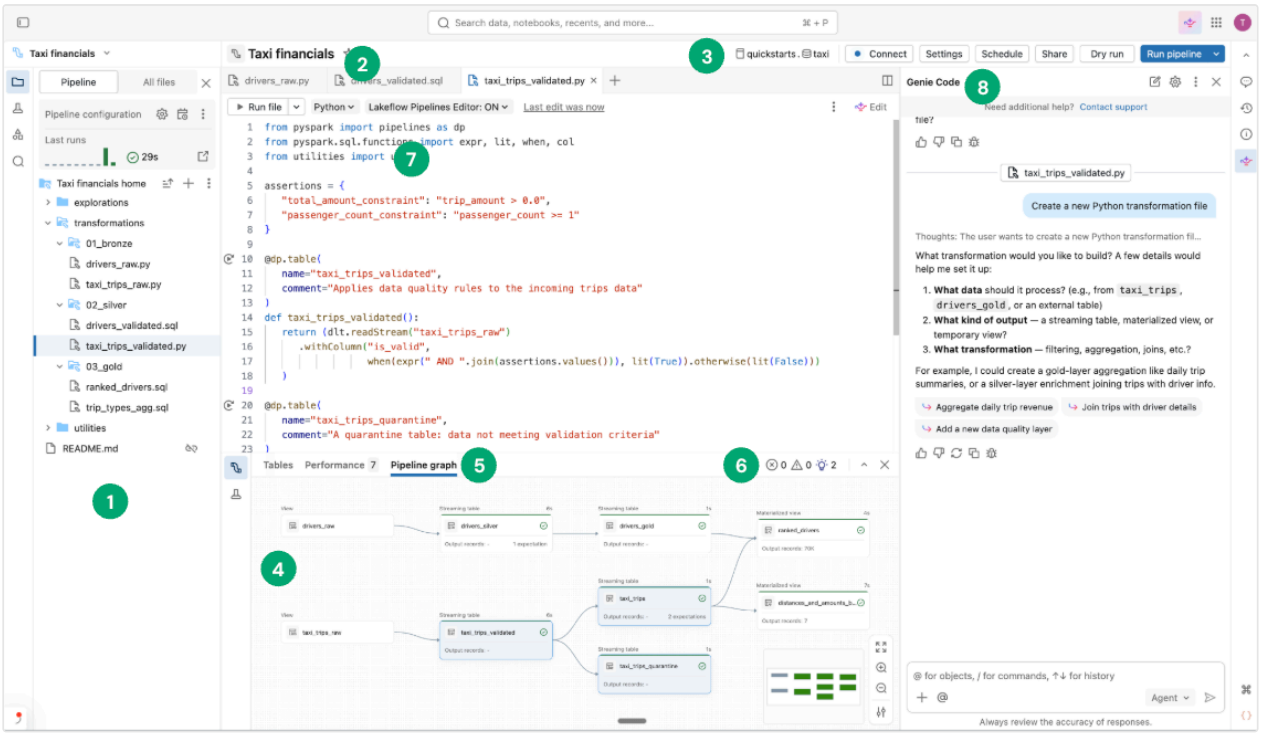
This article describes using the Lakeflow Pipelines Editor to develop and debug ETL (extract, transform, and load) pipelines in Lakeflow Spark Declarative Pipelines (SDP).

What is the Lakeflow Pipelines Editor?

The Lakeflow Pipelines Editor is an IDE built for developing pipelines. It combines all pipeline development tasks on a single surface, supporting code-first workflows, folder-based code organization, selective execution, data previews, and pipeline graphs. Integrated with the Databricks platform, it also enables version control, code reviews, and scheduled runs.

Overview of the Lakeflow Pipelines Editor UI

The following image shows the Lakeflow Pipelines Editor:



The image shows the following features:

1. **Pipeline asset browser**: Create, delete, rename, and organize pipeline assets. Also includes shortcuts to pipeline configuration.
2. **Multi-file code editor with tabs**: Work across multiple code files associated with a pipeline.
3. **Pipeline-specific toolbar**: Includes pipeline configuration options and has pipeline-level run actions.
4. **Interactive pipeline graph**: Get an overview of your tables, open the data previews bottom bar, and perform other table-related actions.
5. **Table-level execution insights**: Get execution insights for all tables or a single table in a pipeline. The insights refer to the latest pipeline run.
6. **Issues panel**: This feature summarizes errors, warnings, and insights across all files in the pipeline, and you can navigate to where the error occurred inside a specific file. It complements code-affixed error indicators.
7. **Selective execution**: The code editor has features for step-by-step development, such as the ability to refresh only the tables in the current file using the **Run file** action, or to refresh a single table.
8. **Genie Code**: Create, update, and debug your pipelines using Genie Code, an agentic experience that automates multi-step workflows from data discovery and code generation to pipeline execution and resolving data quality issues. Ask Genie

Other important features:

- **Data preview**: Inspect the data of your streaming tables and materialized views.
- **Default pipeline folder structure**: New pipelines include a predefined folder structure and sample code that you can use as a starting point for your pipeline.

Create a new ETL pipeline

To create a new ETL pipeline using the Lakeflow Pipelines Editor, follow these steps:

1. At the top of the sidebar, click + **New** and then select **ETL pipeline**.

   A pipeline is automatically created with the following default settings:

   - Unity Catalog
   - Current channel
   - Serverless compute

   You can adjust these settings from the pipeline toolbar.

2. At the top, give your pipeline a unique name.

3. Next to the name, the default catalog and schema chosen for you are displayed.

   The default catalog and the default schema are where datasets are read from or written to when you do not qualify datasets with a catalog or schema in your code. See Database objects in Databricks for more information.

   Click the catalog and schema to change the defaults for your pipeline.

4. Your pipeline has a blank `my_transformation` file by default. Switch this file between Python and SQL by choosing from the language drop-down list. Write code in this file directly, or choose one of the following options to get started quickly:

   - **Create with Genie Code**: Describe your pipeline using natural language and let Genie Code build it for you.
   - **Use sample code**: Create a default folder structure and sample code in the language of the current file.

For more advanced options, expand the ⋮ menu (to the right of the <> **Use sample code** button) to:

- **Add existing source code**: Associate your pipeline with code files already available in your workspace, including Git folders.
- **Set up as source controlled**: Use a Declarative Automation Bundles project for source control and CI/CD support.
- **Use Hive metastore**: Create a pipeline with legacy settings.

Alternatively, you can create an ETL pipeline from the workspace browser:

1. Click **Workspace** in the left side panel.
2. Select any folder, including Git folders.
3. Click **Create** in the upper-right corner, and click **ETL pipeline**.

You can also create an ETL pipeline from the jobs and pipelines page:

1. In your workspace, click **Jobs & Pipelines** in the sidebar.
2. Under **New**, click **ETL Pipeline**.

> TIP
>
> The Databricks CLI provides commands to create, modify, and manage your Lakeflow Spark Declarative Pipelines pipelines from a terminal. See `pipelines` command group.

Open an existing ETL pipeline

There are multiple ways to open an existing ETL pipeline in the Lakeflow Pipelines Editor:

- Open any source file associated with the pipeline:
  i. Click **Workspace** in the side panel.
  ii. Navigate to a folder with source code files for your pipeline.
  iii. Click the source code file to open the pipeline in the editor.
- Open a recently edited pipeline:

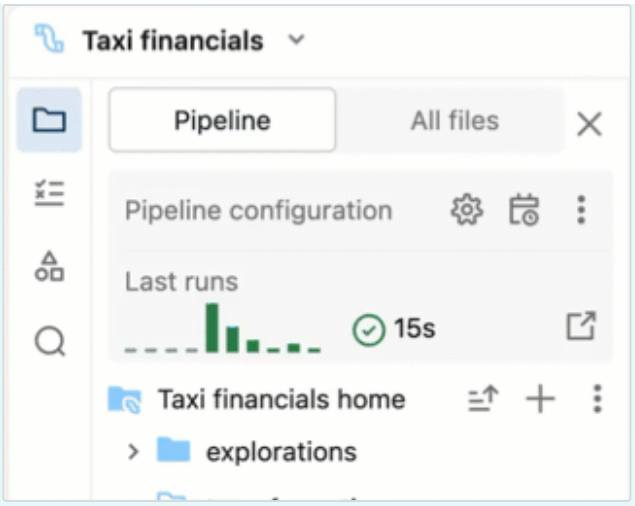
    - From the editor, you can navigate to other pipelines you have recently edited by clicking the name of the pipeline at the top of the asset browser and choosing another pipeline from the recents list that appears.
    - From outside the editor, from the **Recents** page on the left sidebar, open a pipeline or a file configured as the source code for a pipeline.

- When viewing a pipeline across the product, you can choose to edit the pipeline:
    - In the pipeline monitoring page, click **Edit pipeline**.
    - On the **Jobs & Pipelines** page in the left sidebar, click to edit the pipeline.
    - When you edit a job and add a pipeline task, you can click the button when you choose a pipeline under **Pipeline**.
- If you are browsing **All files** in the asset browser, and open a source code file from another pipeline, a banner is shown at the top of the editor, prompting you to open that associated pipeline.

Pipeline asset browser

When you are editing a pipeline, the left workspace sidebar uses a special mode called the *pipeline asset browser*. By default, the pipeline asset browser focuses on the pipeline root, and folders and files within the root. You can also choose to view **All files** to see files outside the root of the pipeline. The tabs opened in the pipeline editor while editing a specific pipeline are remembered, and when you switch to another pipeline, the tabs open the last time you edited that pipeline are restored.

> **NOTE**
>
> The editor also has contexts for editing SQL files (called the *Databricks SQL Editor*) and a general context for editing workspace files that are not SQL files or pipeline files. Each of these contexts remembers and restores the tabs that you had open the last time that you used that context. You can switch context from the top of the left sidebar. Click the header to choose between Workspace, SQL Editor, or recently edited pipelines.

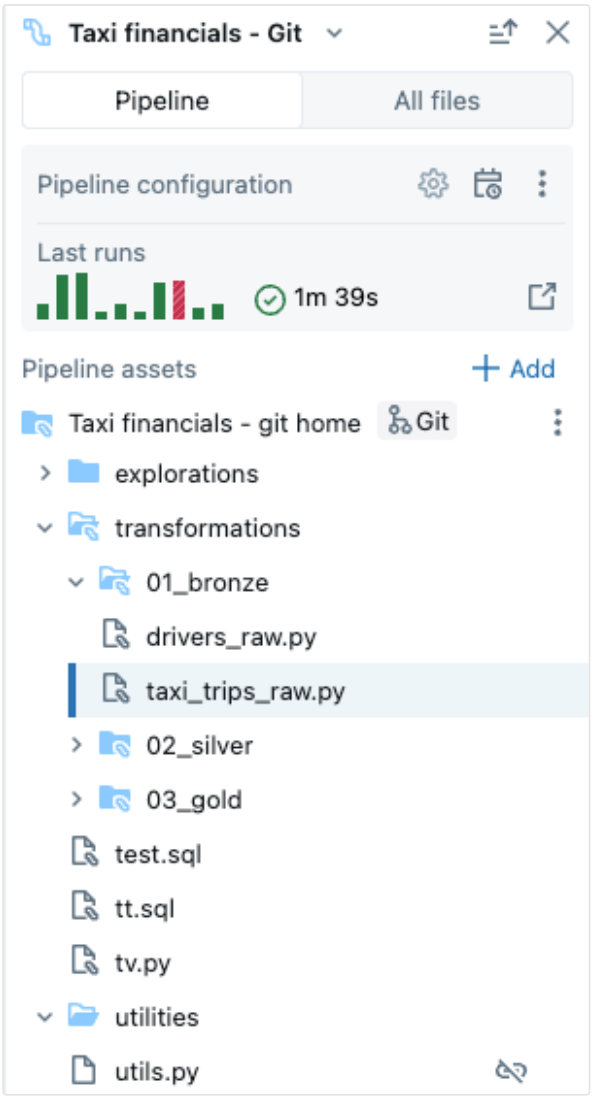


When you open a file from the Workspace browser page, it opens in the corresponding editor for that file. If the file is associated with a pipeline, that is the Lakeflow Pipelines Editor.

To open a file that isn't part of the pipeline, but retain the pipeline context, open the file from the asset browser's **All files** tab.

The pipeline asset browser has two tabs:

- **Pipeline**: This is where you can find all files associated with the pipeline. You can create, delete, rename, and organize them into folders. This tab also includes shortcuts for pipeline configuration, and a graphical view of recent runs.
- **All files**: All other workspace assets are available here. This can be useful for finding files to add to the pipeline, or viewing other files related to the pipeline, such as a YAML file that defines a Declarative Automation Bundles.



You can have the following types of files in your pipeline:

- **Source code files**: These files are part of the pipeline's source code definition, which can be seen in **Settings**. Databricks recommends always storing source code files inside the pipeline root folder; otherwise, they are shown in an external file section at the bottom of the browser and have a less rich feature set.
- **Non-source code files**: These files are stored inside the pipeline root folder but are not part of the pipeline source code definition.

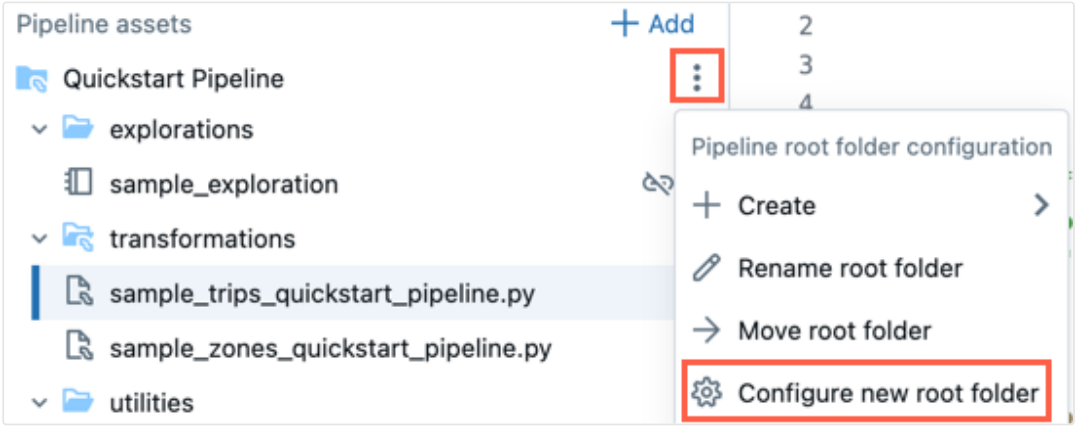
> **IMPORTANT**
>
> You must use the pipeline asset browser under the **Pipeline** tab to manage files and folders for your pipeline. This updates the pipeline settings correctly. Moving or renaming files and folders from your workspace browser or the **All files** tab breaks the pipeline configuration, and you must then resolve this manually in **Settings**.

Root folder

The pipeline asset browser is anchored in a pipeline root folder. When you create a new pipeline, the pipeline root folder is created in your user home folder.

You can change the root folder in the pipeline asset browser. This is useful if you created a pipeline in a folder and later want to move everything to a different folder. For example, you created the pipeline in a normal folder and want to move the source code to a Git folder for version control.

1. Click the ⋮ overflow menu for the root folder.
2. Click **Configure new root folder**.
3. Under **Pipeline root folder** click 🗀 and choose another folder as the pipeline root folder.
4. Click **Save**.



In the ⋮ for the root folder, you can also click **Rename root folder** to rename the folder name. Here, you can also click **Move root folder** to move the root folder, for example, into a Git folder.

You can also change the pipeline root folder in settings:

1. Click **Settings**.
2. Under **Code assets** click **Configure paths**.
3. Click 🗀 to change the folder under **Pipeline root folder**.
4. Click **Save**.

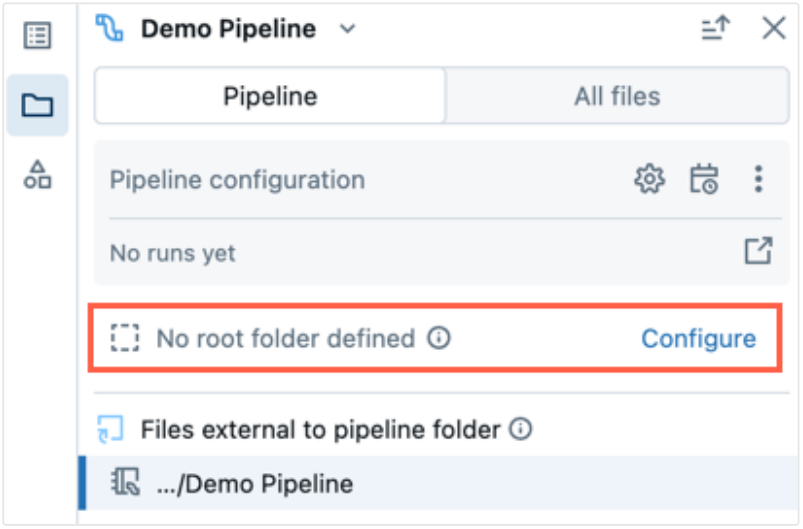
> ⓘ **NOTE** Ask Genie

> If you change the pipeline root folder, the file list displayed by the pipeline asset browser is affected, as the files in the previous root folder are shown as external files.

Existing pipeline with no root folder

An existing pipeline created using the legacy notebook editing experience won't have a root folder configured. When you open a pipeline that doesn't have a root folder configured, if you wish to configure the root folder for your pipeline, follow these steps:

1. In the pipeline asset browser, click **Configure**.
2. Click 🗀 to select the root folder under **Pipeline root folder**.
3. Click **Save**.



Default folder structure

When you create a new pipeline, a default folder structure is created. This is the recommended structure for organizing your pipeline source and non-source code files, as described below.

A small number of sample code files are created in this folder structure.

Folder name	Recommended location for these types of files
`<pipeline_root_folder>`	Root folder that contains all folders and files for your pipeline.
`transformations`	Source code files, such as Python or SQL code files with table definitions.
`explorations`	Non-source code files, such as notebooks, queries, and code files used for explorative data analysis.
`utilities`	Non-source code files with Python modules that can be imported from other code files. If you choose SQL as your language for sample code, this folder is not created.

You can rename the folder names or change the structure to fit your workflow. To add a new source code folder, follow these steps:

1. Click **Add** in the pipeline asset browser.
2. Click **Create pipeline source code folder**.
3. Enter a folder name and click **Create**.

Source code files

Source code files are part of the pipeline's source code definition. When you run the pipeline, these files are evaluated. Files and folders part of the source code definition have a special icon with a mini Pipeline icon superimposed.

To add a new source code file:

1. Click ⊞ next to the root folder.
2. Click **Transformation**.
3. Enter a **Name** for the file and select **Python** or **SQL** as the **Language**.
4. Click **Create**.

Use the in-line helpers to start writing code using Genie Code or generate short code snippets for the desired dataset type (for example, materialized view or streaming table).

A `transformations` folder for source code is created by default when you create a new pipeline. This folder is the recommended location for pipeline source code, such as Python or SQL code files with pipeline table definitions.

Non-source code files

Non-source code files are stored inside the pipeline root folder but are not part of the pipeline source code definition. These files are not evaluated when you run the pipeline. Non-source code files cannot be external files.

You can use this for files related to your work on the pipeline that you'd like to store together with the source code. For example:

- Notebooks that you use for ad hoc explorations executed on non-Lakeflow Spark Declarative Pipelines compute outside the lifecycle of a pipeline.
- Python modules that are not to be evaluated with your source code unless you explicitly import these modules inside your source code files.

To add a new non-source code file:

1. Click ⊕ next to the root folder.
2. Click **Exploration** or **Utility**.
3. Enter a **Name** for the file.
4. Click **Create**.

When you create a new pipeline, the following folders for non-source code files are created by default:

Folder name	Description
`explorations`	This folder is the recommended location for notebooks, queries, dashboards, and other files and then run them on non-Lakeflow Spark Declarative Pipelines compute, as you would normally do outside of a pipeline's execution lifecycle.
`utilities`	This folder is the recommended location for Python modules that can be imported from other files via direct imports expressed as `from <filename> import`, as long as their parent folder is hierarchically under the root folder.

You can also import Python modules located outside the root folder, but in that case, you must append the folder path to `sys.path` in your Python code:

Python

```
import sys, os
sys.path.append(os.path.abspath('<alternate_path_for_utilities>/utilities'))
from utils import \*
```

External files

The pipeline browser's **External files** section shows source code files outside the root folder.

To move an external file to the root folder, such as the `transformations` folder, follow these steps:

1. Click ⋮ for the file in the assets browser and click **Move**.
2. Choose the folder to which you want to move the file and click **Move**.

Files associated with multiple pipelines

A badge is shown in the file's header if a file is associated with more than one pipeline. It has a count of associated pipelines and allows switching to the other ones.

All files section

In addition to the **Pipeline** section, there is an **All files** section, where you can open any file in your workspace. Here you can:

- Open files outside the root folder in a tab without leaving the Lakeflow Pipelines Editor.
- Navigate to another pipeline's source code files and open them. This opens the file in the editor, and give you a banner with the option to switch focus in the editor to this second pipeline.
- Move files to the pipeline's root folder.
- Include files outside the root folder in the pipeline source code definition.

Edit pipeline source files

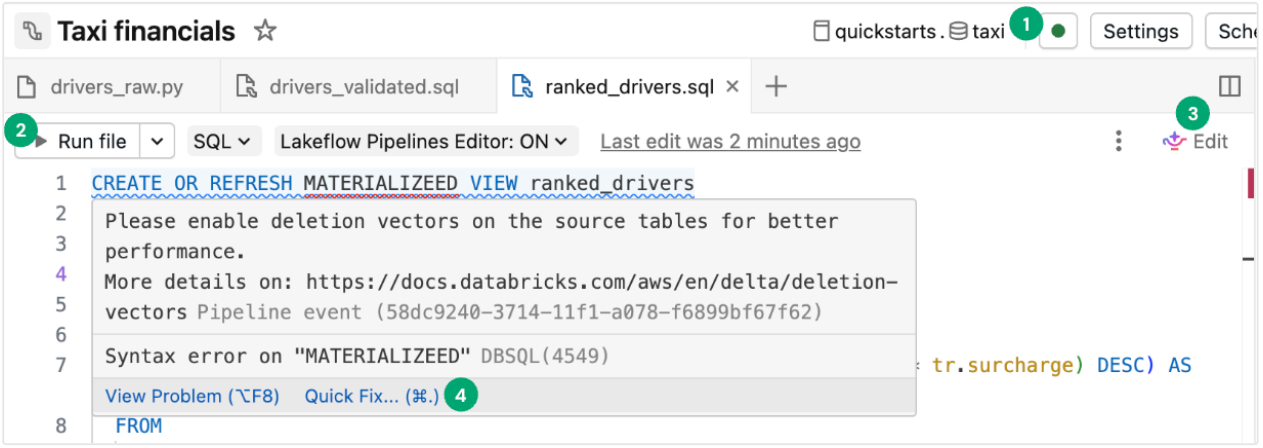
When you open a pipeline source file from the workspace browser, or the pipeline asset browser, it opens in an editor tab in the Lakeflow Pipelines Editor. Opening more files opens separate tabs, allowing you to edit multiple files at once.

> **NOTE**
>
> Opening a file that isn't associated with a pipeline from the workspace browser will open the editor in a different context (either the general **Workspace** editor or, for SQL files, the **SQL Editor**).
>
> When you open a non-pipeline file from the **All files** tab of the pipeline asset browser, it opens in a new tab in the pipeline context.

Pipeline source code includes multiple files. By default the source files are in the **transformations** folder in the pipeline asset browser. Source code files can be Python (**.py*) or SQL (**.sql*) files. Your source can include a mix of both Python and SQL files in a single pipeline, and the code in one file can reference a table or view defined in another file.

You can also include markdown (**.md*) files in your **tranformations** folder. Markdown files can be used for documentation or notes, but are ignored when running a pipeline update.

The following features are specific to the Lakeflow Pipelines Editor:



1. **Connect**: Connect to either serverless or classic compute to run the pipeline. All files associated with the pipeline use the same compute connection, so once you have connected, you don't need to connect for other files in the same pipeline. For more information about compute options, see Compute configuration options.

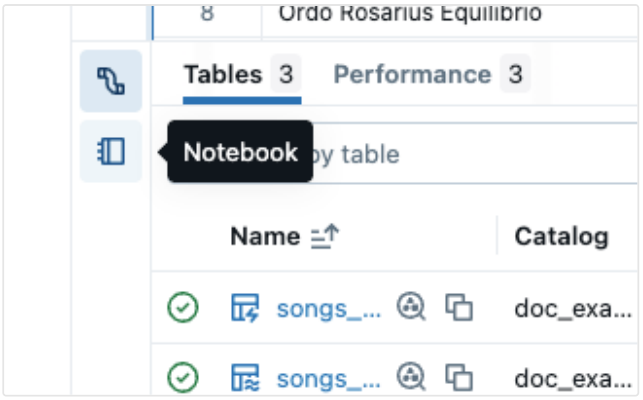
   For non-pipeline files, such as an exploratory notebook, the connect option is available, but applies only to that individual file.

2. **Run file**: Run the code to update the tables defined in this source file. The next section describes different ways to run your pipeline code.

3. **Edit**: Use the Genie Code to edit or add code in the file.

4. **Quick fix**: Use Genie Code to fix errors or act on insights in your code.

The bottom panel also adjusts, based on the current tab. Viewing pipeline information in the bottom panel is always available. Non-pipeline associated files, such as SQL editor files, also show their output in the bottom panel in a separate tab. The following image shows a vertical tab selector to switch the bottom panel between viewing pipeline information or information for the selected notebook.

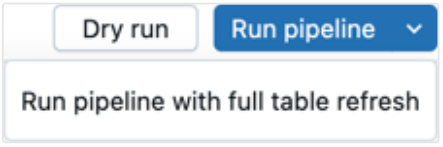


Run pipeline code

You have four options to run your pipeline code:

1. **Run all source code files in the pipeline**

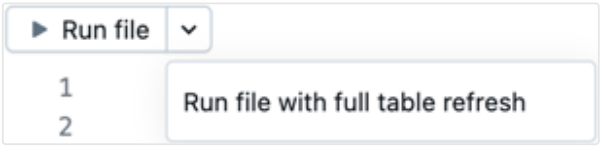
   Click **Run pipeline** or **Run pipeline with full table refresh** to run all table definitions in all files defined as pipeline source code. For details on refresh types, see Pipeline refresh semantics.

   You can also click **Dry run** to validate the pipeline without updating any data.

2. **Run the code in a single file**

   Click **Run file** or **Run file with full table refresh** to run all table definitions in the current file. Other files in the pipeline are not evaluated.

   This option is useful for debugging when quickly editing and iterating on a file. There are side effects when only running the code in a single file.

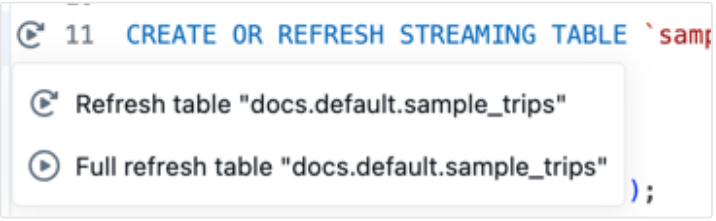
   - When other files are not evaluated, errors in those files are not found.

- Tables materialized in other files use the most recent materialization of the table, even if there is more recent source data.
- You could run into errors if a referenced table has not yet been materialized.
- The pipeline graph may be incorrect or disjointed for tables in other files that have not been materialized. Databricks does a best effort to keep the graph correct, but does not evaluate other files to do so.

When you are finished debugging and editing a file, Databricks recommends running all source code files in the pipeline to verify that the pipeline works end-to-end before putting the pipeline in production.

3. **Run the code for a single table**

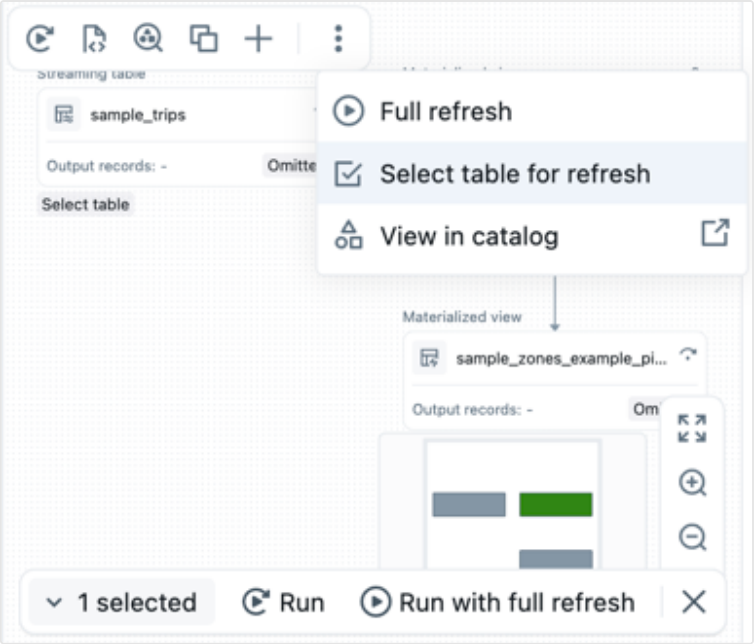
   Next to the definition of a table in the source code file, click the **Run table icon** and then choose either **Refresh table** or **Full refresh table** from the drop-down. Running the code for a single table has similar side-effects as running the code in a single file.



> **NOTE**
> Running the code for a single table is available for streaming tables and materialized views. Sinks and views are not supported.

4. **Run the code for a set of tables**

   You can select tables from the pipeline graph to create a list of tables to run. Hover over the table in the pipeline graph, click the ⋮, and choose **Select table for refresh**. After you have chosen the tables to refresh, choose either the **Run** or **Run with full refresh** option from the bottom of the pipeline graph.

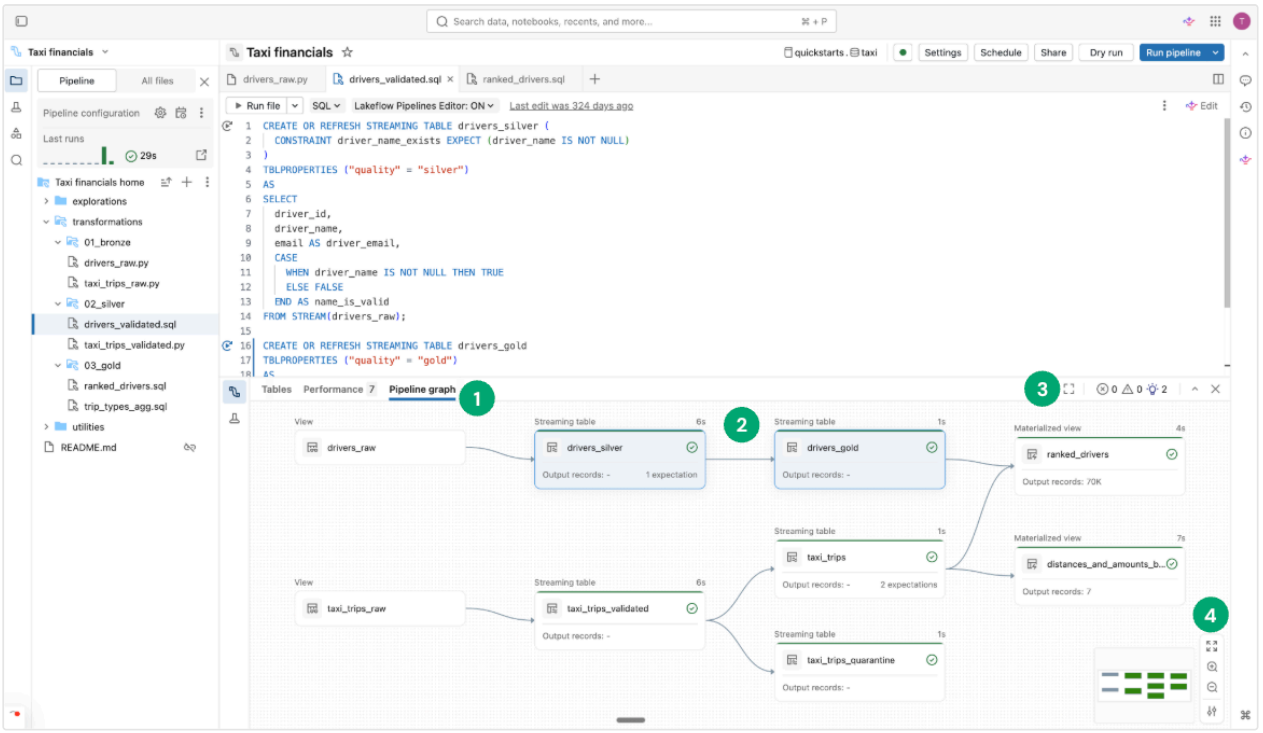


5. **Run selected code**

   Highlight SQL code and click **Run selected code** to quickly inspect outputs without materializing the data. Outputs are displayed in the **Query Results** tab in the bottom panel.

Pipeline graph

After you have run or validated all source code files in the pipeline, you see the *pipeline graph*, also called the directed acyclic graph (DAG). The graph shows the table dependency graph. Each node has different states along the pipeline lifecycle, such as validated, running, or error.



1. **Pipeline graph**: Open the graph by clicking the **Pipeline graph** tab in the bottom panel.
2. **Nodes**: Display the dependencies of the tables that are part of your pipeline as well as any metrics pertaining to them. Nodes that are part of the currently open files are highlighted in the pipeline graph. Hovering over a node displays a toolbar with options, including refresh the query. Right-clicking a node gives you the same options in a context menu. Clicking a node shows the data preview and table definition. When you edit a file, the tables defined in that file are highlighted in the graph.
3. **Open in tab**: To maximize the graph, select the icon on the top right of the bottom panel to open it in a separate tab.
4. **More options**: Additional options are at the bottom right, including zoom options and **More options** to display the graph in a vertical or horizontal layout.

Data previews

The data preview section shows sample data for a selected table.

You see a preview of the table's data when you click a node in the pipeline graph. To navigate to the data preview of a different table directly in the bottom panel, select

Back to graph or click a different node if you have the pipeline graph open in a separate tab.

Alternatively, go to the **Tables** section and click **View data preview** . If you have chosen a table, click **All tables** to return to all tables.

When you preview the table data, you can filter or sort the data in-place. If you want to do more complex analysis, you can use or create a notebook in the **Explorations** folder (assuming that you kept the default folder structure). By default, source code in this folder is not run during a pipeline update, so you can create queries without affecting the pipeline output.

Execution insights

You can see the table execution insights about the latest pipeline update in the panels at the editor's bottom.

Panel	Description
Tables	Lists all tables with their statuses and metrics. If you select one table, you see the metrics and performance for that table and a tab for the data preview.
Performance	Query history and profiles for all flows in this pipeline. You can access execution metrics and detailed query plans during and after execution. See Access query history for pipelines for more information.
Issues panel	Click the panel for a simplified view of errors, warnings, and insights for the pipeline. Click an entry to see more details, then navigate to the place in the code where the error occurred. If the error is in a file other than the one currently displayed, this redirects you to the file where the error is. Click **View details** to see the corresponding event log entry for complete details. Click **View logs** to see the complete event log.

Panel	Description
	Click **Diagnose error** to debug the issue with Genie Code. Code-affixed error indicators are shown for errors associated with a specific part of the code. To get more details, click the **error** icon or hover over the red line. A pop-up with more information appears. You can then click **Quick fix** to reveal a set of actions to troubleshoot the error.
Event log	All events triggered during the last pipeline run. Click **View logs** or any entry in the issues tray.

Pipeline configuration

You can configure your pipeline from the pipeline editor. You can make changes to the pipeline settings, schedule, or permissions.

Each of these can be accessed from a button in the header of the editor, or from icons on the asset browser (the left sidebar).

- **Settings** (or choose in the asset browser):

  You can edit settings for the pipeline from the settings panel, including general information, root folder and source code configuration, compute configuration, notifications, advanced settings, and more.

- **Schedule** (or choose in the asset browser):

  You can create one or more schedules for your pipeline from the schedule dialog. For example, if you want to run it daily, you can set that here. It creates a job to run the pipeline on the schedule you choose. You can add a new schedule or remove an existing schedule from the schedule dialog.

- **Share** (or, from the menu in the asset browser, choose):

  You can manage permissions on the pipeline for users and groups from the pipeline permissions dialog.

Event Log

You can publish the event log for a pipeline to Unity Catalog. By default, the event log for your pipeline is shown in the UI, and accessible for querying by the owner.

1. Open **Settings**.
2. Click the › arrow next to **Advanced settings**.
3. Click **Edit advanced settings**.
4. Under **Event logs**, click **Publish to catalog**.
5. Provide a name, catalog, and schema for the event log.
6. Click **Save**.

Your pipeline events are published to the table you specified.

To learn more about using the pipeline event log, see Query the event log.

Pipeline environment

You can create an environment for your source code by adding dependencies in **Settings**.

1. Open **Settings**.
2. Under **Pipeline environment**, click **Edit environment**.
3. Click **Add dependency** to add a dependency, as if you were adding it to a `requirements.txt` file. For more information about dependencies, see Add dependencies to the notebook.

Databricks recommends that you pin the version with `==`. See PyPI package.

The environment applies to all source code files in your pipeline.

Notifications

You can add notifications using the **Pipeline settings**.

1. Open **Settings**.
2. In the **Notifications** section, click **Add notification**.

3. Add one or more email addresses and the events you want them to be sent.
4. Click **Add notification**.

> **NOTE**
>
> Create custom responses to events, including notifications or custom handling, by using Python event hooks.

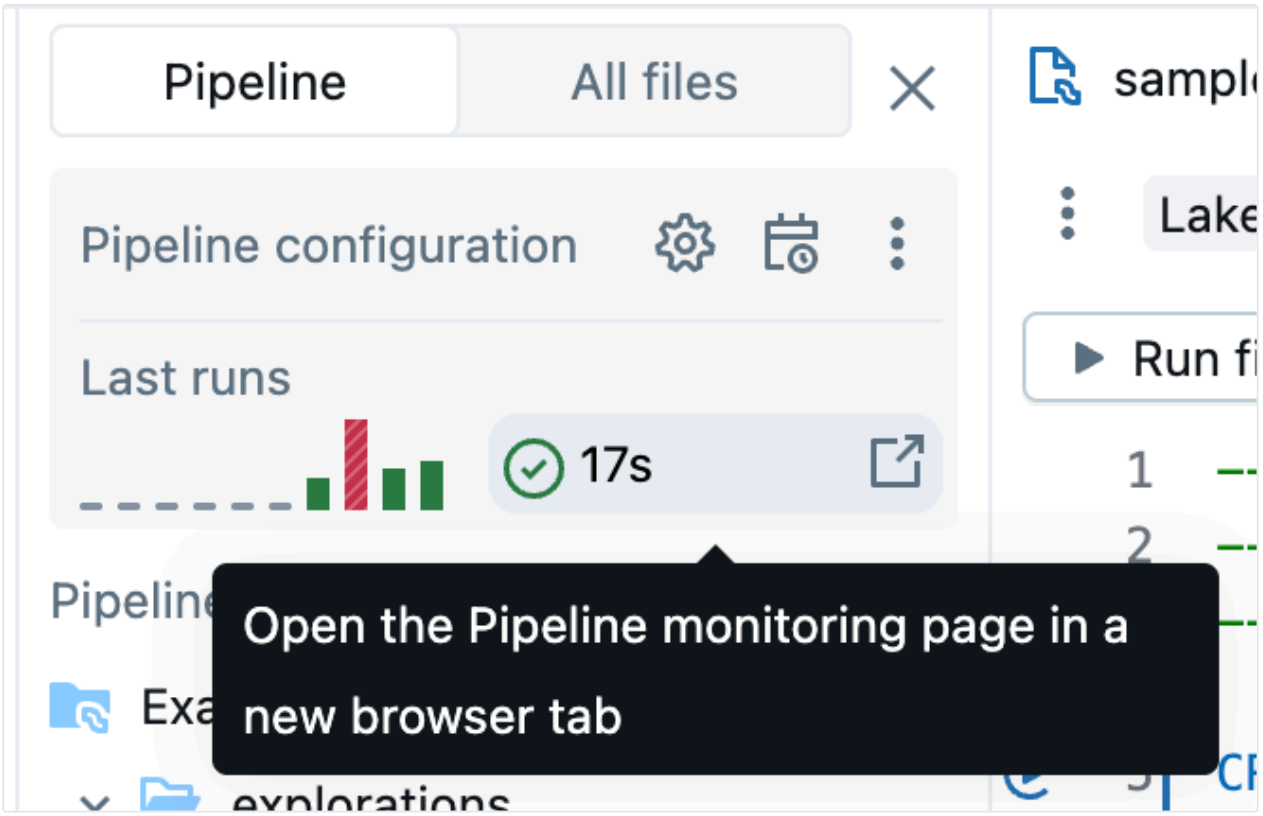
Monitoring pipelines

Databricks also provides features to monitor running pipelines. The editor shows the results and execution insights about the most recent run. It is optimized to help you to iterate efficiently while you develop your pipeline interactively.

The pipeline monitoring page allows you to view historical runs, which is useful when a pipeline is running on a schedule using a Job.

> **NOTE**
>
> There is a default monitoring experience, and an updated preview monitoring experience. The following section describes how to enable or disable the preview monitoring experience. For information about both experiences, see Monitor pipelines in the UI.

The monitoring experience is available from the **Jobs & Pipelines** button on the left side of your workspace. You can also jump directly to the monitoring page from the editor by clicking the run results in the pipeline asset browser.



For more information about the monitoring page, see Monitor pipelines in the UI. The monitoring UI includes the ability to return to the Lakeflow Pipelines Editor by selecting **Edit pipeline** from the header of the UI.

Data Engineering Agent

> **PUBLIC PREVIEW**
>
> This feature is in Public Preview.

The Lakeflow Pipelines Editor integrates with the Genie Code Data Engineering Agent, which can generate, modify, and debug entire Lakeflow Spark Declarative Pipelines directly from natural language. For more information, see Use Genie Code for pipeline development.

Limitations and known issues

See the following limitations and known issues for the ETL pipeline editor in Lakeflow Spark Declarative Pipelines:

1. The workspace browser sidebar does not focus on the pipeline if you start by opening a file in the `explorations` folder or a notebook, as these files or notebooks are not part of the pipeline source code definition.

   To enter the pipeline focus mode in the workspace browser, open a file associated with the pipeline.

2. Data previews are not supported for regular views.

3. Python modules are not found from within a UDF, even if they are in your root folder or are on your `sys.path`. You can access these modules by appending the path to the `sys.path` from within the UDF, for example: `sys.path.append(os.path.abspath("/Workspace/Users/path/to/modules"))`

4. `%pip install` is not supported from files (the default asset type with the new editor). You can add dependencies in settings. See Pipeline environment.

   Alternately, you can continue to use `%pip install` from a notebook associated with a pipeline, in its source code definition.

FAQ

1. **Why use files and not notebooks for source code?**

   The cell-based execution of notebooks is not compatible with pipelines. Standard features of notebooks are either disabled or changed when working with pipelines, which leads to confusion for users familiar with notebook behavior.

   In the Lakeflow Pipelines Editor, the file editor is used as a foundation for a first-class editor for pipelines. Features are targeted explicitly to pipelines, like **Run table**, rather than overloading familiar features with different behavior.

2. **Can I still use notebooks as source code?**

   Yes, you can. However, some features, such as **Run table** or **Run file**, are not present.

   If you have an existing pipeline using notebooks, it still works in the new editor. However, Databricks recommends switching to files for new pipelines.

3. **How can I add existing code to a newly created Pipeline?**

   You can add existing source code files to a new pipeline. To add a folder with existing files, follow these steps:

   i. Click **Settings**.
   ii. Under **Source code** click **Configure paths**.
   iii. Click **Add path** and choose the folder for the existing files.
   iv. Click **Save**.

   You can also add individual files:

   i. Click **All files** in the pipeline asset browser.
   ii. Navigate to your file, click ⋮, and click **Include in pipeline**.

   Consider moving these files to the pipeline root folder. If left outside the pipeline root folder, they are shown in the **External files** section.

4. **Can I manage the Pipeline source code in Git?**

   You can manage your pipeline source in Git by choosing a Git folder when you initially create the pipeline.

   > ⓘ **NOTE**
   > Managing your source in a Git folder adds version control for your source code. However, to version control your configuration, Databricks recommends using Declarative Automation Bundles to define the pipeline configuration in bundle configuration files that can be stored in Git (or another version control system). For more information, see What are Declarative Automation Bundles?.

   If you didn't create the pipeline in a Git folder initially, you can move your source to a Git folder. Databricks recommends using the editor action to move the entire root folder to a Git folder. This updates all settings accordingly. See Root folder.

   To move the root folder to a Git folder in the pipeline asset browser:

   i. Click ⋮ for the root folder.
   ii. Click **Move root folder**.

iii. Choose a new location for your root folder and click **Move**.

See the Root folder section for more information.

After the move, you see the familiar Git icon next to your root folder's name.

> **IMPORTANT**
> To move the pipeline root folder, use the pipeline asset browser and the above steps. Moving it any other way breaks the pipeline configurations, and you must manually configure the correct folder path in **Settings**.

5. **Can I have multiple Pipelines in the same root folder?**

   You can, but Databricks recommends only having a single Pipeline per root folder.

6. **When should I run a dry run?**

   Click **Dry run** to check your code without updating the tables.

7. **When should I use temporary Views, and when should I use materialized views in my code?**

   Use temporary views when you do not want to materialize the data. For example, this is a step in a sequence of steps to prepare the data before it is ready to materialize using a streaming table or materialized view registered in the Catalog.

*Last updated on **May 4, 2026***

Develop and debug pipelines with a notebook (legacy)

PREVIEW

This feature is in Public Preview.

This article describes how to use the legacy notebook editing experience in Lakeflow Spark Declarative Pipelines to develop and debug ETL pipelines.

IMPORTANT

This page describes the legacy notebook editing experience. This feature can no longer be enabled and is only accessible in workspaces that previously opted out of the Lakeflow Pipelines Editor. The notebook editing experience is deprecated and will be removed.

The default experience is the Lakeflow Pipelines Editor. Use the Lakeflow Pipelines Editor to edit notebooks, or Python or SQL code files for a pipeline. For more information, see Develop and debug ETL pipelines with the Lakeflow Pipelines Editor.

Overview of notebooks in Lakeflow Spark Declarative Pipelines

When you work on a Python or SQL notebook that is configured as source code for an existing pipeline, you can connect the notebook directly to the pipeline. When the notebook is connected to the pipeline, the following features are available:

- Start and validate the pipeline from the notebook.

- View the pipeline's dataflow graph and event log for the latest update in the notebook.
- View pipeline diagnostics in the notebook editor.
- View the status of the pipeline's cluster in the notebook.
- Access the Lakeflow Spark Declarative Pipelines UI from the notebook.

Prerequisites

- You must have an existing pipeline with a Python or SQL notebook configured as source code.
- You must either be the owner of the pipeline or have the `CAN_MANAGE` privilege.

Limitations

- The features covered in this article are only available in Databricks notebooks. Workspace files are not supported.
- The web terminal is not available when attached to a pipeline. As a result, it is not visible as a tab in the bottom panel.

Connect a notebook to a pipeline

Inside the notebook, click on the drop-down menu used to select compute. The drop-down menu shows all your Lakeflow Spark Declarative Pipelines with this notebook as source code. To connect the notebook to a pipeline, select it from the list.

View the pipeline's cluster status

To easily understand the state of your pipeline's cluster, its status is shown in the compute drop-down menu with a green color to indicate that the cluster is running.

Validate pipeline code

You can validate the pipeline to check for syntax errors in your source code without processing any data.

To validate a pipeline, do one of the following:

- In the top-right corner of the notebook, click **Validate**.
- Press `Shift+Enter` in any notebook cell.
- In a cell's dropdown menu, click **Validate Pipeline**.

> **NOTE**
>
> If you attempt to validate your pipeline while an existing update is already running, a dialog box displays asking if you want to terminate the existing update. If you click **Yes**, the existing update stops, and a *validate* update automatically starts.

Start a pipeline update

To start an update of your pipeline, click the **Start** button in the top-right corner of the notebook. See Run a pipeline update.

View the status of an update

The top panel in the notebook displays whether a pipeline update is:

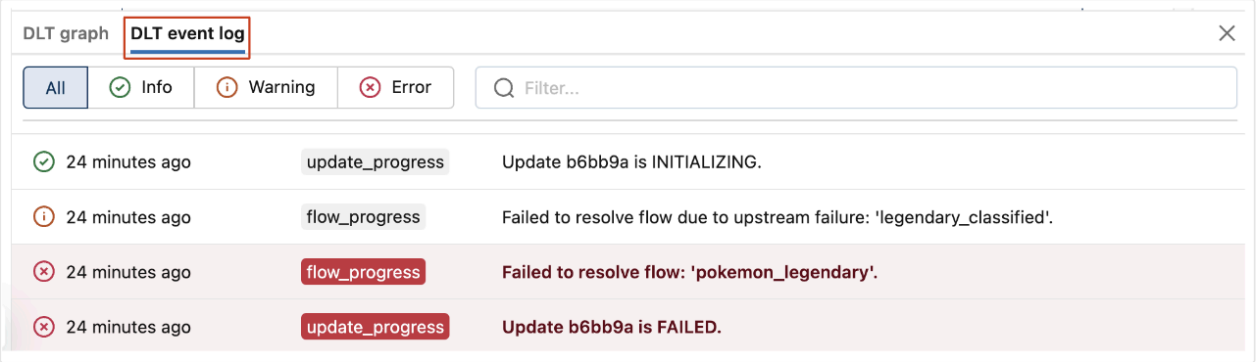
- Starting
- Validating
- Stopping

View errors and diagnostics

After you start a pipeline update or validation, any errors are shown inline with a red underline. Hover over an error to see more information.

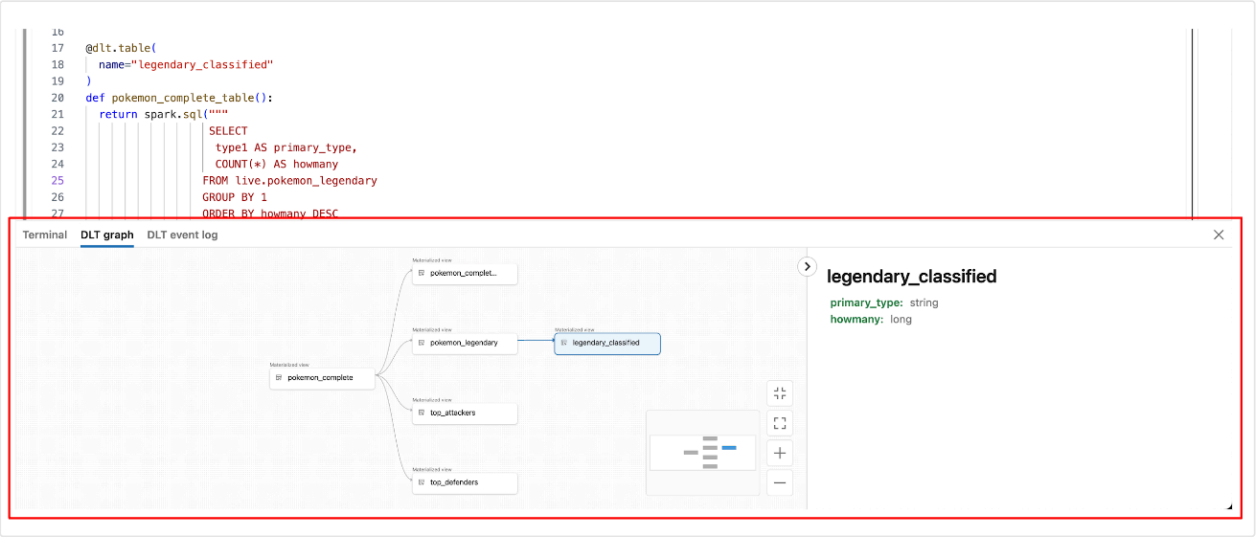
View pipeline events

When attached to a pipeline, there is a Lakeflow Spark Declarative Pipelines event log tab at the bottom of the notebook.



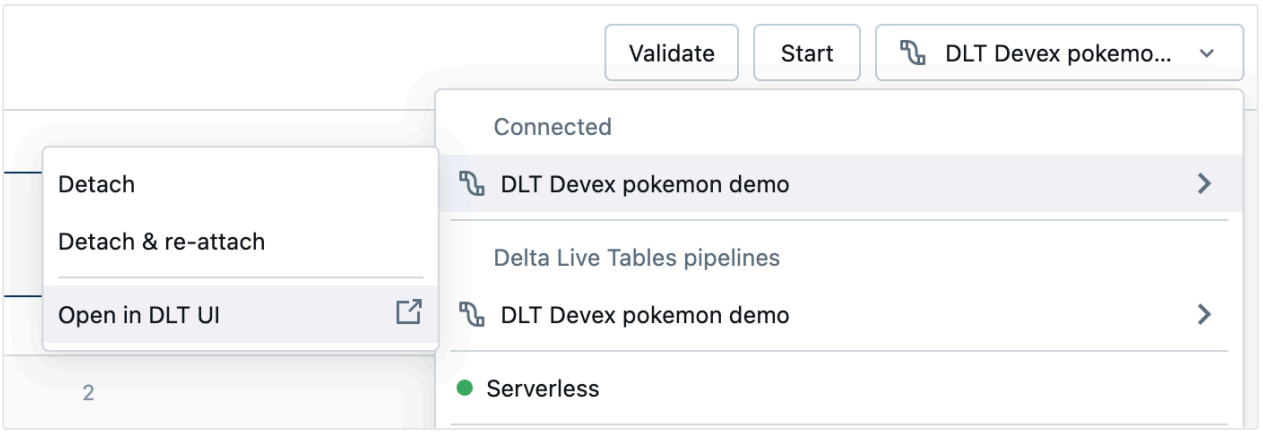
View the pipeline Dataflow Graph

To view a pipeline's dataflow graph, use the Lakeflow Spark Declarative Pipelines graph tab at the bottom of the notebook. Selecting a node in the graph displays its schema in the right panel.



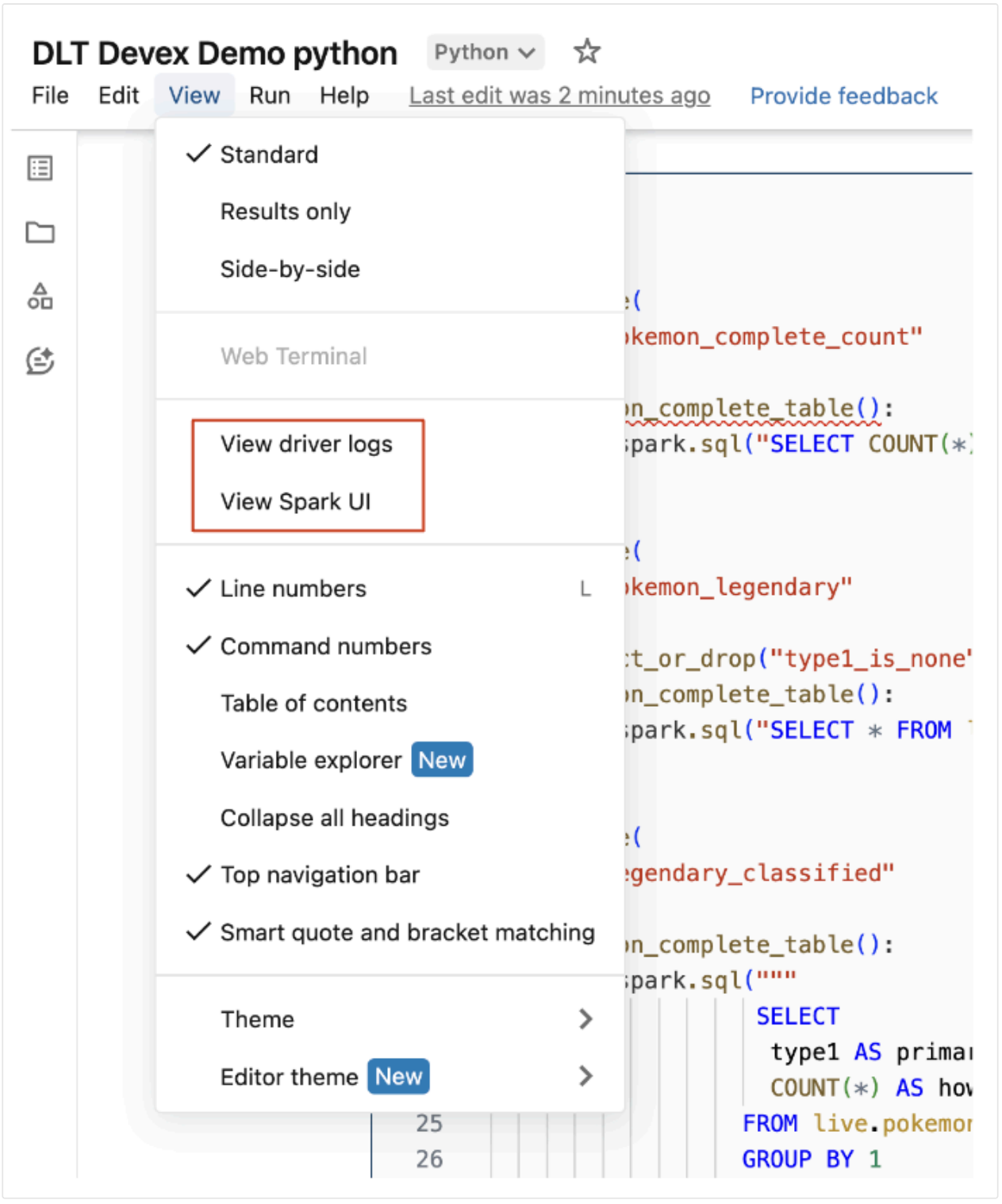
How to access the Lakeflow Spark Declarative Pipelines UI from the notebook

To easily jump to the Lakeflow Spark Declarative Pipelines UI, use the menu in the top-right corner of the notebook.



Access driver logs and the Spark UI from the notebook

The driver logs and Spark UI associated with the pipeline being developed can be easily accessed from the notebook's **View** menu.



this page

as this page helpful?

Ask Genie

Yes

No

Last updated on ***Apr 28, 2026***

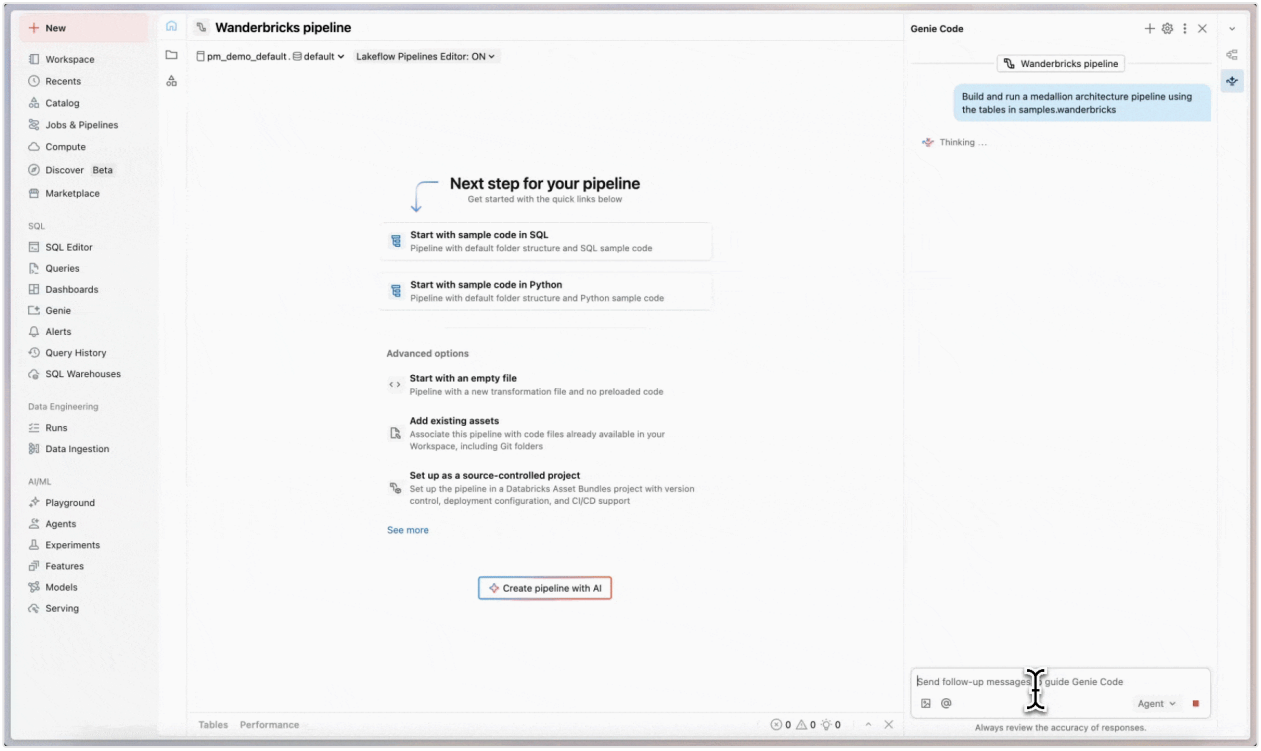
Use Genie Code for pipeline development

> **PREVIEW**
>
> This feature is in Public Preview.

This page introduces Genie Code for pipeline development, an AI data agent available by selecting Agent mode in Genie Code. Designed specifically for Lakeflow Spark Declarative Pipelines (SDP) and the Lakeflow Pipelines Editor, it explores data, generates and runs pipeline code, and fixes errors, all from a single prompt.

What is Genie Code for pipeline development?

Genie Code in Agent mode is an autonomous partner that can automate entire multi-step data engineering workflows in SDP and the Lakeflow Pipelines Editor.



Compared to Genie Code chat mode, Agent mode has expanded capabilities: planning a solution, retrieving relevant assets, running code, using pipeline outputs to improve results, fixing errors automatically, and more.

Genie Code in Agent mode can plan and generate entire pipelines end-to-end from scratch, or accelerate working on an existing pipeline. The agent works with you to approve its plans and confirm its next steps before proceeding. With your approval, Genie Code can use tools to perform tasks like searching tables, editing a SQL or Python source file, running pipeline updates, and reading pipeline datasets.

Genie Code's access and actions are governed by the user's permissions. It can only access data that you have access to and perform operations that you have permissions for.

> **NOTE**
>
> When you turn on Agent mode in Genie Code, Genie Code adapts its capabilities based on the features you are currently using in Databricks. For example, in the Lakeflow Pipelines Editor, Genie Code focuses on pipeline editing and data engineering tasks. In notebooks and the SQL Editor, Genie Code supports data exploration and analysis. See Use Genie Code for data science for more information.
>
> Ask Genie

Requirements

To use Genie Code for data engineering, your workspace needs the following:

- Partner-powered AI features enabled for both the account and workspace. See Partner-powered AI features.
- Your workspace must be in a supported region. Genie Code is a Designated Service that uses Geos to manage data residency. See Geo availability of Genie Code features.

Use Genie Code for pipeline development

To use Genie Code's agentic capabilities for pipeline development:

1. From Lakeflow Pipelines Editor, open Genie Code side panel by clicking **Genie Code** in the upper-right corner of your workspace.

2. In the lower-right corner, select **Agent**. This toggles on Genie Code's Agent mode, allowing you to Genie Code's agentic data engineering capabilities.

3. Enter a prompt for Genie Code. For example, you can ask it questions about your pipeline, such as "describe this pipeline". You can also ask it to add new datasets, for example, "create silver_sales_data in a new file that reads from bronze_sales_data and cleans the data and adds useful quality expectations."

   > **NOTE**
   >
   > Genie Code respects the user's Unity Catalog permissions, so it can only access the data and pipeline source that you have access to.

4. As Genie Code generates its response, it often pauses to get your input:

   - For more complex tasks, Genie Code may create a step-by-step plan and ask clarifying questions. Answer its clarifying questions to help it hone its plan.

- When Genie Code needs to run code or update a pipeline, it asks for your approval before proceeding. **Allow** or **Decline** its request. You can also select **Allow in this thread** (referring to Genie Code conversation thread) or **Always allow**.

  > **IMPORTANT**
  >
  > Genie Code in Agent mode can generate and execute code in your pipeline. While it has guardrails to prevent dangerous actions, there is still risk. You should only use it with data you trust, and you should review code before running it.

- As Genie Code continues its work, you may be prompted to select **Continue** or **Reject.** Review its existing work, then select **Continue** to allow it to continue to its next steps or **Reject** to tell it to try something else.

- To stop Genie Code while it is working, click the red ■.

Genie Code can create new files, generate text, queries, and code, run the files or pipelines, and access the output datasets to interpret the results.

> **NOTE**
>
> In order for Genie Code to continue its work and take next steps, you need to stay on the current tab that it is working in.

> **TIP**
>
> You can add instructions for the Genie Code to use in most responses. For example, if you have code conventions you want to use, or preferred libraries to use, you can add these guidelines to instructions for Genie Code. You can also create skills to extend Genie Code with specialized capabilities for your domain-specific tasks. For more details and other tips, see Tips to improve Genie Code responses.

Capabilities

In Agent mode, Genie Code can help with most pipeline development tasks. Key capabilities include:

- **Data discovery**: Genie Code can search tables in the workspace to help you find the required data for a task.
- **Pipeline code edits**: Genie Code can create and edit multiple files at a time. It keeps you informed about which files it is changing and shows you the code diff in each file, so you can review the changes individually or all together at the end.
- **Pipeline execution**: Genie Code can run individual files, dry-run/run the pipeline, or do a full refresh. When Genie Code wants to proceed, it asks for your confirmation before doing so.
- **Understanding and improving pipeline behavior**: Genie Code can inspect datasets and pipeline outputs to help you understand what a pipeline is doing end-to-end and why. For example, it can summarize transformations, trace how data flows into downstream tables, and highlight unexpected changes in row counts or schemas. When it surfaces potential data quality issues, Genie Code can help you reason about their cause and suggest where and how to address them in the pipeline.

These capabilities support common use cases such as:

- **Authoring a new pipeline**: Genie Code can help with all steps of creating a new medallion architecture pipeline, from ingesting data, to standardizing and cleaning the data, to transforming and analyzing the data.
- **Explain a pipeline**: Genie Code can analyze and explain an existing pipeline to help you ramp up quickly.
- **Fix issues**: When you have errors, Genie Code can help diagnose and fix the problems, iterating through multiple files until the issue is resolved.

Examples

Try the following prompts to get started:

- "Build and run a medallion architecture pipeline for fraud detection using the table transactions and customers in my_catalog.my_schema."
- "Explain every step of this pipeline."

- "Fix the failure in this pipeline."

Next steps

- Learn more about Databricks AI assistive features
- Get tips to Tips to improve Genie Code responses
- Use Genie Code for data science, for data discovery and exploration
- Use Genie Code for dashboard authoring
- Explore the Lakeflow Pipelines Editor

Last updated on ***Apr 21, 2026***

Load and process data incrementally with Lakeflow Spark Declarative Pipelines flows

Data is processed in pipelines through *flows*. Each flow consists of a *query* and, typically, a *target*. The flow processes the query, either as a batch, or incrementally as a stream of data into the target. A flow lives within a pipeline in Lakeflow Spark Declarative Pipelines.

Typically, flows are defined automatically when you create a query in a pipeline that updates a target, but you can also explicitly define additional flows for more complex processing, such as appending to a single target from multiple sources.

Updates

A flow is run each time that its defining pipeline is updated. The flow will create or update tables with the most recent data available. Depending on the type of flow and the state of the changes to the data, the update may perform an incremental refresh, which processes only new records, or perform a full refresh, that reprocesses all records from the data source.

- For more information about pipeline updates, see Run a pipeline update.
- For more information about scheduling and triggering updates, see Triggered vs. continuous pipeline mode.

Create a default flow

When you create a pipeline, you typically define a table or a view along with the query that supports it. For example, in this SQL query, you create a streaming table called `customers_silver` by reading from the table called `customers_bronze`.

SQL

```sql
CREATE OR REFRESH STREAMING TABLE customers_silver
AS SELECT * FROM STREAM(customers_bronze)
```

You can create the same streaming table in Python, as well. In Python, you use pipelines by creating a query function that returns a dataframe, with decorators to add Lakeflow Spark Declarative Pipelines functionality:

Python

```python
from pyspark import pipelines as dp

@dp.table()
def customers_silver():
  return spark.readStream.table("customers_bronze")
```

In this example, you created a *streaming table*. You can also create materialized views with similar syntax in both SQL and Python. For more information, see Streaming tables and Materialized views.

This example creates a default flow along with the streaming table. The default flow for a streaming table is an *append* flow, which add new rows with each trigger. This is the most common way to use pipelines: create a flow and the target in a single step. You can use this style to ingest data or to transform data.

Append flows also support processing that requires reading data from multiple streaming sources to update a single target. For example, you can use append flow functionality when you have an existing streaming table and flow and want to add a new streaming source that writes to this existing streaming table.

Using multiple flows to write to a single target

In the previous example, you created a flow and a streaming table in a single step. You can create flows for a previously created table, as well. In this example, you can see creating a table and the flow associated with it in separate steps. This code has identical results as creating a default flow, including using the same name for the streaming table and the flow.

Python SQL

Python

```
from pyspark import pipelines as dp

# create streaming table
dp.create_streaming_table("customers_silver")

# add a flow
@dp.append_flow(
  target = "customers_silver")
def customer_silver():
  return spark.readStream.table("customers_bronze")
```

Creating a flow independently from the target means that you can also create multiple flows that append data to the same target.

Use the `@dp.append_flow` decorator in the Python interface or the `CREATE FLOW...INSERT INTO` clause in the SQL interface to create a new flow, for example to target a streaming table from multiple streaming sources. Use append flow for processing tasks such as the following:

- Add streaming sources that append data to an existing streaming table without requiring a full refresh. For example, you might have a table combining regional data from every region you operate in. As new regions are rolled out, you can add the new region data to the table without performing a full refresh. For an example of adding streaming sources to existing streaming table, see Example: Write to a streaming table from multiple Kafka topics.

- Update a streaming table by appending missing historical data (backfilling). You can use the `INSERT INTO ONCE` syntax to create a historical backfill append that runs one time. For example, you have an existing streaming table that is written to by an Apache Kafka topic. You also have historical data stored in a table that you need inserted exactly once into the streaming table, and you cannot stream the data because your processing includes performing a complex aggregation before inserting the data. For an example of a backfill, see Backfilling historical data with pipelines.
- Combine data from multiple sources and write to a single streaming table instead of using the `UNION` clause in a query. Using append flow processing instead of `UNION` allows you to update the target table incrementally without running a full refresh update. For an example of a union done in this way, see Example: Use append flow processing instead of `UNION`.

The target for the records output by the append flow processing can be an existing table or a new table. For Python queries, use the create_streaming_table() function to create a target table.

The following example adds two flows for the same target, creating a union of the two source tables:

Python SQL

Python

```python
from pyspark import pipelines as dp

# create a streaming table
dp.create_streaming_table("customers_us")

# add the first append flow
@dp.append_flow(target = "customers_us")
def append1():
  return spark.readStream.table("customers_us_west")

# add the second append flow
@dp.append_flow(target = "customers_us")
def append2():
  return spark.readStream.table("customers_us_east")
```

IMPORTANT

- If you need to define data quality constraints with expectations, define the expectations on the target table as part of the `create_streaming_table()` function or on an existing table definition. You cannot define expectations in the `@append_flow` definition.
- Flows are identified by a *flow name*, and this name is used to identify streaming checkpoints. The use of the flow name to identify the checkpoint means the following:
  - If an existing flow in a pipeline is renamed, the checkpoint does not carry over, and the renamed flow is effectively an entirely new flow.
  - You cannot reuse a flow name in a pipeline, because the existing checkpoint won't match the new flow definition.

Types of flows

The default flows for streaming tables and materialized views are append flows. You can also create flows to read from *change data capture* data sources. The following table describes the different types of flows.

Flow type	Description
Append	*Append* flows are the most common type of flow, where new records in the source are written to the target with each update. They correspond to append mode in structured streaming. You can add the `ONCE` flag, indicating a batch query whose data should be inserted into the target only once, unless the target is fully refreshed. Any number of append flows can write to a particular target. Default flows (created with the target streaming table or materialized view) will have the same name as the target. Other targets do not have default flows.
Auto CDC (previously	An *Auto CDC* flow ingests a query containing change data capture (CDC) data. Auto CDC flows can only target streaming

Flow type	Description
apply changes)	tables, and the source must be a streaming source (even in the case of `ONCE` flows). Multiple auto CDC flows can target a single streaming table. A streaming table that acts as a target for an auto CDC flow can only be targeted by other auto CDC flows. For more information about CDC data, see The AUTO CDC APIs: Simplify change data capture with pipelines.

Additional information

For more information on flows and their use, see the following topics:

- Examples of flows in Lakeflow Spark Declarative Pipelines
- The AUTO CDC APIs: Simplify change data capture with pipelines
- Backfilling historical data with pipelines
- Writing pipelines in Python or SQL
- Streaming tables
- Materialized views
- Sinks in Lakeflow Spark Declarative Pipelines

*Last updated on **May 8, 2026***

Streaming tables

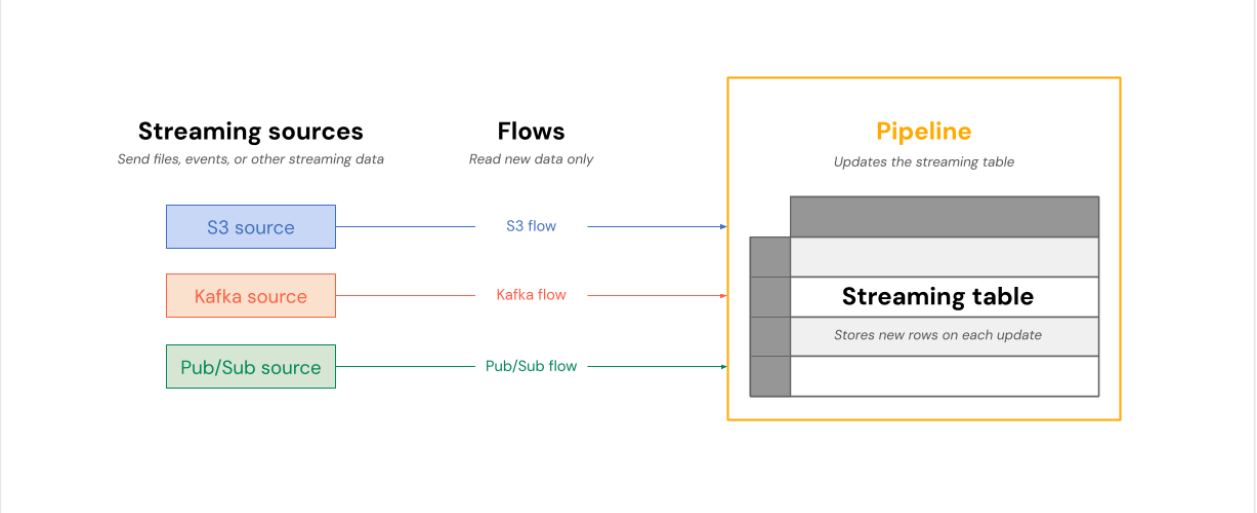
A Streaming table is a Delta table with additional support for streaming or incremental data processing. A Streaming table can be targeted by one or more flows in a pipeline.

Streaming tables are a good choice for data ingestion for the following reasons:

- Each input row is handled only once, which models the vast majority of ingestion workloads (that is, by appending or upserting rows into a table).
- They can handle large volumes of append-only data.

Streaming tables are also a good choice for low-latency streaming transformations because they can reason over rows and windows of time, handle high volumes of data, and provide low-latency processing.

The following diagram shows how flows read from streaming sources and write incrementally to a Streaming table within a pipeline.



On each update, the flows associated with a Streaming table read the changed information in a streaming source, and append new information to that table.

Streaming tables are owned and updated by a single pipeline. You explicitly define streaming tables in the source code of the pipeline. Tables defined by a pipeline can't be changed or updated by any other pipeline. You can define multiple flows to append to a single streaming table.

Databricks creates internal tables to support Streaming table processing. These tables appear in `system.information_schema.tables` but are not visible in Catalog Explorer or other workspace UI pages.

> **NOTE**
>
> When you create a streaming table outside a pipeline using Databricks SQL, Databricks creates a pipeline that is used to update the table. You can see the pipeline by selecting **Jobs & Pipelines** from the left navigation in your workspace. You can add the **Pipeline type** column to your view. Streaming tables defined in a pipeline have a type of `ETL`. Streaming tables created in Databricks SQL have a type of `MV/ST`.

For more information about flows, see Load and process data incrementally with Lakeflow Spark Declarative Pipelines flows.

Streaming tables for ingestion

Streaming tables are designed for append-only data sources and process inputs only once. This makes them well-suited for ingestion workloads where data arrives continuously and must be reliably captured without reprocessing existing records. Databricks supports ingesting data from cloud storage and streaming message buses.

Ingest files from cloud storage

You can use a Streaming table to ingest new files from cloud storage. These examples use Auto Loader to incrementally process new files as they arrive.

Python SQL

Python

```python
from pyspark import pipelines as dp

# Create a streaming table
@dp.table
def customers_bronze():
  return (
    spark.readStream.format("cloudFiles")
     .option("cloudFiles.format", "json")
     .option("cloudFiles.inferColumnTypes", "true")
     .load("/Volumes/path/to/files")
  )
```

To create a Streaming table, the data set definition must be a stream type. When you use the `spark.readStream` function in a data set definition, it returns a streaming dataset.

Ingest streaming messages

You can also use streaming tables to ingest data from message buses. The following example demonstrates how to create a Streaming table that reads from a Pub/Sub topic.

Python SQL

Python

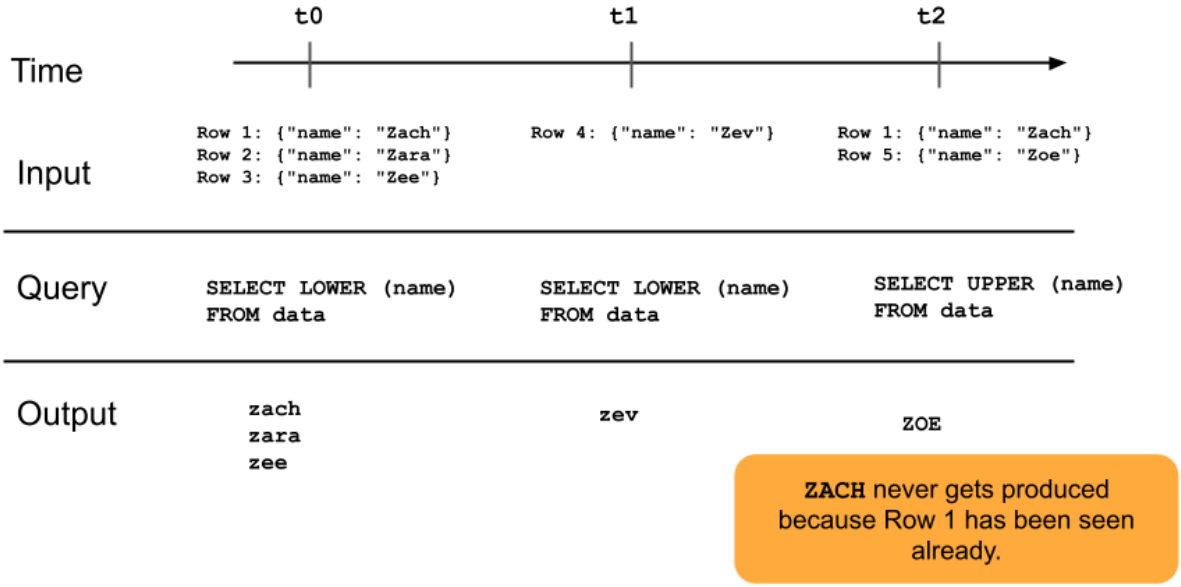
```python
@dp.table
def pubsub_raw():
  auth_options = {
    "clientId": client_id,
    "clientEmail": client_email,
    "privateKey": private_key,
    "privateKeyId": private_key_id
  }
  return (
    spark.readStream
      .format("pubsub")
      .option("subscriptionId", "my-subscription")
      .option("topicId", "my-topic")
```

```
        .option("projectId", "my-project")
        .options(auth_options)
        .load()
    )
```

Databricks recommends using secrets when providing authorization options. See Configure access to Pub/Sub for all authentication options.

For more details on loading data into streaming table, see Load data in pipelines.

The following diagram illustrates how append-only streaming tables work.



A row that has already been appended to a Streaming table will not be re-queried with later updates to the pipeline. If you modify the query (for example, from `SELECT LOWER (name)` to `SELECT UPPER (name)`), existing rows will not update to be uppercase, but new rows will be uppercase. You can trigger a full refresh to requery all previous data from the source table to update all rows in the Streaming table.

Streaming tables and low-latency streaming

Streaming tables are designed for low-latency streaming over bounded state. Streaming tables use checkpoint management, which makes them well-suited for low-

latency streaming. However, they expect streams that are naturally bounded or bounded with a watermark.

A naturally bounded stream is produced by a streaming data source that has a well-defined start and end. An example of a naturally bounded stream is reading data from a directory of files where no new files are being added after an initial batch of files is placed. The stream is considered bounded because the number of files is finite, and the stream ends after all of the files have been processed.

You can also use a watermark to bound a stream. A watermark in Structured Streaming is a mechanism that helps handle late data by specifying how long the system should wait for delayed events before considering the window of time as complete. An unbounded stream that does not have a watermark can cause a pipeline to fail due to memory pressure.

For more information about stateful stream processing, see Optimize stateful processing with watermarks.

Stream-snapshot joins

Stream-snapshot joins connect a streaming dataset to a dimension table that is snapshotted at stream start. Because the dimension table is treated as fixed at that point in time, any changes made to it after the stream starts are not reflected in the join. This is acceptable when small discrepancies don't matter — for example, when the number of transactions is many orders of magnitude larger than the number of customers.

The following code sample joins a dimension table with two rows called `customers` with an ever-increasing data set, `transactions`. It materializes a join between these two datasets in a table called `sales_report`. If an outside process updates the customers table by adding a new row (`customer_id=3, name=Zoya`), this new row will not be present in the join because the static dimension table was snapshotted when streams were started.

Python

```
from pyspark import pipelines as dp
```

Ask Genie

```python
@dp.temporary_view
# assume this table contains an append-only stream of rows about
transactions
# (customer_id=1, value=100)
# (customer_id=2, value=150)
# (customer_id=3, value=299)
# ... <and so on> ...
def v_transactions():
  return spark.readStream.table("transactions")

# assume this table contains only these two rows about customers
# (customer_id=1, name=Bilal)
# (customer_id=2, name=Olga)
@dp.temporary_view
def v_customers():
  return spark.read.table("customers")

@dp.table
def sales_report():
  facts = spark.readStream.table("v_transactions")
  dims = spark.read.table("v_customers")

  return facts.join(dims, on="customer_id", how="inner")
```

Streaming table limitations

Streaming tables have the following limitations:

- **Limited evolution:** You can change the query without recomputing the entire data set. Without a full refresh, a Streaming table only sees each row once, so different queries will have processed different rows. For example, if you add `UPPER()` to a field in the query, only rows processed after the change will be in uppercase. This means you must be aware of all previous versions of the query that are running on your data set. To reprocess existing rows that were processed prior to the change, a full refresh is required.
- **State management:** Streaming tables are low-latency and require streams that are naturally bounded or bounded with a watermark. For more information, see Optimize stateful processing with watermarks.
- **Joins don't recompute:** Joins in streaming tables do not recompute when dimensions change. This characteristic can be good for “fast-but-wrong” scenarios. If you want your view to always be correct, you might want to use a

Materialized view. Materialized views are always correct because they automatically recompute joins when dimensions change. For more information, see Materialized views.

Last updated on ***Mar 11, 2026***

Materialized views

Like standard views, materialized views are the results of a query and you access them the same way you would a table. Unlike standard views, which recompute results on every query, materialized views cache the results and refreshes them on a specified interval. Because a materialized view is precomputed, queries against it can run much faster than against regular views.

A materialized view is a declarative pipeline object. It includes a *query* that defines it, a flow to update it, and the cached results for fast access. A materialized view:

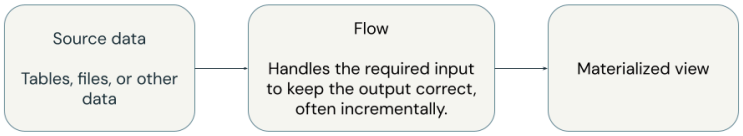
- Tracks changes in upstream data.
- On trigger, incrementally processes the changed data and applies the necessary transformations.
- Maintains the output table, in sync with the source data, based on a specified refresh interval.

Materialized views are a good choice for many transformations:

- You apply reasoning over cached results instead of rows. In fact, you simply write a query.
- They are always correct. All required data is processed, even if it arrives late or out of order.
- They are often incremental. Databricks will try to choose the appropriate strategy that minimizes the cost of updating a materialized view.

How materialized views work

The following diagram illustrates how materialized views work.



Materialized views are defined and updated by a single pipeline. You can explicitly define materialized views in the source code of the pipeline. Tables defined by a pipeline can't be changed or updated by any other pipeline.

> **NOTE**
>
> When you create a materialized view outside of a pipeline, using Databricks SQL, Databricks creates a pipeline which is used to update the view. You can see the pipeline by selecting **Jobs & Pipelines** from the left navigation in your workspace. You can add the **Pipeline type** column to your view. Materialized views defined in a pipeline have a type of `ETL`. Materialized views created in Databricks SQL have a type of `MV/ST`.

Databricks uses Unity Catalog to store metadata about the view, including the query and additional system views that are used for incremental updates. The cached data is materialized in cloud storage.

> **NOTE**
>
> Databricks creates internal tables to support materialized view incremental refresh. These tables appear in `system.information_schema.tables` but are not visible in Catalog Explorer or other workspace UI surfaces.

The following example joins two tables together and keeps the result up to date using a materialized view.

Python SQL

Python

```python
from pyspark import pipelines as dp

@dp.materialized_view
def regional_sales():
```

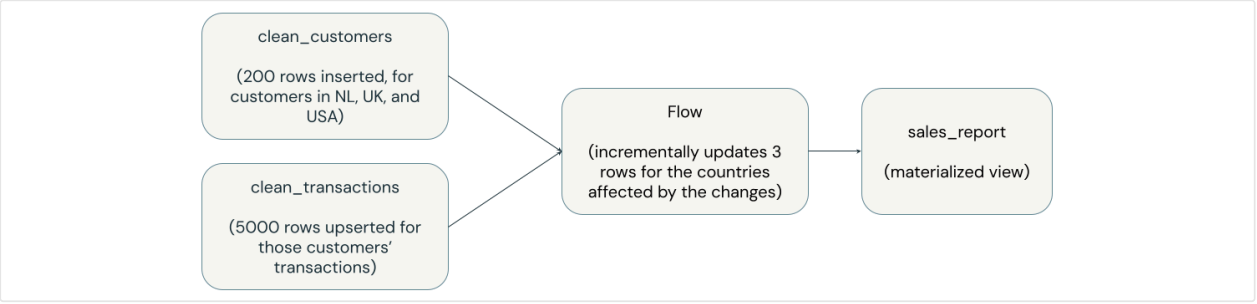
```
    partners_df = spark.read.table("partners")
    sales_df = spark.read.table("sales")

    return (
      partners_df.join(sales_df, on="partner_id", how="inner")
    )
```

Automatic incremental updates

When the pipeline defining a materialized view is triggered, the view is automatically kept up to date, often incrementally. Databricks attempts to process only the data that must be processed to keep the materialized view up to date. A materialized view always shows the correct result, even if it requires fully recomputing the query result from scratch, but often Databricks makes only incremental updates to a materialized view, which can be far less costly than a full recomputation.

The diagram below shows a materialized view called `sales_report`, which is the result of joining two upstream tables called `clean_customers` and `clean_transactions`, and grouping by country. An upstream process inserts 200 rows into `clean_customers` in three countries (USA, Netherlands, UK) and updates 5,000 rows in `clean_transactions` corresponding to these new customers. The `sales_report` materialized view is incrementally updated for only the countries that have new customers or corresponding transactions. In this example, we see three rows updated instead of the entire sales report.



For more details about how incremental refresh works in materialized views, see Incremental refresh for materialized views.

Materialized views limitations

Materialized views have the following limitations:

- Since updates create correct queries, some changes to inputs will require a full recomputation of a materialized view, which can be expensive.
- They are not designed for low-latency use cases. The latency of updating a materialized view is in the seconds or minutes, not milliseconds.
- Not all computations can be incrementally computed.
- Databricks attempts to detect when a UDF used in a materialized view changes behavior and perform a full refresh to apply the updated UDF. However, UDFs that call other functions or libraries may change behavior in ways that Databricks does not recognize. One example of this is when a called library is upgraded. When the behavior of a UDF changes, it is your responsibility to perform a full refresh on any materialized view that uses it.

*Last updated on **Jan 23, 2026***

Sinks in Lakeflow Spark Declarative Pipelines

By default when you create a flow, your pipeline writes the resulting query to a Delta table, typically a materialized view or a streaming table. Pipelines also provide functionality to let you write to a wide range of sinks, or even programmatically transform and stream data to any target (or targets) that you can write to with Python.

The following topics describe the sink functionality in pipelines.

Topic	Description
Lakeflow Spark Declarative Pipelines sinks	Use the `sink` API with flows to write records transformed by a pipeline to a supported external data sink. External data sinks include Unity Catalog managed and external tables, and event streaming services such as Apache Kafka or Azure Event Hubs.
Python custom sinks	Use the `sink` API with a Python custom data source to write to an arbitrary data store.
ForEachBatch sinks	Use the `foreachBatch` API to write to an arbitrary data store and perform other tranformations on the data or write to multiple sinks within a single flow.

More information

- Lakeflow Spark Declarative Pipelines
- Flows

this page

as this page helpful?

Yes

No

Send feedback

Ask Genie

*Last updated on **May 8, 2026***

Load data in pipelines

You can load data from any data source supported by Apache Spark on Databricks using pipelines. You can define datasets—tables and views—in Lakeflow Spark Declarative Pipelines against any query that returns a Spark DataFrame, including streaming DataFrames and Pandas for Spark DataFrames. For data ingestion tasks, Databricks recommends using streaming tables for most use cases. Streaming tables are useful for ingesting data from cloud object storage using Auto Loader or from message buses like Kafka.

Not all data sources have SQL support for ingestion. However, you can mix SQL and Python sources in the same pipeline to use Python where needed. For details on working with libraries not packaged in Lakeflow Spark Declarative Pipelines by default, see Manage Python dependencies for pipelines. For general information about ingestion in Databricks, see Standard connectors in Lakeflow Connect.

The following examples demonstrate some common data loading patterns.

Load from an existing table

Load data from any existing table in Databricks. You can transform the data using a query or load the table for further processing in your pipeline.

Python SQL

Python

```
@dp.table(
  comment="A table summarizing counts of the top baby names for New York
for 2021."
)
```

```python
def top_baby_names_2021():
  return (
    spark.read.table("baby_names_prepared")
      .filter(expr("Year_Of_Birth == 2021"))
      .groupBy("First_Name")
      .agg(sum("Count").alias("Total_Count"))
      .sort(desc("Total_Count"))
  )
```

Load files from cloud object storage

Databricks recommends using Auto Loader in pipelines for most data ingestion tasks from cloud object storage or from files in a Unity Catalog volume. Auto Loader and pipelines are designed to incrementally and idempotently load ever-growing data as it arrives in cloud storage. See What is Auto Loader? and Load data from object storage.

The following example reads data from cloud storage using Auto Loader.

Python SQL

Python

```python
@dp.table
def customers():
  return (
    spark.readStream.format("cloudFiles")
      .option("cloudFiles.format", "json")
      .load("s3://mybucket/analysis/*/*/*.json")
  )
```

The following examples use Auto Loader to create datasets from CSV files in a Unity Catalog volume.

Python SQL

Python

```
@dp.table
def customers():
  return (
    spark.readStream.format("cloudFiles")
      .option("cloudFiles.format", "csv")
      .load("/Volumes/my_catalog/retail_org/customers/")
  )
```

NOTE

- If you use Auto Loader with file notifications and run a full refresh for your pipeline or streaming table, you must manually clean up your resources. You can use the CloudFilesResourceManager in a notebook to perform cleanup.
- To load files with Auto Loader in a Unity Catalog enabled pipeline, you must use external locations. To learn more about using Unity Catalog with pipelines, see Use Unity Catalog with pipelines.

Authenticate to cloud storage

Auto Loader uses Unity Catalog external locations to authenticate against cloud storage. You must configure an external location for the storage path you want to read from and grant the `READ FILES` privilege to the executing user.

To ingest from Amazon S3, configure an external location backed by a storage credential that references an S3 bucket. For more information, see Connect to cloud object storage using Unity Catalog.

Load data from a message bus

You can configure pipelines to ingest data from message buses. Databricks recommends using streaming tables with continuous execution and enhanced autoscaling to provide the most efficient ingestion for low-latency loading from message buses. For more information, see Optimize the cluster utilization of Lakeflow Spark Declarative Pipelines with Autoscaling.

For example, the following code configures a streaming table to ingest data from Kafka using the read_kafka function.

Python SQL

Python

```python
from pyspark import pipelines as dp

@dp.table
def kafka_raw():
  return (
    spark.readStream
      .format("kafka")
      .option("kafka.bootstrap.servers", "kafka_server:9092")
      .option("subscribe", "topic1")
      .load()
  )
```

To ingest from other message bus sources, see:

- Kinesis: read_kinesis
- Pub/Sub topic: read_pubsub
- Pulsar: read_pulsar

Load data from Azure Event Hubs

Azure Event Hubs is a data streaming service that provides an Apache Kafka compatible interface. You can use the Structured Streaming Kafka connector, included in the Lakeflow Spark Declarative Pipelines runtime, to load messages from Azure Event Hubs. To learn more about loading and processing messages from Azure Event Hubs, see Use Azure Event Hubs as a pipeline data source.

Load data from external systems

Lakeflow Spark Declarative Pipelines supports loading data from any data source supported by Databricks. See Connect to data sources and external services. You can also load external data using Lakehouse Federation for supported data sources.

Because Lakehouse Federation requires Databricks Runtime 13.3 LTS or above, to use Lakehouse Federation, configure your pipeline to use the preview channel.

Some data sources don't have equivalent SQL support. If you cannot use Lakehouse Federation with one of these data sources, you can use Python to ingest data from the source. You can add Python and SQL source files to the same pipeline. The following example declares a materialized view to access the current state of data in a remote PostgreSQL table.

Python

```python
import dp

@dp.table
def postgres_raw():
  return (
    spark.read
      .format("postgresql")
      .option("dbtable", table_name)
      .option("host", database_host_url)
      .option("port", 5432)
      .option("database", database_name)
      .option("user", username)
      .option("password", password)
      .load()
  )
```

Load small or static datasets from cloud object storage

You can load small or static datasets using Apache Spark load syntax. Lakeflow Spark Declarative Pipelines supports all of the file formats supported by Apache Spark on Databricks. For a full list, see Data format options.

The following examples demonstrate loading JSON to create a table.

Python SQL

Python

```
@dp.table
def clickstream_raw():
  return (spark.read.format("json").load("/databricks-datasets/wikipedia-datasets/data-001/clickstream/raw-uncompressed-json/2015_2_clickstream.json"))
```

> **NOTE**
>
> The `read_files` SQL function is common to all SQL environments on Databricks. It is the recommended pattern for direct file access using SQL in pipelines. For more information, see Options.

Load data from a Python custom data source

Python custom data sources allow you to load data in custom formats. You can write code to read from and write to a specific external data source or use your existing Python code to read data from your own internal systems. For more detail about developing Python data sources, see PySpark custom data sources.

The following example registers a custom data source with the format name `my_custom_datasource` and reads from it in both batch and streaming modes.

Python

```
from pyspark import pipelines as dp

# Assume `my_custom_datasource` is a custom Python custom data
# source that supports both batch and streaming reads, and has
# been registered using `spark.dataSource.register`.

# This creates a materialized view
@dp.table(name = "read_from_batch")
def read_from_batch():
    return spark.read.format("my_custom_datasource").load()

# This creates a streaming table
```

```python
@dp.table(name = "read_from_streaming")
def read_from_streaming():
    return spark.readStream.format("my_custom_datasource").load()
```

Configure a streaming table to ignore changes in a source streaming table

By default, streaming tables require append-only sources. If your source streaming table requires updates or deletes—for example, for GDPR "right to be forgotten" processing—use the `skipChangeCommits` flag to ignore those changes. This flag works only with `spark.readStream` using the `option()` function and cannot be used when the source streaming table is the target of a create_auto_cdc_flow() function. For more information, see Handle changes to source Delta tables.

Python

```python
@dp.table
def b():
   return spark.readStream.option("skipChangeCommits", "true").table("A")
```

Securely access storage credentials with secrets in a pipeline

You can use Databricks secrets to store credentials such as access keys or passwords. To configure the secret in your pipeline, use a Spark property in the pipeline settings cluster configuration. See Configure classic compute for pipelines.

The following example uses a secret to store an access key required to read input data from an Azure Data Lake Storage storage account using Auto Loader. You can use this same method to configure any secret required by your pipeline, for example, AWS keys to access S3, or the password to an Apache Hive metastore.

To learn more about working with Azure Data Lake Storage, see Connect to Azure Data Lake Storage and Blob Storage.

> **NOTE**
>
> You must add the `spark.hadoop.` prefix to the `spark_conf` configuration key that sets the secret value.

JSON

```
{
  "id": "43246596-a63f-11ec-b909-0242ac120002",
  "storage": "abfss://<container-name>@<storage-account-
name>.dfs.core.windows.net/<path>",
  "clusters": [
    {
      "spark_conf": {
        "spark.hadoop.fs.azure.account.key.<storage-account-
name>.dfs.core.windows.net": "{{secrets/<scope-name>/<secret-name>}}"
      },
      "autoscale": {
        "min_workers": 1,
        "max_workers": 5,
        "mode": "ENHANCED"
      }
    }
  ],
  "development": true,
  "continuous": false,
  "libraries": [
    {
      "notebook": {
        "path": "/Users/user@databricks.com/:re[LDP] Notebooks/:re[LDP]
quickstart"
      }
    }
  ],
  "name": ":re[LDP] quickstart using ADLS2"
}
```

In this code sample, replace the following values.

Placeholder	Replace with
`<container-name>`	The name of the Azure storage account container.

Placeholder	Replace with
`<storage-account-name>`	The ADLS storage account name.
`<path>`	The path for pipeline output data and metadata.
`<scope-name>`	The Databricks secret scope name.
`<secret-name>`	The name of the key containing the Azure storage account access key.

Python

```
from pyspark import pipelines as dp

json_path = "abfss://<container-name>@<storage-account-name>.dfs.core.windows.net/<path-to-input-dataset>"
@dp.create_table(
  comment="Data ingested from an ADLS2 storage account."
)
def read_from_ADLS2():
  return (
    spark.readStream.format("cloudFiles")
      .option("cloudFiles.format", "json")
      .load(json_path)
  )
```

In this code sample, replace the following values.

Placeholder	Replace with
`<container-name>`	The name of the Azure storage account container that stores the input data.
`<storage-account-name>`	The ADLS storage account name.

Placeholder	Replace with
`<path-to-input-dataset>`	The path to the input dataset.

this page

as this page helpful?

Yes | No

Send feedback

Last updated on ***Apr 21, 2026***

Transform data with pipelines

This article describes how you can use pipelines to declare transformations on datasets and specify how records are processed through query logic. It also contains examples of common transformation patterns for building pipelines.

You can define a dataset against any query that returns a DataFrame. You can use Apache Spark built-in operations, UDFs, custom logic, and MLflow models as transformations in Lakeflow Spark Declarative Pipelines. After data has been ingested into your pipeline, you can define new datasets against upstream sources to create new streaming tables, materialized views, and views.

To learn how to effectively perform stateful processing in a pipeline, see Optimize stateful processing with watermarks.

When to use views, materialized views, and streaming tables

When implementing your pipeline queries, choose the best dataset type to ensure they are efficient and maintainable.

Consider using a view to do the following:

- Break a large or complex query that you want into easier-to-manage queries.
- Validate intermediate results using expectations.
- Reduce storage and compute costs for results you don't need to persist. Because tables are materialized, they require additional computation and storage resources.

Consider using a materialized view when:

- Multiple downstream queries consume the table. Because views are computed on demand, the view is re-computed every time the view is queried.
- Other pipelines, jobs, or queries consume the table. Because views are not materialized, you can only use them in the same pipeline.
- You want to view the results of a query during development. Because tables are materialized and can be viewed and queried outside of the pipeline, using tables during development can help validate the correctness of computations. After validating, convert queries that do not require materialization into views.

Consider using a streaming table when:

- A query is defined against a data source that is continuously or incrementally growing.
- Query results should be computed incrementally.
- The pipeline needs high throughput and low latency.

> **NOTE**
>
> Streaming tables are always defined against streaming sources. You can also use streaming sources with `AUTO CDC ... INTO` to apply updates from CDC feeds. See The AUTO CDC APIs: Simplify change data capture with pipelines.

Exclude tables from the target schema

If you must calculate intermediate tables not intended for external consumption, you can prevent them from being published to a schema using the `PRIVATE` keyword. Private tables still store and process data according to Lakeflow Spark Declarative Pipelines semantics but should not be accessed outside the current pipeline. A private table persists for the lifetime of the pipeline that creates it. Use the following syntax to declare private tables:

SQL Python

SQL

```
CREATE PRIVATE STREAMING TABLE private_table
AS SELECT ... ;
```

Combine streaming tables and materialized views in a single pipeline

Streaming tables inherit the processing guarantees of Apache Spark Structured Streaming and are configured to process queries from append-only data sources, where new rows are always inserted into the source table rather than modified.

> **NOTE**
>
> Although, by default, streaming tables require append-only data sources, when a streaming source is another streaming table that requires updates or deletes, you can override this behavior with the skipChangeCommits flag

A common streaming pattern involves ingesting source data to create the initial datasets in a pipeline. These initial datasets are commonly called bronze tables and often perform simple transformations.

By contrast, the final tables in a pipeline, commonly called gold tables, often require complicated aggregations or reading from targets of an `AUTO CDC ... INTO` operation. Because these operations inherently create updates rather than appends, they are not supported as inputs to streaming tables. These transformations are better suited for materialized views.

By mixing streaming tables and materialized views into a single pipeline, you can simplify your pipeline, avoid costly re-ingestion or re-processing of raw data, and have the full power of SQL to compute complex aggregations over an efficiently encoded and filtered dataset. The following example illustrates this type of mixed processing:

> **NOTE**
>
> These examples use Auto Loader to load files from cloud storage. To load files with Auto Loader in a Unity Catalog enabled pipeline, you must use external

locations. To learn more about using Unity Catalog with pipelines, see Use Unity Catalog with pipelines.

Python SQL

Python

```python
@dp.table
def streaming_bronze():
  return (
    # Since this is a streaming source, this table is incremental.
    spark.readStream.format("cloudFiles")
      .option("cloudFiles.format", "json")
      .load("s3://path/to/raw/data")
  )

@dp.table
def streaming_silver():
  # Since we read the bronze table as a stream, this silver table is also
  # updated incrementally.
  return spark.readStream.table("streaming_bronze").where(...)

@dp.materialized_view
def live_gold():
  # This table will be recomputed completely by reading the whole silver
table
  # when it is updated.
  return spark.read.table("streaming_silver").groupBy("user_id").count()
```

Learn more about using Auto Loader to incrementally ingest JSON files from S3.

Stream-static joins

Stream-static joins are a good choice when denormalizing a continuous stream of append-only data with a primarily static dimension table.

With each pipeline update, new records from the stream are joined with the most current snapshot of the static table. If records are added or updated in the static table after corresponding data from the streaming table has been processed, the resultant records are not recalculated unless a full refresh is performed.

In pipelines configured for triggered execution, the static table returns results as of the time the update started. In pipelines configured for continuous execution, the most recent version of the static table is queried each time the table processes an update.

The following is an example of a stream-static join:

Python SQL

Python

```
@dp.table
def customer_sales():
  return
spark.readStream.table("sales").join(spark.read.table("customers"),
["customer_id"], "left")
```

Calculate aggregates efficiently

You can use streaming tables to incrementally calculate simple distributive aggregates like count, min, max, or sum, and algebraic aggregates like average or standard deviation. Databricks recommends incremental aggregation for queries with a limited number of groups, such as a query with a `GROUP BY country` clause. Only new input data is read with each update.

To learn more about writing Lakeflow Spark Declarative Pipelines queries that perform incremental aggregations, see Perform windowed aggregations with watermarks.

Use MLflow models in Lakeflow Spark Declarative Pipelines

NOTE

To use MLflow models in a Unity Catalog-enabled pipeline, your pipeline must be configured to use the `preview` channel. To use the `current` channel, you must configure your pipeline to publish to the Hive metastore.

You can use MLflow-trained models in pipelines. MLflow models are treated as transformations in Databricks, meaning they act upon a Spark DataFrame input and return results as a Spark DataFrame. Because Lakeflow Spark Declarative Pipelines defines datasets against DataFrames, you can convert Apache Spark workloads that use MLflow into pipelines with just a few lines of code. For more on MLflow, see MLflow on Databricks.

If you already have a Python script calling an MLflow model, you can adapt this code to a pipeline by using the `@dp.table` or `@dp.materialized_view` decorator and ensuring functions are defined to return transformation results. Lakeflow Spark Declarative Pipelines does not install MLflow by default, so confirm that you have installed the MLFlow libraries with `%pip install mlflow` and have imported `mlflow` and `dp` at the top of your source. For an introduction to pipeline syntax, see Develop pipeline code with Python.

To use MLflow models in pipelines, complete the following steps:

1. Obtain the run ID and model name of the MLflow model. The run ID and model name are used to construct the URI of the MLflow model.
2. Use the URI to define a Spark UDF to load the MLflow model.
3. Call the UDF in your table definitions to use the MLflow model.

The following example shows the basic syntax for this pattern:

Python

```
%pip install mlflow

from pyspark import pipelines as dp
import mlflow

run_id= "<mlflow-run-id>"
model_name = "<the-model-name-in-run>"
model_uri = f"runs:/{run_id}/{model_name}"
loaded_model_udf = mlflow.pyfunc.spark_udf(spark, model_uri=model_uri)

@dp.materialized_view
def model_predictions():
  return spark.read.table(<input-data>)
    .withColumn("prediction", loaded_model_udf(<model-features>))
```

As a complete example, the following code defines a Spark UDF named `loaded_model_udf` that loads an MLflow model trained on loan risk data. The data columns used to make the prediction are passed as an argument to the UDF. The table `loan_risk_predictions` calculates predictions for each row in `loan_risk_input_data`.

Python

```
%pip install mlflow

from pyspark import pipelines as dp
import mlflow
from pyspark.sql.functions import struct

run_id = "mlflow_run_id"
model_name = "the_model_name_in_run"
model_uri = f"runs:/{run_id}/{model_name}"
loaded_model_udf = mlflow.pyfunc.spark_udf(spark, model_uri=model_uri)

categoricals = ["term", "home_ownership", "purpose",
  "addr_state","verification_status","application_type"]

numerics = ["loan_amnt", "emp_length", "annual_inc", "dti",
"delinq_2yrs",
  "revol_util", "total_acc", "credit_length_in_years"]

features = categoricals + numerics

@dp.materialized_view(
  comment="GBT ML predictions of loan risk",
  table_properties={
    "quality": "gold"
  }
)
def loan_risk_predictions():
  return spark.read.table("loan_risk_input_data")
    .withColumn('predictions', loaded_model_udf(struct(features)))
```

Retain manual deletes or updates

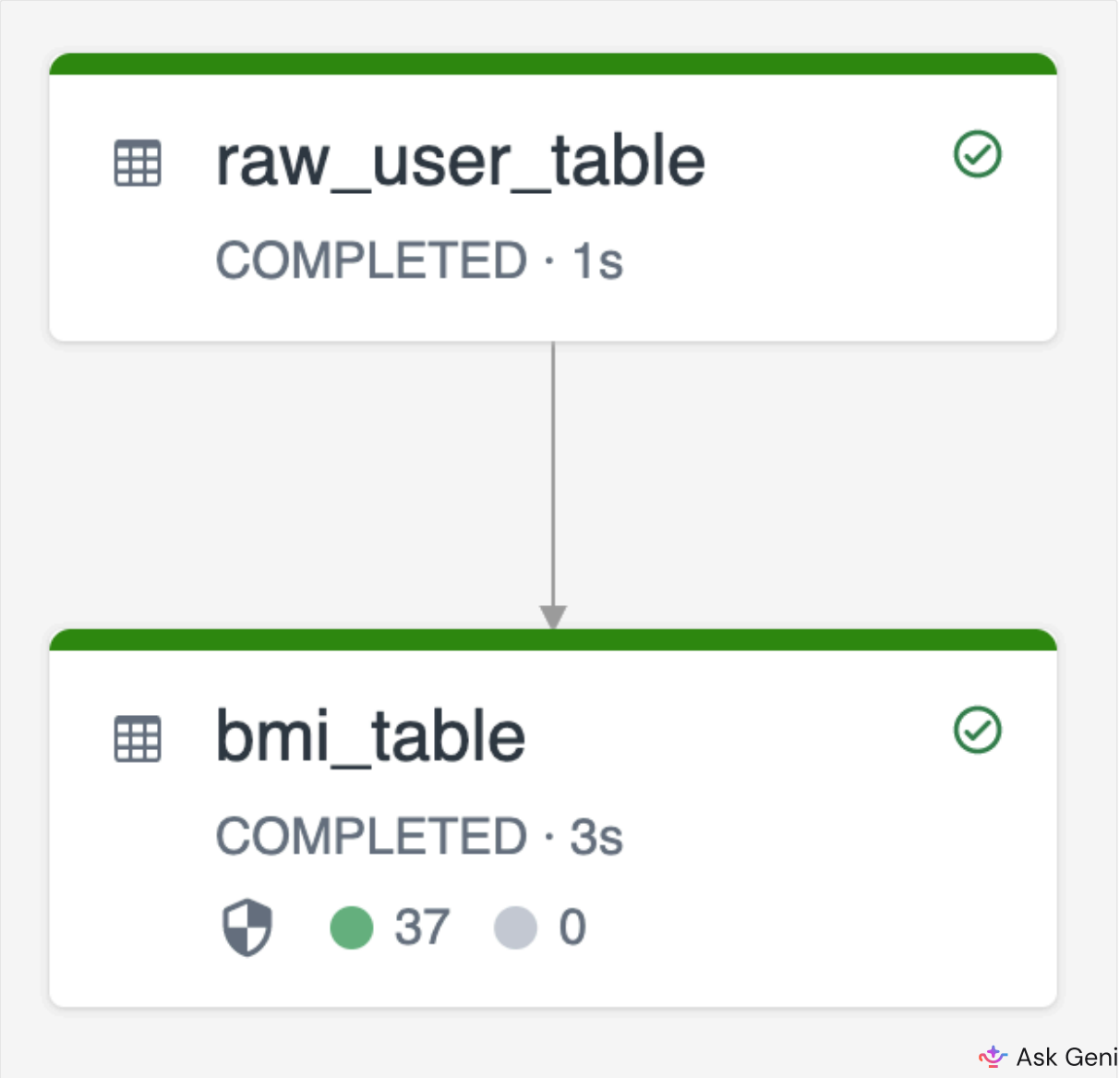
Lakeflow Spark Declarative Pipelines allows you to manually delete or update records from a table and do a refresh operation to recompute downstream tables.

By default, pipelines recompute table results based on input data each time they are updated, so you must ensure the deleted record isn't reloaded from the source data. Setting the `pipelines.reset.allowed` table property to `false` prevents refreshes to a table but does not prevent incremental writes to the tables or new data from flowing into the table.

The following diagram illustrates an example using two streaming tables:

- `raw_user_table` ingests raw user data from a source.
- `bmi_table` incrementally computes BMI scores using weight and height from `raw_user_table`.

You want to manually delete or update user records from the `raw_user_table` and recompute the `bmi_table`.



The following code demonstrates setting the `pipelines.reset.allowed` table property to `false` to disable full refresh for `raw_user_table` so that intended changes are retained over time, but downstream tables are recomputed when a pipeline update is run:

SQL

```
CREATE OR REFRESH STREAMING TABLE raw_user_table
TBLPROPERTIES(pipelines.reset.allowed = false)
AS SELECT * FROM STREAM read_files("/databricks-datasets/iot-stream/data-user", format => "csv");

CREATE OR REFRESH STREAMING TABLE bmi_table
AS SELECT userid, (weight/2.2) / pow(height*0.0254,2) AS bmi FROM STREAM(raw_user_table);
```

*Last updated on **Jan 23, 2026***

LIVE schema (legacy)

This article provides an overview of the legacy syntax and behavior for the `LIVE` virtual schema.

The `LIVE` virtual schema is a legacy feature of Lakeflow Spark Declarative Pipelines and is considered deprecated. You can still use legacy publishing mode and the `LIVE` virtual schema for pipelines that were created with this mode.

Databricks recommends migrating all pipelines to the new publishing mode. You have two choices for migration:

- Move tables (including materialized views and streaming tables) from a legacy pipeline to a pipeline that uses the default publishing mode. To learn how to move tables between pipelines, see Move tables between pipelines.
- Enable the default publishing mode in a pipeline that currently uses the legacy publishing mode. See Enable the default publishing mode in a pipeline.

Both of these methods are one-way migrations. You can't migrate tables back to the legacy mode.

Support for legacy `LIVE` virtual schema and legacy publishing mode will be removed in a future version of Databricks.

> **NOTE**
>
> Legacy publishing mode pipelines are indicated in the **Summary** field of the Lakeflow Spark Declarative Pipelines settings UI. You can also confirm a pipeline uses legacy publishing mode if the `target` field is set in the JSON specification for the pipeline.
>
> You cannot use the pipeline configuration UI to create new pipelines with the legacy publishing mode. If you need to deploy new pipelines using leg

syntax, contact your Databricks account representative.

What is the LIVE virtual schema?

> **NOTE**
>
> The `LIVE` virtual schema is no longer needed to analyze dataset dependency in the default publishing mode for Lakeflow Spark Declarative Pipelines.

The `LIVE` schema is a programming concept in Lakeflow Spark Declarative Pipelines that defines a virtual boundary for all datasets created or updated in a pipeline. By design, the `LIVE` schema is not tied directly to datasets in a published schema. Instead, the `LIVE` schema allows logic in a pipeline to be planned and run even if a user does not want to publish datasets to a schema.

In legacy publishing mode pipelines, you can use the `LIVE` keyword to reference other datasets in the current pipeline for reads, for example, `SELECT * FROM LIVE.bronze_table`. In the default publishing mode for new Lakeflow Spark Declarative Pipelines, this syntax is silently ignored, meaning that unqualified identifiers use the current schema. See Set the target catalog and schema.

Legacy publishing mode for pipelines

The `LIVE` virtual schema is used with the legacy publishing mode for Lakeflow Spark Declarative Pipelines. All tables created before February 5, 2025, use legacy publishing mode by default.

The following table describes the behavior for all materialized views and streaming tables created or updated in a pipeline in the legacy publishing mode:

Storage option	Storage location or catalog	Target schema	Behavior
Hive metastore	None specified	None specified	Dataset metadata and data are stored to the DBFS root. No database objects are registered to the Hive metastore.
Hive metastore	A URI or file path to cloud object storage.	None specified	Dataset metadata and data are stored to the specified storage location. No database objects are registered to the Hive metastore.
Hive metastore	None specified	An existing or new schema in the Hive metastore.	Dataset metadata and data are stored to the DBFS root. All materialized views and streaming tables in the pipeline are published to the specified schema in Hive metastore.
Hive metastore	A URI or file path to cloud object storage.	An existing or new schema in the Hive metastore.	Dataset metadata and data are stored to the specified storage location. All materialized views and streaming tables in the pipeline are published to the specified schema in Hive metastore.
Unity Catalog	An existing Unity Catalog catalog.	None specified	Dataset metadata and data are stored in the default storage location associated with the target catalog. No database objects are registered to the Unity Catalog.
Unity Catalog	An existing Unity	An existing or new schema	Dataset metadata and data are stored in the default storage

Storage option	Storage location or catalog	Target schema	Behavior
	Catalog catalog.	in Unity Catalog.	location associated with the target schema or catalog. All materialized views and streaming tables in the pipeline are published to the specified schema in Unity Catalog.

Update source code from LIVE schema

Pipelines configured to run with the new default publishing mode silently ignore the `LIVE` schema syntax. By default, all table reads use the catalog and schema specified in the pipeline configuration.

For most existing pipelines, this behavior change has no impact, as the legacy `LIVE` virtual schema behavior also directs reads to the catalog and schema specified in the pipeline configuration.

IMPORTANT

Legacy code with reads that leverage the workspace default catalog and schema require code updates. Consider the following materialized view definition:

SQL

```
CREATE MATERIALIZED VIEW silver_table
AS SELECT * FROM raw_data
```

In legacy publishing mode, an unqualified read from the `raw_data` table uses the workspace default catalog and schema, for example, `main.default.raw_data`. In the new default pipeline mode, the catalog and schema used by default are those configured in the pipeline configuration. To ensure that this code continues to

work as expected, update the reference to use the fully qualified identifier for the table, as in the following example:

SQL

```
CREATE MATERIALIZED VIEW silver_table
AS SELECT * FROM main.default.raw_data
```

Work with event log for Unity Catalog legacy publishing mode pipelines

IMPORTANT

The `event_log` TVF is available for legacy publishing mode pipelines that publish tables to Unity Catalog. Default behavior for new pipelines publishes the event log to the target catalog and schema configured for the pipeline. See Query the event log.

Tables configured with Hive metastore also have different event log support and behavior. See Work with event log for Hive metastore pipelines.

If your pipeline publishes tables to Unity Catalog with legacy publishing mode, you must use the `event_log` *table valued function* (TVF) to fetch the event log for the pipeline. You retrieve the event log for a pipeline by passing the pipeline ID or a table name to the TVF. For example, to retrieve the event log records for the pipeline with ID `04c78631-3dd7-4856-b2a6-7d84e9b2638b`:

SQL

```
SELECT * FROM event_log("04c78631-3dd7-4856-b2a6-7d84e9b2638b")
```

To retrieve the event log records for the pipeline that created or owns the table `my_catalog.my_schema.table1`:

SQL

```
SELECT * FROM event_log(TABLE(my_catalog.my_schema.table1))
```

To call the TVF, you must use a shared cluster or a SQL warehouse. For example, you can use the SQL editor connected to a SQL warehouse.

To simplify querying events for a pipeline, the owner of the pipeline can create a view over the `event_log` TVF. The following example creates a view over the event log for a pipeline. This view is used in the example event log queries included in this article.

> **NOTE**
>
> - The `event_log` TVF can be called only by the pipeline owner.
> - You cannot use the `event_log` table valued function in a pipeline or query to access the event logs of multiple pipelines.
> - You cannot share a view created over the `event_log` table valued function with other users.

SQL

```
CREATE VIEW event_log_raw AS SELECT * FROM event_log("<pipeline-ID>");
```

Replace `<pipeline-ID>` with the unique identifier for the pipeline. You can find the ID in the **Pipeline details** panel in the Lakeflow Spark Declarative Pipelines UI.

Each instance of a pipeline run is called an *update*. You often want to extract information for the most recent update. Run the following query to find the identifier for the most recent update and save it in the `latest_update_id` temporary view. This view is used in the example event log queries included in this article:

SQL

```
CREATE OR REPLACE TEMP VIEW latest_update AS SELECT origin.update_id AS
id FROM event_log_raw WHERE event_type = 'create_update' ORDER BY
timestamp DESC LIMIT 1;
```

*Last updated on **May 8, 2026***

Configure Pipelines

This page describes the basic configuration for pipelines using the workspace UI.

Databricks recommends developing new pipelines using serverless. For configuration instructions for serverless pipelines, see Configure a serverless pipeline.

The configuration instructions in this page use Unity Catalog. For instructions for configuring pipelines with legacy Hive metastore, see Use Lakeflow Spark Declarative Pipelines with legacy Hive metastore.

This page discusses functionality for the current default publishing mode for pipelines. Pipelines created before February 5, 2025 might use the legacy publishing mode and `LIVE` virtual schema. See LIVE schema (legacy).

> **NOTE**
>
> The UI has an option to display and edit settings in JSON. You can configure most settings with either the UI or a JSON specification. Some advanced options are only available using the JSON configuration.
>
> JSON configuration files are also helpful when deploying pipelines to new environments or using the CLI or REST API.
>
> For a complete reference to the pipeline JSON configuration settings, see Pipeline configurations.

Configure a new pipeline

To configure a new pipeline, do the following:

1. At the top of the sidebar, click + **New** and then select **ETL pipeli**

2. At the top, give your pipeline a unique name.

3. Under the name, you can see the default catalog and schema that have been chosen for you. Change these to give your pipeline different defaults.

   The default catalog and the default schema are where datasets are read from or written to when you do not qualify datasets with a catalog or schema in your code. See Database objects in Databricks for more information.

4. Select your preferred option to create a pipeline:

   - **Start with sample code in SQL** to create a new pipeline and folder structure, including sample code in SQL.
   - **Start with sample code in Python** to create a new pipeline and folder structure, including sample code in Python.
   - **Start with a single transformation** to create a new pipeline and folder structure, with a new blank code file.
   - **Add existing assets** to create a pipeline that you can associate with existing code files in your workspace.
   - **Create a source-controlled project** to create a pipeline with a new Declarative Automation Bundles project, or to add the pipeline to an existing bundle.

   You can have both SQL and Python source code files in your ETL pipeline. When creating a new pipeline and choosing a language for the sample code, the language is only for the sample code included in your pipeline by default.

5. When you make your selection, you are redirected to the newly created pipeline.

   The ETL pipeline is created with the following default settings:

   - Unity Catalog
   - Current channel
   - Serverless compute

   This configuration is recommended for many use cases, including development and testing, and is well-suited to production workloads that should run on a schedule. For details on scheduling pipelines, see Pipeline task for jobs.

You can adjust these settings from the pipeline toolbar.

Alternatively, you can create an ETL pipeline from the workspace browser:

1. Click **Workspace** in the left side pane.
2. Select any folder, including Git folders.
3. Click **Create** in the upper-right corner, and click **ETL pipeline**.

You can also create an ETL pipeline from the jobs and pipelines page:

1. In your workspace, click **Jobs & Pipelines** in the sidebar.
2. Under **New**, click **ETL Pipeline**.

Compute configuration options

Databricks recommends always using **Enhanced autoscaling**. Default values for other compute configurations work well for many pipelines.

Serverless pipelines remove compute configuration options. For configuration instructions for serverless pipelines, see Configure a serverless pipeline.

Use the following settings to customize compute configurations:

- Workspace admins can configure a **Cluster policy**. Compute policies allow admins to control what compute options are available to users. See Select a compute policy.
- You can optionally configure **Cluster mode** to run with **Fixed size** or **Legacy autoscaling**. See Optimize the cluster utilization of Lakeflow Spark Declarative Pipelines with Autoscaling.
- For workloads with autoscaling enabled, set **Min workers** and **Max workers** to set limits for scaling behaviors. See Configure classic compute for pipelines.
- You can optionally turn off Photon acceleration. See What is Photon?.
- Select an **Instance profile** if your pipeline uses an instance profile to control access to cloud services. See Cloud storage configuration.
- Use **Cluster tags** to help monitor costs associated with pipelines. See Configure compute tags.

- Configure **Instance types** to specify the type of virtual machines used to run your pipeline. See Select instance types to run a pipeline.
  - Select a **Worker type** optimized for the workloads configured in your pipeline.
  - You can optionally select a **Driver type** that differs from your worker type. This can be useful for reducing costs in pipelines with large worker types and low driver compute utilization or for choosing a larger driver type to avoid out-of-memory issues in workloads with many small workers.

Set the run-as user

Run-as user allows you to change the identity that a pipeline uses to run, and the ownership of the tables it creates or updates. This is useful in situations where the original user who created the pipeline has been deactivated—for example, if they left the company. In those cases, the pipeline can stop working, and the tables it published can become inaccessible to others. By updating the pipeline to run as a different identity—such as a service principal—and reassigning ownership of the published tables, you can restore access and ensure the pipeline continues to function. Running pipelines as service principals is considered a best practice because they are not tied to individual users, making them more secure, stable, and reliable for automated workloads.

Required permissions

For the user making the change:

- **CAN_MANAGE** permissions on the pipeline
- **CAN_USE** role on the service principal (if setting run-as to a service principal)

For the run-as user or service principal:

- **Workspace Access:**
  - **Workspace access** permission to operate within the workspace
  - **Can use** permission on cluster policies used by the pipeline
  - Compute creation permission in the workspace
- **Source Code Access:**

    - **Can read** permission on all notebooks included in the pipeline source code
    - **Can read** permission on workspace files if the pipeline uses them

- **Unity Catalog Permissions** (for pipelines using Unity Catalog):
    - `USE CATALOG` on the target catalog
    - `USE SCHEMA` and `CREATE TABLE` on the target schema
    - `MODIFY` permission on existing tables that the pipeline updates
    - `CREATE SCHEMA` permission if the pipeline creates new schemas

- **Legacy Hive metastore Permissions** (for pipelines using Hive metastore):
    - `SELECT` and `MODIFY` permissions on target databases and tables

- **Additional Cloud Storage Access** (if applicable):
    - Permissions to read from source storage locations
    - Permissions to write to target storage locations

How to set the run-as user

You can set the `run-as` user through the pipeline settings from the pipeline monitoring page or the pipeline editor. To change the user from the pipeline monitoring page:

1. Click **Jobs & Pipelines** to open the list of pipelines, and select the name of the pipeline you wish to edit.
2. On the pipline monitoring page, click **Settings**.
3. In the **Pipeline settings** sidebar, click **Edit** next to **Run as**.
4. In the edit widget, select one of the following options:
    - Your own user account
    - A service principal for which you have **CAN_USE** permission
5. Click **Save** to apply the changes.

When you successfully update the run-as user:

- The pipeline identity changes to use the new user or service principal for all future runs

- In Unity Catalog pipelines, the owner of tables published by the pipeline is updated to match the new run-as identity
- Future pipeline updates use the permissions and credentials of the new run-as identity
- Continuous pipelines automatically restart with the new identity. Triggered pipelines do not automatically restart, and the run-as change can interrupt an active update

> **NOTE**
>
> If the update of run-as fails, you receive an error message explaining the reason for the failure. Common issues include insufficient permissions on the service principal.

Other configuration considerations

The following configuration options are also available for pipelines:

- The **Advanced** product edition gives you access to all Lakeflow Spark Declarative Pipelines features. You can optionally run pipelines using the **Pro** or **Core** product editions. See Choose a product edition.
- You might choose to use the **Continuous** pipeline mode when running pipelines in production. See Triggered vs. continuous pipeline mode.
- If your workspace is not configured for Unity Catalog or your workload needs to use legacy Hive metastore, see Use Lakeflow Spark Declarative Pipelines with legacy Hive metastore.
- Add **Notifications** for email updates based on success or failure conditions. See Add email notifications for pipeline events.
- Use the **Configuration** field to set key-value pairs for the pipeline. These configurations serve two purposes:
  - Set arbitrary parameters you can reference in your source code. See Use parameters with pipelines.
  - Configure pipeline settings and Spark configurations. See Pipeline properties reference.
  - Configure **Tags**. Tags are key-value pairs for the pipeline that are visible in the Jobs & Pipelines list. Pipeline tags are not associated with billing.

- Use the **Preview** channel to test your pipeline against pending Lakeflow Spark Declarative Pipelines runtime changes and trial new features.

Choose a product edition

Select the Lakeflow Spark Declarative Pipelines product edition with the best features for your pipeline requirements. The following product editions are available:

- `Core` to run streaming ingest workloads. Select the `Core` edition if your pipeline doesn't require advanced features such as change data capture (CDC) or Lakeflow Spark Declarative Pipelines expectations.
- `Pro` to run streaming ingest and CDC workloads. The `Pro` product edition supports all of the `Core` features, plus support for workloads that require updating tables based on changes in source data.
- `Advanced` to run streaming ingest workloads, CDC workloads, and workloads that require expectations. The `Advanced` product edition supports the features of the `Core` and `Pro` editions and includes data quality constraints with Lakeflow Spark Declarative Pipelines expectations.

You can select the product edition when you create or edit a pipeline. You can choose a different edition for each pipeline. See the Lakeflow Spark Declarative Pipelines product page.

The product edition also determines how long past pipeline updates remain visible in the pipeline UI and the Pipelines API:

Edition	Past update retention
`Core`	5 days
`Pro`	30 days
`Advanced`	30 days

Active (non-terminal) updates always appear regardless of edition. If an update is older than the retention window, Databricks excludes it from list responses but keeps it in the underlying event log.

Note: If your pipeline includes features not supported by the selected product edition, such as expectations, you receive an error message explaining the reason for the error. You can then edit the pipeline to select the appropriate edition.

Configure source code

You can use the asset browser in the Lakeflow Pipelines Editor to configure the source code defining your pipeline. Pipeline source code is defined in SQL or Python scripts stored in workspace files. When you create or edit your pipeline, you can add one or more files. By default, pipeline source code is located in the `transformations` folder in your pipeline's root folder.

Because Lakeflow Spark Declarative Pipelines automatically analyzes dataset dependencies to construct the processing graph for your pipeline, you can add source code assets in any order.

For more details on using the Lakeflow Pipelines Editor, see Develop and debug ETL pipelines with the Lakeflow Pipelines Editor.

Manage external dependencies for pipelines that use Python

Pipelines support using external dependencies in your pipelines, such as Python packages and libraries. To learn about options and recommendations for using dependencies, see Manage Python dependencies for pipelines.

Use Python modules stored in your Databricks workspace

In addition to implementing your Python code in pipeline source code files, you can use Databricks Git Folders or workspace files to store your code as Python modules. Storing your code as Python modules is especially useful when you have common functionality you want to use in multiple pipelines or notebooks in the same pipeline.

To learn how to use Python modules with your pipelines, see Import Python modules from Git folders or workspace files.

Last updated on ***May 12, 2026***

Run a pipeline update

This article explains pipeline updates and provides details on how to trigger an update.

What is a pipeline update?

After you create a pipeline and are ready to run it, you start an *update*. A pipeline update does the following:

- Starts a cluster with the correct configuration.
- Discovers all the defined tables and views and checks for any analysis errors such as not valid column names, missing dependencies, and syntax errors.
- Creates or updates tables and views with the most recent data available.

Using a dry run, you can check for problems in a pipeline's source code without waiting for tables to be created or updated. This feature is useful when developing or testing pipelines because it lets you quickly find and fix errors in your pipeline, such as incorrect table or column names.

How are pipeline updates triggered?

Use one of the following options to start pipeline updates:

Update trigger	Details
Manual	You can manually trigger pipeline updates from the Lakeflow Pipelines Editor, or the pipelines list. See Manually trigger a pipeline update.

Update trigger	Details
Scheduled	You can schedule updates for pipelines using jobs. See Pipeline task for jobs.
Programmatic	You can programmatically trigger updates using third-party tools, APIs, and CLIs. See Run pipelines in a workflow and pipeline REST API.

Manually trigger a pipeline update

Use one of the following options to manually trigger a pipeline update:

- Run the full pipeline, or a subset of the pipeline (a single source file, or a single table), from the Lakeflow Pipelines Editor. For more information, see Run pipeline code.
- Run the full pipeline from the **Jobs & Pipelines** list. Click ▶ in the same row as the pipeline in the list.
- From the pipeline monitoring page, click the 

button.

> **NOTE**
> The default behavior for manually triggered pipeline updates is to refresh all datasets defined in the pipeline.

Pipeline refresh semantics

The following table describes the default refresh, full refresh, and reset checkpoints behavior for materialized views and streaming tables:

Update type	Materialized view	Streaming table
Refresh (default)	Updates results to reflect the current results for the defining query. It examines the costs, and performs an incremental refresh if it is more cost-efficient. See Incremental refresh for materialized views	Processes new records through logic defined in streaming tables and flows.
Full refresh	Updates results to reflect the current results for the defining query.	Clears data from streaming tables, clears state information (checkpoints) from flows, and reprocesses all records from the data source. See Full refresh for streaming tables
Reset streaming flow checkpoints	Not applicable to materialized views.	Clears state information (checkpoints) from flows but does not clear data from streaming tables, and reprocesses all records from the data source.

By default, all materialized views and streaming tables in a pipeline refresh with each update. You can optionally omit tables from updates using the following features:

- **Select tables for refresh**: Use this UI to add or remove materialized views and streaming tables before running an update. See Start a pipeline update for selected tables.
- **Refresh failed tables**: Start an update for failed materialized views and streaming tables, including downstream dependencies. See Start a pipeline update for failed tables.

Both of these features support default refresh semantics or full refresh. You can optionally use the **Select tables for refresh** dialog to exclude additional tables when

running a refresh for failed tables.

For streaming tables, you can choose to clear the streaming checkpoints for selected flows and not the data from the associated streaming tables. To clear the checkpoints for selected flows, use the Databricks REST API to start a refresh. See Start a pipeline update to clear selective streaming flows' checkpoints.

Should I use a full refresh?

Databricks recommends running full refreshes only when necessary. A full refresh always reprocesses all records from the specified data sources through the logic that defines the dataset. The time and resources to complete a full refresh are correlated to the size of the source data.

Materialized views return the same results whether default or full refresh is used. Using a full refresh with streaming tables resets all state processing and checkpoint information and can result in dropped records if input data is no longer available. See Full refresh for streaming tables

Databricks only recommends full refresh when the input data sources contain the data needed to recreate the desired state of the table or view. Consider the following scenarios where input source data is no longer available and the outcome of running a full refresh:

Data source	Reason input data is absent	Outcome of full refresh
Kafka	Short retention threshold	Records no longer present in the Kafka source are dropped from the target table.
Files in object storage	Lifecycle policy	Data files no longer present in the source directory are dropped from the target table.
Records in a table	Deleted for compliance	Only records present in the source table are processed.

To prevent full refreshes from being run on a table or view, set the table property `pipelines.reset.allowed` to `false`. See Pipeline table properties. You can also use an append flow to append data to an existing streaming table without requiring a full refresh.

Start a pipeline update for selected tables

You can optionally reprocess data for only selected tables in your pipeline. For example, during development, you only change a single table and want to reduce testing time, or a pipeline update fails and you want to refresh only the failed tables.

The Lakeflow Pipelines Editor has options for reprocessing a source file, selected tables, or a single table. For details, see Run pipeline code.

Start a pipeline update for failed tables

If a pipeline update fails because of errors in one or more tables in the pipeline graph, you can start an update of only failed tables and any downstream dependencies.

> **NOTE**
> Excluded tables are not refreshed, even if they depend on a failed table.

To update failed tables, on the pipeline monitoring page, click **Refresh failed tables**.

To update only selected failed tables from the pipeline monitoring page:

1. Click ▾ next to the **Refresh failed tables** button and click **Select tables for refresh**. The **Select tables for refresh** dialog appears.
2. To select the tables to refresh, click each table. The selected tables are highlighted and labeled. To remove a table from the update, click the table again.
3. Click **Refresh selection**.

> **NOTE**
>
> The **Refresh selection** button displays the number of selected tables in parentheses.

To reprocess data already ingested for the selected tables, click  next to the **Refresh selection** button and click **Full Refresh selection**.

Start a pipeline update to clear selective streaming flows' checkpoints

You can optionally reprocess data for selected streaming flows in your pipeline without clearing any already ingested data.

> **NOTE**
>
> Flows that are not selected are run using a REFRESH update. You can also specify `full_refresh_selection` or `refresh_selection` to selectively refresh other tables.

To start an update to refresh the selected streaming checkpoints, use the updates request in the Lakeflow Spark Declarative Pipelines REST API.

The `reset_checkpoint_selection` parameter accepts a list of flow names. You must use the flow name as it appears in the pipeline graph:

- If you defined a flow with an explicit name (for example, using the `flow_name` parameter in `create_auto_cdc_flow`), use that name.
- If you did not set an explicit flow name, the default flow name is the fully qualified table name in `catalog.schema.table` format. Using only the table name (for example, `gold` instead of `my_catalog.my_schema.gold`) causes the pipeline update to fail with an `IllegalArgumentException`.

You can find flow names in the pipeline UI or in the pipeline event logs.

The following example uses the `curl` command to call the `updates` request to start a pipeline update:

Bash

```
curl -X POST \
-H "Authorization: Bearer <your-token>" \
-H "Content-Type: application/json" \
-d '{
"reset_checkpoint_selection": ["my_catalog.my_schema.my_streaming_table"]
}' \
https://<your-databricks-instance>/api/2.0/pipelines/<your-pipeline-id>/updates
```

The following example resets the checkpoint for a flow with a custom name:

Bash

```
curl -X POST \
-H "Authorization: Bearer <your-token>" \
-H "Content-Type: application/json" \
-d '{
"reset_checkpoint_selection": ["my_custom_flow_name"]
}' \
https://<your-databricks-instance>/api/2.0/pipelines/<your-pipeline-id>/updates
```

Check a pipeline for errors without waiting for tables to update

PREVIEW

The pipeline `Dry run` feature is in Public Preview.

To check whether a pipeline's source code is valid without running a full update, use a *dry run*. A dry run resolves the definitions of datasets and flows defined in the pipeline but does not materialize or publish any datasets. Errors found during the dry run, such as incorrect table or column names, are reported in the UI.

To start a dry run, click  on the pipeline details page next to **Start** and click **Dry run**.

After the dry run is complete, any errors are shown in the event tray in the bottom panel. Clicking the event tray will display any issues found in the bottom panel. Additionally, the event log shows events related only to the dry run, and no metrics are displayed in the pipeline graph. If errors are found, details are available in the event log.

You can see results for only the most recent dry run. If the dry run was the most recently run update, you can see the results by selecting it in the update history. If another update is run after the dry run, the results are no longer available in the UI.

Update run behavior

The behavior of a pipeline update is determined by how you trigger it:

- **Updates triggered from the pipeline monitoring UI** using **Run now** use fast-start, debugging-focused behavior.
- **Updates triggered from Jobs, the Pipelines API, or continuous pipelines** use automatic retry and restart behavior.

For triggered pipelines, you can override the default behavior for a specific run by choosing **Run now with different settings** from the drop-down in the Lakeflow Pipelines Editor or the pipeline monitoring page.

Fast-start, debugging-focused behavior

Used for UI **Run now** and ad-hoc updates. These runs optimize for rapid iteration:

- Reuses a cluster to avoid the overhead of restarts. By default, clusters run for two hours. You can change this with the `pipelines.clusterShutdown.delay` setting in the Configure classic compute for pipelines.
- Disables pipeline retries so you can immediately detect and fix errors.

Automatic retry and restart behavior

Used for Jobs, API-triggered updates, and continuous pipelines. These runs prioritize reliability and cost efficiency:

- Restarts the cluster for specific recoverable errors, including memory leaks and stale credentials.
- Retries execution in the event of specific errors, such as a failure to start a cluster.
- The cluster shuts down immediately after the run completes.

> **NOTE**
>
> Run behavior only controls cluster and pipeline execution. Storage locations and target schemas in the catalog for publishing tables must be configured as part of pipeline settings and are not affected by run behavior.

Last updated on ***Feb 4, 2026***

Use `ALTER` statements with pipeline datasets

Lakeflow Spark Declarative Pipelines (SDP) defines pipelines in source code that is specific to SDP. You can edit pipeline source in either SQL or Python, for example, in the Lakeflow Pipelines Editor.

Lakeflow Connect creates pipelines that ingest data, and create ingestion streaming tables.

Databricks also provides a SQL environment called Databricks SQL. You can create materialized views and streaming tables with Databricks SQL using pipeline functionality outside of SDP (see Standalone pipelines). Typically, Databricks SQL is not used with Lakeflow Spark Declarative Pipelines.

However you can use `ALTER` SQL statements in Databricks SQL to modify the properties of a dataset created with either SDP, Databricks SQL, or Lakeflow Connect. Use these SQL statements from any Databricks SQL environment, whether you are modifying SDP datasets, Databricks SQL pipeline datasets, or Lakeflow Connect datasets.

- Streaming tables - ALTER STREAMING TABLE
- Materialized views - ALTER MATERIALIZED VIEW

> **NOTE**
>
> You can't modify the schedule or trigger of a dataset defined in SDP with an `ALTER` statement.

Limitation: Pipeline updates and changes made with `ALTER`

There are cases where `ALTER` statements conflict with the definition of the pipeline-created datasets. The SQL that defines a table or view in a pipeline is re-run on each update. This can undo changes you make with an `ALTER` statement.

For example, if you have a SQL statement that defines a materialized view, like the following:

SQL

```
CREATE OR REPLACE MATERIALIZED VIEW masked_view (
    id int,
    name string,
    region string,
    ssn string MASK catalog.schema.ssn_mask_fn
  )
  WITH ROW FILTER catalog.schema.us_filter_fn ON (region)
  AS SELECT id, name, region, ssn
       FROM employees;
```

Then you try to remove the mask from the `ssn` column using an `ALTER` statement, like this:

SQL

```
ALTER MATERIALIZED VIEW masked_view ALTER COLUMN ssn DROP MASK;
```

The mask is removed, but the next time the materialized view is updated the SQL definition adds it back.

To safely remove the mask, you must edit the SQL definition to remove the mask and then run the `ALTER` command to `DROP` the mask.

> **NOTE**
>
> To edit the definition of a pipeline defined in SDP, edit your pipeline source using the pipeline editor. To edit the definition of a pipeline defined in Databricks SQL,

run the modified SQL statement in any Databricks SQL environment.

Additional resources

- Standalone pipelines
- Develop Lakeflow Spark Declarative Pipelines
- Develop Lakeflow Spark Declarative Pipelines code with SQL

Last updated on ***Feb 3, 2026***

Move tables between pipelines

This article describes how to move streaming tables and materialized views between pipelines. After the move, the pipeline you move the flow to updates the table, rather than the original pipeline. This is useful in many scenarios, including:

- Split a large pipeline into smaller ones.
- Merge multiple pipelines in a single larger one.
- Change the refresh frequency for some tables in a pipeline.
- Move tables from a pipeline that uses the legacy publishing mode to the default publishing mode. For details about the legacy publishing mode, see Legacy publishing mode for pipelines. To see how you can migrate the publishing mode for an entire pipeline at once, see Enable the default publishing mode in a pipeline.
- Move tables across pipelines in different workspaces.

Requirements

The following are requirements for moving a table between pipelines.

- You must use Databricks Runtime 16.3 or above when running the `ALTER ...` command, and Databricks Runtime 17.2 for cross-workspace table move.
- Both source and destination pipelines must be in workspaces that share a metastore. To check the metastore, see `current_metastore` function.
- The user account or service principal running the operation must be the run-as user of both source and destination pipeline

- The destination pipeline must use the default publishing mode. This enables you to publish tables to multiple catalogs and schemas.

  Alternately, both pipelines must use the legacy publishing mode and both must have the same catalog and target value in settings. For information about the legacy publishing mode, see LIVE schema (legacy).

> **NOTE**
>
> This feature does not support moving a pipeline using the default publishing mode to a pipeline using the legacy publishing mode.

Move a table between pipelines

The following instructions describe how to move a streaming table or materialized view from one pipeline to another.

1. Stop the source pipeline if it is running. Wait for it to completely stop.

2. Remove the table's definition from the source pipeline's code and store it somewhere for future reference.

   Include any supporting queries or code that is needed for the pipeline to run correctly.

3. From a notebook or a SQL editor, run the following SQL command to reassign the table from the source pipeline to the destination pipeline:

   SQL

   ```
   ALTER [MATERIALIZED VIEW | STREAMING TABLE | TABLE] <table-name>
     SET TBLPROPERTIES("pipelines.pipelineId"="<destination-pipeline-
   id>");
   ```

   Note that the SQL command must be run from the source pipeline's workspace.

   The command uses `ALTER MATERIALIZED VIEW` and `ALTER STREAMING TABLE` for Unity Catalog managed materialized views and streaming tables, respectively. To

perform the same action on an Hive metastore table, use `ALTER TABLE`.

For example, if you want to move a streaming table named `sales` to a pipeline with the ID `abcd1234-ef56-ab78-cd90-1234efab5678`, you would run the following command:

SQL

```
ALTER STREAMING TABLE sales
  SET TBLPROPERTIES("pipelines.pipelineId"="abcd1234-ef56-ab78-cd90-1234efab5678");
```

> **NOTE**
>
> The `pipelineId` must be a valid pipeline identifier. The `null` value is not permitted.

4. Add the table's definition to the destination pipeline's code.

> **NOTE**
>
> If the catalog or target schema differ between the source and destination, copying the query exactly might not work. Partially qualified tables in the definition can resolve differently. You might need to update the definition while moving to fully qualify the table names.

> **NOTE**
>
> Remove or comment out any append once flows (in Python, queries with append_flow(once=True), in SQL, queries with INSERT INTO ONCE) from the destination pipeline's code. For more details, see Limitations.

The move is complete. You can now run both the source and destination pipelines. The destination pipeline updates the table.

Troubleshooting

The following table describes errors that could happen when moving a table between pipelines.

Error	Description
`DESTINATION_PIPELINE_NOT_IN_DIRECT_PUBLISHING_MODE`	The source pipeline is in the default publish mode, and the destination uses the LIVE schema (legacy) mode. This is not supported. If the source uses the default publishing mode, then the destination must, as well.
`PIPELINE_TYPE_NOT_WORKSPACE_PIPELINE_TYPE`	Only moving tables between pipelines is supported. Moving streaming tables and materialized views created with Databricks SQL are not supported.
`DESTINATION_PIPELINE_NOT_FOUND`	The `pipelines.pipelineId` must be a valid pipeline. The `pipelineId` can't be null.
Table fails to update in the destination after the move.	To quickly mitigate in this case, move the table back to the source pipeline following the same instructions.

Error	Description
`PIPELINE_PERMISSION_DENIED_NOT_OWNER`	Both the source and destination pipelines must be owned by the user performing the move operation.
`TABLE_ALREADY_EXISTS`	The table listed in the error message already exists. This can happen if a backing table for the pipeline already exists. In this case, `DROP` the table listed in the error.

Example with multiple tables in a pipeline

Pipelines can contain more than one table. You can still move one table at a time between pipelines. In this scenario, there are three tables (`table_a`, `table_b`, `table_c`) that read from each other sequentially in the source pipeline. We want to move one table, `table_b`, to another pipeline.

Initial source pipeline code:

Python

```python
from pyspark import pipelines as dp
from pyspark.sql.functions import col

@dp.table
def table_a():
    return spark.read.table("source_table")

# Table to be moved to new pipeline:
@dp.table
def table_b():
    return (
```

```
        spark.read.table("table_a")
        .select(col("column1"), col("column2"))
    )

@dp.table
def table_c():
    return (
        spark.read.table("table_b")
        .groupBy(col("column1"))
        .agg(sum("column2").alias("sum_column2"))
    )
```

We move `table_b` to another pipeline by copying and removing the table definition from the source and updating `table_b`'s pipelineId.

First, pause any schedules and wait for updates to complete on both the source and target pipelines. Then modify the source pipeline to remove code for the table being moved. The updated source pipeline example code becomes:

Python

```
from pyspark import pipelines as dp
from pyspark.sql.functions import col

@dp.table
def table_a():
    return spark.read.table("source_table")

# Removed, to be in new pipeline:
# @dp.table
# def table_b():
#     return (
#         spark.read.table("table_a")
#         .select(col("column1"), col("column2"))
#     )

@dp.table
def table_c():
    return (
        spark.read.table("table_b")
        .groupBy(col("column1"))
        .agg(sum("column2").alias("sum_column2"))
    )
```


Go to the SQL editor to run the `ALTER pipelineId` command.

SQL

```
ALTER MATERIALIZED VIEW table_b
  SET TBLPROPERTIES("pipelines.pipelineId"="<new-pipeline-id>");
```

Next, go to the destination pipeline and add the definition of `table_b`. If the default catalog and schema are the same in the pipeline settings, no code changes are required.

The target pipeline code:

Python

```
from pyspark import pipelines as dp
from pyspark.sql.functions import col

@dp.table(name="table_b")
def table_b():
    return (
        spark.read.table("table_a")
        .select(col("column1"), col("column2"))
    )
```

If the default catalog and schema differ in the pipeline settings, you must add in the fully qualified name using the pipeline's catalog and schema.

For example, the target pipeline code could be:

Python

```
from pyspark import pipelines as dp
from pyspark.sql.functions import col

@dp.table(name="source_catalog.source_schema.table_b")
def table_b():
    return (
        spark.read.table("source_catalog.source_schema.table_a")
        .select(col("column1"), col("column2"))
    )
```


Run (or re-enable schedules) for both the source and target pipelines.

The pipelines are now disjoint. The query for `table_c` reads from `table_b` (now in the target pipeline) and `table_b` reads from `table_a` (in the source pipeline). When you do a triggered execution on the source pipeline `table_b` is not updated because it is no longer managed by the source pipeline. The source pipeline treats `table_b` as a table external to the pipeline. This is comparable to defining a materialized view reading from a Delta table in Unity Catalog that is not managed by the pipeline.

Limitations

The following are limitations for moving tables between pipelines.

- Materialized views and streaming tables created with Databricks SQL are not supported.
- The append once flows - Python append_flow(once=True) flows and SQL INSERT INTO ONCE flows - are not supported. Their run status are not preserved, they may run again in the destination pipeline. Remove or comment out append once flows from the destination pipeline to avoid running these flows again.
- Private tables or views are not supported.
- The source and destination pipelines must be pipelines. Null pipelines are not supported.
- Source and destination pipelines must either be in the same workspace, or in different workspaces that share the same metastore.
- Both source and destination pipelines must be owned by the user running the move operation.
- If the source pipeline uses the default publishing mode, the destination pipeline must also be using the default publishing mode. You can't move a table from a pipeline using the default publishing mode to a pipeline that uses the LIVE schema (legacy). See LIVE schema (legacy).
- If the source and destination pipelines are both using the LIVE schema (legacy), then they must have the same `catalog` and `target` values in settings.

*Last updated on **May 12, 2026***

Pipeline properties reference

This article provides a reference for pipeline JSON setting specification and table properties in Lakeflow Spark Declarative Pipelines. For more details on using these various properties and configurations, see the following articles:

- Configure Pipelines
- Pipelines REST API

Pipeline configurations

- **`id`**

  Type: `string`

  A globally unique identifier for this pipeline. The identifier is assigned by the system and cannot be changed.

- **`name`**

  Type: `string`

  A user-friendly name for this pipeline. The name can be used to identify pipeline jobs in the UI.

- **`configuration`**

  Type: `object`

  An optional list of settings to add to the Spark configuration of the cluster that will run the pipeline. These settings are read by the Lakeflow Spark Declarative Pipelines runtime and available to pipeline queries through the Spar configuration.

Elements must be formatted as `key:value` pairs.

- **`libraries`**

  Type: `array of objects`

  An array of code files containing the pipeline code and required artifacts.

- **`clusters`**

  Type: `array of objects`

  An array of specifications for the clusters to run the pipeline.

  If this is not specified, pipelines will automatically select a default cluster configuration for the pipeline.

- **`development`**

  Type: `boolean`

  A flag indicating whether to run the pipeline in `development` or `production` mode.

  The default value is `false`.

- **`notifications`**

  Type: `array of objects`

  An optional array of specifications for email notifications when a pipeline update completes, fails with a retryable error, fails with a non-retryable error, or a flow fails.

- **`continuous`**

  Type: `boolean`

  A flag indicating whether to run the pipeline continuously.

  The default value is `false`.

- **`catalog`**

Type: `string`

The name of the default catalog for the pipeline, where all datasets and metadata for pipeline are published. Setting this value enables Unity Catalog for the pipeline.

If left unset, the pipeline publishes to the legacy Hive metastore using the location specified in `storage`.

In legacy publishing mode, specifies the catalog containing the target schema where all datasets from the current pipeline are published. See LIVE schema (legacy).

- **`schema`**

  Type: `string`

  The name of the default schema for the pipeline, where all datasets and metadata for the pipeline are published by default. See Set the target catalog and schema.

- **`target` (legacy)**

  Type: `string`

  The name of the target schema where all datasets defined in the current pipeline are published.

  Setting `target` instead of `schema` configures the pipeline to use legacy publishing mode. See LIVE schema (legacy).

- **`storage` (legacy)**

  Type: `string`

  A location on DBFS or cloud storage where output data and metadata required for pipeline execution are stored. Tables and metadata are stored in subdirectories of this location.

  When the `storage` setting is not specified, the system will default to a location in `dbfs:/pipelines/`.

  The `storage` setting cannot be changed after a pipeline is created.

- **`channel`**

  Type: `string`

  The version of the Lakeflow Spark Declarative Pipelines runtime to use. The supported values are:

  - `preview` to test your pipeline with upcoming changes to the runtime version.
  - `current` to use the current runtime version.

  The `channel` field is optional. The default value is `current`. Databricks recommends using the current runtime version for production workloads.

- **`edition`**

  Type `string`

  The Lakeflow Spark Declarative Pipelines product edition to run the pipeline. This setting allows you to choose the best product edition based on the requirements of your pipeline:

  - `CORE` to run streaming ingest workloads.
  - `PRO` to run streaming ingest and change data capture (CDC) workloads.
  - `ADVANCED` to run streaming ingest workloads, CDC workloads, and workloads that require expectations to enforce data quality constraints.

  The `edition` field is optional. The default value is `ADVANCED`.

- **`photon`**

  Type: `boolean`

  A flag indicating whether to use What is Photon? to run the pipeline. Photon is the Databricks high performance Spark engine. Photon-enabled pipelines are billed at a different rate than non-Photon pipelines.

  The `photon` field is optional. The default value is `false`.

- **`pipelines.maxFlowRetryAttempts`**

  Type: `int`

If a retryable failure occurs during a pipeline update, this is the maximum number of times to retry a flow before failing the pipeline update.

Use this to bound retries on a single flow that's prone to retryable failures so it can't stall an entire update.

Default: Two retry attempts. When a retryable failure occurs, the Lakeflow Spark Declarative Pipelines runtime attempts to run the flow three times, including the original attempt.

- **`pipelines.numUpdateRetryAttempts`**

  Type: `int`

  If a retryable failure occurs during an update, this is the maximum number of times to retry the update before permanently failing the update. The retry is run as a full update.

  Use this to bound retries on an entire update, so a stuck update fails permanently rather than retrying indefinitely.

  This parameter applies only to pipelines using automatic retry and restart behavior. Retries are not attempted for ad-hoc updates run from the editor or when you run a `Validate` update.

  Default:

  - Five for triggered pipelines.
  - Unlimited for continuous pipelines.

Pipeline table properties

In addition to the table properties supported by Delta Lake, you can set the following table properties.

- **`pipelines.autoOptimize.zOrderCols`**

  Default: None

An optional string containing a comma-separated list of column names to z-order this table by. For example, `pipelines.autoOptimize.zOrderCols = "year,month"`

Databricks recommends liquid clustering instead of Z-ordering for optimizing data layout in pipeline tables. See Use liquid clustering for tables.

- **`pipelines.reset.allowed`**

  Default: `true`

  Controls whether a full refresh is allowed for this table.

- **`pipelines.autoOptimize.managed`**

  Default: `true`

  Enables or disables automatically scheduled optimization of this table.

  For pipelines managed by predictive optimization, this property is not used.

Pipelines trigger interval

You can specify a pipeline trigger interval for the entire pipeline or as part of a dataset declaration. See Set trigger interval for continuous pipelines.

- `pipelines.trigger.interval`

  The default is based on flow type:

  - Five seconds for streaming queries.
  - One minute for complete queries when all input data is from Delta sources.
  - Ten minutes for complete queries when some data sources may be non-Delta.

  The value is a number plus the time unit. The following are the valid time units:

  - `second`, `seconds`
  - `minute`, `minutes`
  - `hour`, `hours`

- `day`, `days`

You can use the singular or plural unit when defining the value, for example:

- `{"pipelines.trigger.interval" : "1 hour"}`
- `{"pipelines.trigger.interval" : "10 seconds"}`
- `{"pipelines.trigger.interval" : "30 second"}`
- `{"pipelines.trigger.interval" : "1 minute"}`
- `{"pipelines.trigger.interval" : "10 minutes"}`
- `{"pipelines.trigger.interval" : "10 minute"}`

Cluster attributes that are not user settable

Because Lakeflow Spark Declarative Pipelines (SDP) manages cluster lifecycles, many cluster settings are set by the system and cannot be manually configured by users, either in a pipeline configuration or in a cluster policy used by a pipeline. The following table lists these settings and why they cannot be manually set.

- **`cluster_name`**

  SDP sets the names of the clusters used to run pipeline updates. These names cannot be overridden.

- **`data_security_mode`**

  `access_mode`

  These values are automatically set by the system.

- **`spark_version`**

  SDP clusters run on a custom version of Databricks Runtime that is continually updated to include the latest features. The version of Spark is bundled with the Databricks Runtime version and cannot be overridden.

- **`autotermination_minutes`**

Because SDP manages cluster auto-termination and reuse logic, the cluster auto-termination time cannot be overridden.

- `runtime_engine`

  Although you can control this field by enabling Photon for your pipeline, you cannot set this value directly.

- `effective_spark_version`

  This value is automatically set by the system.

- `cluster_source`

  This field is set by the system and is read-only.

- `docker_image`

  Because SDP manages the cluster lifecycle, you cannot use a custom container with pipeline clusters.

- `workload_type`

  This value is set by the system and cannot be overridden.

*Last updated on **Jan 23, 2026***

Use parameters with pipelines

This article explains how to configure parameters for your pipelines.

Reference parameters

During updates, your pipeline source code can access pipeline parameters using syntax to get values for Spark configurations.

You reference pipeline parameters using the key. The value is injected into your source code as a string before your source code logic evaluates.

The following example syntax uses a parameter with key `source_catalog` and value `dev_catalog` to specify the data source for a materialized view:

SQL Python

SQL

```
CREATE OR REFRESH MATERIALIZED VIEW transation_summary AS
SELECT account_id,
  COUNT(txn_id) txn_count,
  SUM(txn_amount) account_revenue
FROM ${source_catalog}.sales.transactions_table
GROUP BY account_id
```

Set parameters

Pass parameters to pipelines by passing arbitrary key-value pairs as configurations for the pipeline. You can set parameters while defining or editing a pipeline c
using the workspace UI or JSON. See Configure Pipelines.

Pipeline parameter keys can only contain `_ - .` or alphanumeric characters. Parameter values are set as strings.

Pipeline parameters do not support dynamic values. You must update the value associated with a key in the pipeline configuration.

> **IMPORTANT**
> Do not use keywords that conflict with reserved pipeline or Apache Spark configuration values.

Parameterize dataset declarations in Python or SQL

The Python and SQL code that defines your datasets can be parameterized by the pipeline's settings. Parameterization enables the following use cases:

- Separating long paths and other variables from your code.
- Reducing the amount of data processed in development or staging environments to speed up testing.
- Reusing the same transformation logic to process from multiple data sources.

The following example uses the `startDate` configuration value to limit the development pipeline to a subset of the input data:

SQL

```sql
CREATE OR REFRESH MATERIALIZED VIEW customer_events
AS SELECT * FROM sourceTable WHERE date > '${mypipeline.startDate}';
```

Python

```python
@dp.table
def customer_events():
  start_date = spark.conf.get("mypipeline.startDate")
  return read("sourceTable").where(col("date") > start_date)
```

JSON

```
{
  "name": "Data Ingest - DEV",
  "configuration": {
    "mypipeline.startDate": "2021-01-02"
  }
}
```

JSON

```
{
  "name": "Data Ingest - PROD",
  "configuration": {
    "mypipeline.startDate": "2010-01-02"
  }
}
```

Control data sources with parameters

You can use pipeline parameters to specify different data sources in different configurations of the same pipeline.

For example, you can specify different paths in development, testing, and production configurations for a pipeline using the variable `data_source_path` and then reference it using the following code:

SQL Python

SQL

```
CREATE STREAMING TABLE bronze AS
SELECT *, _metadata.file_path AS source_file_path
FROM STREAM read_files(
  '${data_source_path}',
  format => 'csv',
  header => true
)
```

Ask Genie

This pattern is beneficial for testing how ingestion logic might handle schema or malformed data during initial ingestion. You can use the identical code throughout your entire pipeline in all environments while switching out datasets.

Last updated on ***Apr 13, 2026***

Pipeline developer reference

This section contains reference and instructions for pipeline developers.

Data loading and transformations are implemented in pipelines by queries that define streaming tables and materialized views. To implement these queries, Lakeflow Spark Declarative Pipelines supports SQL and Python interfaces. Because these interfaces provide equivalent functionality for most data processing use cases, pipeline developers can choose the interface that they are most comfortable with.

Python development

Create pipelines using Python code.

Topic	Description
Develop pipeline code with Python	An overview of developing pipelines in Python.
Lakeflow Spark Declarative Pipelines Python language reference	Python reference documentation for the `pipelines` module.
Manage Python dependencies for pipelines	Instructions for managing Python libraries in pipelines.
Import Python modules from Git folders or workspace files	Instructions for using Python modules that you have stored in Databricks.

SQL development

Create pipelines using SQL code.

Topic	Description
Develop Lakeflow Spark Declarative Pipelines code with SQL	An overview of developing pipelines in SQL.
Pipeline SQL language reference	Reference documentation for SQL syntax for Lakeflow Spark Declarative Pipelines.
Standalone pipelines	Use Databricks SQL to work with pipelines.

Other development topics

The following topics describe other ways to develop piplines.

Topic	Description
Convert a pipeline into a bundle project	Convert an existing pipeline to a bundle, which allows you to manage your data processing configuration in a source-controlled YAML file for easier maintenance and automated deployments to target environments.
Metaprogramming with Lakeflow Spark Declarative Pipelines	Create pipelines with dlt-meta. Use the open source `dlt-meta` library to automate the creation of pipelines with a metadata-driven framework. Tutorial: Create multiple flows with different parameters. Create multiple flows in a loop in Python.

Topic	Description
Develop pipeline code in your local development environment	An overview of options for developing pipelines locally.

*Last updated on **Jan 23, 2026***

Develop pipeline code with Python

Lakeflow Spark Declarative Pipelines (SDP) introduces several new Python code constructs for defining materialized views and streaming tables in pipelines. Python support for developing pipelines builds upon the basics of PySpark DataFrame and Structured Streaming APIs.

For users unfamiliar with Python and DataFrames, Databricks recommends using the SQL interface. See Develop Lakeflow Spark Declarative Pipelines code with SQL.

For a full reference of Lakeflow SDP Python syntax, see Lakeflow Spark Declarative Pipelines Python language reference.

Basics of Python for pipeline development

Python code that creates pipline datasets must return DataFrames.

All Lakeflow Spark Declarative Pipelines Python APIs are implemented in the `pyspark.pipelines` module. Your pipeline code implemented with Python must explicitly import the `pipelines` module at the top of Python source. In our examples, we use the following import command, and use `dp` in examples to refer to `pipelines`.

Python

```
from pyspark import pipelines as dp
```


NOTE

Apache Spark™ includes **declarative pipelines** beginning in Spark 4.1, available through the `pyspark.pipelines` module. The Databricks Runtime extends these open-source capabilities with additional APIs and integrations for managed production use.

Code written with the open source `pipelines` module runs without modification on Databricks. The following features are not part of Apache Spark:

- `dp.create_auto_cdc_flow`
- `dp.create_auto_cdc_from_snapshot_flow`
- `@dp.expect(...)`

Pipeline reads and writes default to the catalog and schema specified during pipeline configuration. See Set the target catalog and schema.

Pipeline-specific Python code differs from other types of Python code in one critical way: Python pipeline code does not directly call the functions that perform data ingestion and transformation to create datasets. Instead, SDP interprets the decorator functions from the `dp` module in all source code files configured in a pipeline and builds a dataflow graph.

IMPORTANT

To avoid unexpected behavior when your pipeline runs, do not include code that might have side effects in your functions that define datasets. To learn more, see the Python reference.

Create a materialized view or streaming table with Python

Use `@dp.table` to create a streaming table from the results of a streaming read. Use `@dp.materialized_view` to create a materialized view from the results of a batch read.

By default, materialized view and streaming table names are inferred from function names. The following code example shows the basic syntax for creating a materialized view and streaming table:

> **NOTE**
>
> Both functions reference the same table in the `samples` catalog and use the same decorator function. These examples highlight that the only difference in the basic syntax for materialized views and streaming tables is using `spark.read` versus `spark.readStream`.
>
> Not all data sources support streaming reads. Some data sources should always be processed with streaming semantics.

Python

```python
from pyspark import pipelines as dp

@dp.materialized_view()
def basic_mv():
  return spark.read.table("samples.nyctaxi.trips")

@dp.table()
def basic_st():
  return spark.readStream.table("samples.nyctaxi.trips")
```

Optionally, you can specify the table name using the `name` argument in the `@dp.table` decorator. The following example demonstrates this pattern for a materialized view and streaming table:

Python

```python
from pyspark import pipelines as dp

@dp.materialized_view(name = "trips_mv")
def basic_mv():
  return spark.read.table("samples.nyctaxi.trips")

@dp.table(name = "trips_st")
def basic_st():
  return spark.readStream.table("samples.nyctaxi.trips")
```

Load data from object storage

Ask Genie

Pipelines support loading data from all formats supported by Databricks. See Data format options.

> **NOTE**
>
> These examples use data available under the `/databricks-datasets` automatically mounted to your workspace. Databricks recommends using volume paths or cloud URIs to reference data stored in cloud object storage. See What are Unity Catalog volumes?.

Databricks recommends using Auto Loader and streaming tables when configuring incremental ingestion workloads against data stored in cloud object storage. See What is Auto Loader?.

The following example creates a streaming table from JSON files using Auto Loader:

Python

```
from pyspark import pipelines as dp

@dp.table()
def ingestion_st():
  return (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .load("/databricks-datasets/retail-org/sales_orders")
  )
```

The following example uses batch semantics to read a JSON directory and create a materialized view:

Python

```
from pyspark import pipelines as dp

@dp.materialized_view()
def batch_mv():
  return spark.read.format("json").load("/databricks-datasets/retail-org/sales_orders")
```

Validate data with expectations

You can use expectations to set and enforce data quality constraints. See Manage data quality with pipeline expectations.

The following code uses `@dp.expect_or_drop` to define an expectation named `valid_data` that drops records that are null during data ingestion:

Python

```
from pyspark import pipelines as dp

@dp.table()
@dp.expect_or_drop("valid_date", "order_datetime IS NOT NULL AND
length(order_datetime) > 0")
def orders_valid():
  return (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .load("/databricks-datasets/retail-org/sales_orders")
  )
```

Query materialized views and streaming tables defined in your pipeline

The following example defines four datasets:

- A streaming table named `orders` that loads JSON data.
- A materialized view named `customers` that loads CSV data.
- A materialized view named `customer_orders` that joins records from the `orders` and `customers` datasets, casts the order timestamp to a date, and selects the `customer_id`, `order_number`, `state`, and `order_date` fields.
- A materialized view named `daily_orders_by_state` that aggregates the daily count of orders for each state.

NOTE

When querying views or tables in your pipeline, you can specify the catalog and schema directly, or you can use the defaults configured in your pipeline. In this example, the `orders`, `customers`, and `customer_orders` tables are written and read from the default catalog and schema configured for your pipeline.

Legacy publishing mode uses the `LIVE` schema to query other materialized views and streaming tables defined in your pipeline. In new pipelines, the `LIVE` schema syntax is silently ignored. See LIVE schema (legacy).

Python

```
from pyspark import pipelines as dp
from pyspark.sql.functions import col

@dp.table()
@dp.expect_or_drop("valid_date", "order_datetime IS NOT NULL AND
length(order_datetime) > 0")
def orders():
  return (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .load("/databricks-datasets/retail-org/sales_orders")
  )

@dp.materialized_view()
def customers():
    return spark.read.format("csv").option("header", True).load("/databricks
datasets/retail-org/customers")

@dp.materialized_view()
def customer_orders():
  return (spark.read.table("orders")
    .join(spark.read.table("customers"), "customer_id")
      .select("customer_id",
        "order_number",
        "state",

col("order_datetime").cast("int").cast("timestamp").cast("date").alias("orde
      )
  )

@dp.materialized_view()
def daily_orders_by_state():
    return (spark.read.table("customer_orders")
      .groupBy("state", "order_date")
```

```
      .count().withColumnRenamed("count", "order_count")
  )
```

Create tables in a `for` loop

You can use Python `for` loops to create multiple tables programmatically. This can be useful when you have many data sources or target datasets that vary by only a few parameters, resulting in less total code to maintain and less code redundancy.

The `for` loop evaluates logic in serial order, but once planning is complete for the datasets, the pipeline runs logic in parallel.

> **IMPORTANT**
>
> When using this pattern to define datasets, ensure that the list of values passed to the `for` loop is always additive. If a dataset previously defined in a pipeline is omitted from a future pipeline run, that dataset is dropped automatically from the target schema.

The following example creates five tables that filter customer orders by region. Here, the region name is used to set the name of the target materialized views and to filter the source data. Temporary views are used to define joins from the source tables used in constructing the final materialized views.

Python

```
from pyspark import pipelines as dp
from pyspark.sql.functions import collect_list, col

@dp.temporary_view()
def customer_orders():
  orders = spark.read.table("samples.tpch.orders")
  customer = spark.read.table("samples.tpch.customer")

  return (orders.join(customer, orders.o_custkey == customer.c_custkey)
    .select(
      col("c_custkey").alias("custkey"),
      col("c_name").alias("name"),
      col("c_nationkey").alias("nationkey"),
      col("c_phone").alias("phone"),
      col("o_orderkey").alias("orderkey"),
```

```
      col("o_orderstatus").alias("orderstatus"),
      col("o_totalprice").alias("totalprice"),
      col("o_orderdate").alias("orderdate"))
  )

@dp.temporary_view()
def nation_region():
  nation = spark.read.table("samples.tpch.nation")
  region = spark.read.table("samples.tpch.region")

  return (nation.join(region, nation.n_regionkey == region.r_regionkey)
    .select(
      col("n_name").alias("nation"),
      col("r_name").alias("region"),
      col("n_nationkey").alias("nationkey")
    )
  )

# Extract region names from region table

region_list =
spark.read.table("samples.tpch.region").select(collect_list("r_name")).colle
[0][0]

# Iterate through region names to create new region-specific materialized vi

for region in region_list:

  @dp.materialized_view(name=f"{region.lower().replace(' ',
'_')}_customer_orders")
  def regional_customer_orders(region_filter=region):

    customer_orders = spark.read.table("customer_orders")
    nation_region = spark.read.table("nation_region")

    return (customer_orders.join(nation_region, customer_orders.nationkey ==
nation_region.nationkey)
      .select(
        col("custkey"),
        col("name"),
        col("phone"),
        col("nation"),
        col("region"),
        col("orderkey"),
        col("orderstatus"),
        col("totalprice"),
        col("orderdate")
      ).filter(f"region = '{region_filter}'")
    )
```

The following is an example of the data flow graph for this pipeline:

Troubleshooting: `for` loop creates many tables with same values

The lazy execution model that pipelines use to evaluate Python code requires that your logic directly references individual values when the function decorated by `@dp.materialized_view()` is invoked.

The following example demonstrates two correct approaches to defining tables with a `for` loop. In both examples, each table name from the `tables` list is explicitly referenced within the function decorated by `@dp.materialized_view()`.

Python

```python
from pyspark import pipelines as dp

# Create a parent function to set local variables

def create_table(table_name):
  @dp.materialized_view(name=table_name)
  def t():
    return spark.read.table(table_name)

tables = ["t1", "t2", "t3"]
for t_name in tables:
  create_table(t_name)

# Call `@dp.materialized_view()` within a for loop and pass values as
variables

tables = ["t1", "t2", "t3"]
for t_name in tables:

  @dp.materialized_view(name=t_name)
  def create_table(table_name=t_name):
    return spark.read.table(table_name)
```

The following example **does not** reference values correctly. This example creates tables with distinct names, but all tables load data from the last value in the `for` loop:

Python

```python
from pyspark import pipelines as dp

# Don't do this!

tables = ["t1", "t2", "t3"]
for t_name in tables:

  @dp.materialized(name=t_name)
  def create_table():
    return spark.read.table(t_name)
```

Permanently delete records from a materialized view or streaming table

To permanently delete records from a materialized view or streaming table with deletion vectors enabled, such as for GDPR compliance, additional operations must be performed on the object's underlying Delta tables. To ensure the deletion of records from a materialized view, see Permanently delete records from a materialized view with deletion vectors enabled. To ensure the deletion of records from a streaming table, see Permanently delete records from a streaming table.

*Last updated on **Jan 23, 2026***

Lakeflow Spark Declarative Pipelines Python language reference

This section has details for the Lakeflow Spark Declarative Pipelines (SDP) Python programming interface.

- For conceptual information and an overview of using Python for pipelines, see Develop pipeline code with Python.
- For SQL reference, see the Pipeline SQL language reference.
- For details specific to configuring Auto Loader, see What is Auto Loader?.

`pipelines` module overview

Lakeflow Spark Declarative Pipelines Python functions are defined in the `pyspark.pipelines` module (imported as `dp`). Your pipelines implemented with the Python API must import this module:

Python

```
from pyspark import pipelines as dp
```

NOTE

The pipelines module is only available in the context of a pipeline. It is not available in Python running outside of pipelines. For more information about editing pipeline code, see Develop and debug ETL pipelines with the Lakeflow Pipelines Editor.

Apache Spark™ pipelines

Apache Spark includes **declarative pipelines** beginning in Spark 4.1, available through the `pyspark.pipelines` module. The Databricks Runtime extends these open source capabilities with additional APIs and integrations for managed production use.

Code written with the open-source `pipelines` module runs without modification on Databricks. The following features are not part of Apache Spark:

- `dp.create_auto_cdc_flow`
- `dp.create_auto_cdc_from_snapshot_flow`
- `@dp.expect(...)`
- `@dp.temporary_view`

The `pipelines` module was previously called `dlt` in Databricks. For details, and more information about the differences from Apache Spark, see What happened to `@dlt`?.

Functions for dataset definitions

Pipelines use Python decorators for defining datasets such as materialized views and streaming tables. See Functions to define datasets.

API reference

- append_flow
- create_auto_cdc_flow
- create_auto_cdc_from_snapshot_flow
- create_sink
- create_streaming_table
- Expectations
- materialized_view
- table
- temporary_view

Coding requirements for Python pipelines

The following are important requirements when you implement pipelines with the Lakeflow Spark Declarative Pipelines (SDP) Python interface:

- SDP evaluates the code that defines a pipeline multiple times during planning and pipeline runs. Python functions that define datasets should include only the code required to define the table or view. Arbitrary Python logic included in dataset definitions might lead to unexpected behavior.
- Do not try to implement custom monitoring logic in your dataset definitions. See Define custom monitoring of pipelines with event hooks.
- The function used to define a dataset must return a Spark DataFrame. Do not include logic in your dataset definitions that does not relate to a returned DataFrame.
- Never use methods that save or write to files or tables as part of your pipeline dataset code.

Examples of Apache Spark operations that should never be used in pipeline code:

- `collect()`
- `count()`
- `toPandas()`
- `save()`
- `saveAsTable()`
- `start()`
- `toTable()`

What happened to `@dlt`?

Previously, Databricks used the `dlt` module to support pipeline functionality. The `dlt` module has been replaced by the `pyspark.pipelines` module. You may still use `dlt`, but Databricks recommends using `pipelines`.

Differences between DLT, SDP, and Apache Spark

The following table shows the differences in syntax and functionality between DLT, Lakeflow Spark Declarative Pipelines, and Apache Spark Declarative Pipelines.

Area	DLT syntax	SDP Syntax (Lakeflow and Apache, where applicable)
Imports	`import dlt`	`from pyspark import pipelines` (`as dp`, optionally)
Streaming table	`@dlt.table` with a streaming dataframe	`@dp.table`
Materialized view	`@dlt.table` with a batch dataframe	`@dp.materialized_view`
View	`@dlt.view`	`@dp.temporary_view`
Append flow	`@dlt.append_flow`	`@dp.append_flow`
SQL – streaming	`CREATE STREAMING TABLE ...`	`CREATE STREAMING TABLE ...`
SQL – materialized	`CREATE MATERIALIZED VIEW ...`	`CREATE MATERIALIZED VIEW ...`
SQL – flow	`CREATE FLOW ...`	`CREATE FLOW ...`
Event log	`spark.read.table("event_log")`	`spark.read.table("event_log"`
Apply Changes (CDC)	`dlt.apply_changes(...)`	`dp.create_auto_cdc_flow(...)`

Area	DLT syntax	SDP Syntax (Lakeflow and Apache, where applicable)
Expectations	`@dlt.expect(...)`	`dp.expect(...)`
Continuous mode	Pipeline config with continuous trigger	(same)
Sink	`@dlt.create_sink(...)`	`dp.create_sink(...)`

Last updated on ***Feb 23, 2026***

Develop Lakeflow Spark Declarative Pipelines code with SQL

Lakeflow Spark Declarative Pipelines (SDP) introduces several new SQL keywords and functions for defining materialized views and streaming tables in pipelines. SQL support for developing pipelines builds upon the basics of Spark SQL and adds support for Structured Streaming functionality.

Users familiar with PySpark DataFrames might prefer developing pipeline code with Python. Python supports more extensive testing and operations that are challenging to implement with SQL, such as metaprogramming operations. See Develop pipeline code with Python.

For a full reference of pipeline SQL syntax, see Pipeline SQL language reference.

Basics of SQL for pipeline development

SQL code that creates pipeline datasets uses the `CREATE OR REFRESH` syntax to define materialized views and streaming tables against query results.

The `STREAM` keyword indicates if the data source referenced in a `SELECT` clause should be read with streaming semantics.

Reads and writes default to the catalog and schema specified during pipeline configuration. See Set the target catalog and schema.

Pipeline source code critically differs from SQL scripts: SDP evaluates all dataset definitions across all source code files configured in a pipeline and builds a dataflow graph before any queries are run. The order of queries appearing in the source files defines the order of code evaluation, but not the order of query execution.

Create a materialized view with SQL

The following code example demonstrates the basic syntax for creating a materialized view with SQL:

SQL

```
CREATE OR REFRESH MATERIALIZED VIEW basic_mv
AS SELECT * FROM samples.nyctaxi.trips;
```

Create a streaming table with SQL

The following code example demonstrates the basic syntax for creating a streaming table with SQL. When reading a source for a streaming table, the `STREAM` keyword indicates to use streaming semantics for the source. Do not use the `STREAM` keyword when creating a materialized view:

SQL

```
CREATE OR REFRESH STREAMING TABLE basic_st
AS SELECT * FROM STREAM samples.nyctaxi.trips;
```

NOTE

Use the STREAM keyword to use streaming semantics to read from the source. If the read encounters a change or deletion to an existing record, an error is thrown. It is safest to read from static or append-only sources. To ingest data that has change commits, you can use the `SkipChangeCommits` option to handle errors.

Example:

SQL

```
CREATE OR REFRESH STREAMING TABLE basic_st
AS SELECT * FROM STREAM samples.nyctaxi.trips WITH
(SKIPCHANGECOMMITS);
```

Load data from object storage

Pipelines support loading data from all formats supported by Databricks. See Data format options.

> **NOTE**
>
> These examples use data available under the `/databricks-datasets` automatically mounted to your workspace. Databricks recommends using volume paths or cloud URIs to reference data stored in cloud object storage. See What are Unity Catalog volumes?.

Databricks recommends using Auto Loader and streaming tables when configuring incremental ingestion workloads against data stored in cloud object storage. See What is Auto Loader?.

SQL uses the `read_files` function to invoke Auto Loader functionality. You must also use the `STREAM` keyword to configure a streaming read with `read_files`.

The following describes the syntax for `read_files` in SQL:

```
CREATE OR REFRESH STREAMING TABLE table_name
AS SELECT *
  FROM STREAM read_files(
    "<file-path>",
    [<option-key> => <option_value>, ...]
  )
```

Options for Auto Loader are key-value pairs. For details on supported formats and options, see Options.

The following example creates a streaming table from JSON files using Auto Loader:

SQL

```
CREATE OR REFRESH STREAMING TABLE ingestion_st
AS SELECT *
FROM STREAM read_files(
  "/databricks-datasets/retail-org/sales_orders",
  format => "json");
```

The `read_files` function also supports batch semantics to create materialized views. The following example uses batch semantics to read a JSON directory and create a materialized view:

SQL

```
CREATE OR REFRESH MATERIALIZED VIEW batch_mv
AS SELECT *
FROM read_files(
  "/databricks-datasets/retail-org/sales_orders",
  format => "json");
```

Validate data with expectations

You can use expectations to set and enforce data quality constraints. See Manage data quality with pipeline expectations.

The following code defines an expectation named `valid_data` that drops records that are null during data ingestion:

SQL

```
CREATE OR REFRESH STREAMING TABLE orders_valid(
  CONSTRAINT valid_date
  EXPECT (order_datetime IS NOT NULL AND length(order_datetime) > 0)
  ON VIOLATION DROP ROW
)
AS SELECT * FROM STREAM read_files("/databricks-datasets/retail-
org/sales_orders");
```

Query materialized views and streaming tables defined in your pipeline

The following example defines four datasets:

- A streaming table named `orders` that loads JSON data.
- A materialized view named `customers` that loads CSV data.
- A materialized view named `customer_orders` that joins records from the `orders` and `customers` datasets, casts the order timestamp to a date, and selects the `customer_id`, `order_number`, `state`, and `order_date` fields.
- A materialized view named `daily_orders_by_state` that aggregates the daily count of orders for each state.

> **NOTE**
>
> When querying views or tables in your pipeline, you can specify the catalog and schema directly, or you can use the defaults configured in your pipeline. In this example, the `orders`, `customers`, and `customer_orders` tables are written and read from the default catalog and schema configured for your pipeline.
>
> Legacy publishing mode uses the `LIVE` schema to query other materialized views and streaming tables defined in your pipeline. In new pipelines, the `LIVE` schema syntax is silently ignored. See LIVE schema (legacy).

SQL

```
CREATE OR REFRESH STREAMING TABLE orders(
  CONSTRAINT valid_date
  EXPECT (order_datetime IS NOT NULL AND length(order_datetime) > 0)
  ON VIOLATION DROP ROW
)
AS SELECT * FROM STREAM read_files("/databricks-datasets/retail-
org/sales_orders");

CREATE OR REFRESH MATERIALIZED VIEW customers
AS SELECT * FROM read_files("/databricks-datasets/retail-org/customers");

CREATE OR REFRESH MATERIALIZED VIEW customer_orders
AS SELECT
  c.customer_id,
```

```
  o.order_number,
  c.state,
  date(timestamp(int(o.order_datetime))) order_date
FROM orders o
INNER JOIN customers c
ON o.customer_id = c.customer_id;

CREATE OR REFRESH MATERIALIZED VIEW daily_orders_by_state
AS SELECT state, order_date, count(*) order_count
FROM customer_orders
GROUP BY state, order_date;
```

Define a private table

You can use the `PRIVATE` clause when creating a materialized view or a streaming table. When you create a private table, you create the table, but do not create the metadata for the table. The `PRIVATE` clause instructs SDP to create a table that is available to the pipeline but should not be accessed outside the pipeline. To reduce processing time, a private table persists for the lifetime of the pipeline that creates it, and not just a single update.

Private tables can have the same name as tables in the catalog. If you specify an unqualified name for a table within a pipeline, if there is both a private table and a catalog table with that name, the private table will be used.

Private tables were previously refererred to as temporary tables.

Permanently delete records from a materialized view or streaming table

To permanently delete records from a streaming table with deletion vectors enabled, such as for GDPR compliance, additional operations must be performed on the object's underlying Delta tables. To ensure the deletion of records from a streaming table, see Permanently delete records from a streaming table.

Materialized views always reflect the data in the underlying tables when they are refreshed. To delete data in a Materialized view, you must delete the data from the source and refresh the materialized view.

Parameterize values used when declaring tables or views with SQL

Use `SET` to specify a configuration value in a query that declares a table or view, including Spark configurations. Any table or view you define in a source file after the `SET` statement has access to the defined value. Any Spark configurations specified using the `SET` statement are used when executing the Spark query for any table or view following the SET statement. To read a configuration value in a query, use the string interpolation syntax `${}`. The following example sets a Spark configuration value named `startDate` and uses that value in a query:

```
SET startDate='2025-01-01';

CREATE OR REFRESH MATERIALIZED VIEW filtered
AS SELECT * FROM src
WHERE date > ${startDate}
```

To specify multiple configuration values, use a separate `SET` statement for each value.

Limitations

The `PIVOT` clause is not supported. The `pivot` operation in Spark requires the eager loading of input data to compute the output schema. This capability is not supported in pipelines.

> **NOTE**
>
> The `CREATE OR REFRESH LIVE TABLE` syntax to create a materialized view is deprecated. Instead, use `CREATE OR REFRESH MATERIALIZED VIEW`.

*Last updated on **Jan 23, 2026***

Pipeline SQL language reference

This section has details for the Lakeflow Spark Declarative Pipelines SQL programming interface.

- For conceptual information and an overview of using Lakeflow Spark Declarative Pipelines SQL, see Develop Lakeflow Spark Declarative Pipelines code with SQL.
- For information on the Python API, see the Lakeflow Spark Declarative Pipelines Python language reference.
- For more information about SQL in Databricks, see SQL language reference.

You can use Python user-defined functions (UDFs) in your SQL queries, but you must define these UDFs in Python files before calling them in SQL source files. See User-defined scalar functions - Python.

SQL statements

Use these statements to work with pipelines in SQL.

- AUTO CDC INTO
- CREATE FLOW
- CREATE MATERIALIZED VIEW
- CREATE STREAMING TABLE
- CREATE TEMPORARY VIEW
- CREATE VIEW

You can also use the following statements to work with pipelines, but they must be run from Databricks SQL, not from within a pipeline. See Get started with da

warehousing using Databricks SQL, and Use `ALTER` statements with pipeline datasets.

- ALTER STREAMING TABLE
- ALTER MATERIALIZED VIEW
- EXPLAIN CREATE MATERIALIZED VIEW

*Last updated on **Jan 23, 2026***

Develop pipeline code in your local development environment

You can author Python pipeline source code in your preferred integrated development environment (IDE).

You cannot validate or run updates on pipeline code written in an IDE. You must deploy source code files back to a Databricks workspace and configure them as part of a pipeline.

This article provides an overview of support for local IDE development. For more interactive development and testing, Databricks recommends using the Lakeflow Pipelines Editor. See Develop and debug ETL pipelines with the Lakeflow Pipelines Editor.

Configure a local IDE for pipeline development

Databricks provides a Python module for local development distributed through PyPI. For installation and usage instructions, see Python stub for DLT.

This module has the interfaces and docstring references for the pipeline Python interface, providing syntax checking, autocomplete, and data type checking as you write code in your IDE.

This module includes interfaces but no functional implementations. You cannot use this library to create or run pipelines locally.

You can use Declarative Automation Bundles to package and deploy source code and configurations to a target workspace, and to trigger running an update on a pipeline configured this way. See Convert a pipeline into a bundle project.

The Databricks extension for Visual Studio Code has additional functionality for working with pipelines using Declarative Automation Bundles. See Bundle Resource Explorer.

Sync pipeline code from your IDE to a workspace

The following table summarizes options for syncing pipeline source code between your local IDE and a Databricks workspace:

Tool or pattern	Details
Declarative Automation Bundles	Use Declarative Automation Bundles to deploy pipeline assets ranging in complexity from a single source code file to configurations for multiple pipelines, jobs, and source code files. See Convert a pipeline into a bundle project.
Databricks extension for Visual Studio Code	Databricks provides an integration with Visual Studio Code that includes easy syncing between your local IDE and workspace files. This extension also provides tools for using Declarative Automation Bundles to deploy pipelines assets. See Databricks extension for Visual Studio Code.
Workspace files	You can use Databricks workspace files to upload your pipeline source code to your Databricks workspace and then import that code into a pipeline. See What are workspace files?.
Git folders	Git folders let you sync code between your local environment and Databricks workspace using a Git repository as the intermediary. See Databricks Git folders.

this page

as this page helpful?

Yes

No

Send feedback

Ask Genie

*Last updated on **May 8, 2026***

Standalone pipelines

Lakeflow Spark Declarative Pipelines is the most common way to work with data in pipelines. You can also define standalone materialized views and streaming tables outside of Lakeflow Spark Declarative Pipelines using simple query syntax, and Databricks manages the underlying pipelines for you. Standalone tables can be created and refreshed from a Databricks SQL warehouse or from a notebook running on serverless general compute.

> **NOTE**
>
> This section was previously called "Pipelines for Databricks SQL." It was renamed to "Standalone pipelines" to reflect new support for creating standalone materialized views and streaming tables from a notebook running on serverless general compute, in addition to a Databricks SQL warehouse.

This section teaches you about using standalone pipelines, including the following topics.

Topic	Description
Requirements for standalone pipelines	Learn about the compute options for standalone materialized views and streaming tables, including feature support and regional availability.
Use standalone streaming tables	Create, refresh, configure, and monitor streaming tables.
Use standalone materialized views	Create, refresh, and query materialized views.

Additional resources

- ETL in Databricks SQL
- Databricks SQL language reference
- Lakeflow Spark Declarative Pipelines SQL language reference
- Develop Lakeflow Spark Declarative Pipelines
- Use `ALTER` statements with pipeline datasets

*Last updated on **May 8, 2026***

Requirements for standalone pipelines

This page describes the requirements for creating and refreshing standalone materialized views and streaming tables.

You can create and refresh standalone materialized views and streaming tables using a SQL warehouse. To submit `CREATE` and `REFRESH` statements, use the SQL editor in the Databricks UI, the Databricks SQL CLI, or the Databricks SQL API.

You can also create and refresh standalone materialized views and streaming tables from a notebook running on serverless general compute (Beta, limited regional availability). See Notebooks.

General requirements

The following requirements apply to all standalone pipelines.

You must have:

- A Databricks account with serverless enabled. See Set up serverless SQL warehouses.
- A workspace with Unity Catalog enabled. See Get started with Unity Catalog.

Permissions to create or refresh

The owner (the user who creates the table) must have the following permissions:

- `SELECT` privilege on the base tables.

- `USE CATALOG` and `USE SCHEMA` privileges on the catalog and schema containing the source tables.
- `USE CATALOG` and `USE SCHEMA` privileges on the target catalog and schema.
- `CREATE MATERIALIZED VIEW` privilege on the schema containing the materialized view.
- `CREATE TABLE` privilege on the schema containing the streaming table. Pipelines using legacy publishing mode also require the `CREATE TABLE` privilege for materialized views.

To refresh a standalone materialized view or streaming table:

- You must be in the workspace that created it.
- You must have the `REFRESH` privilege on the table. Owners have this privilege implicitly.

Source table requirements

For incremental refresh of materialized views from Delta tables, the source tables must have row tracking enabled.

SQL warehouses

To create or refresh standalone materialized views and streaming tables using a SQL warehouse, you must have a Unity Catalog-enabled pro or serverless SQL warehouse.

- Your workspace must be in a region that supports serverless SQL warehouses.
- You must have accepted the serverless terms of use.

Notebooks

You can create and refresh standalone materialized views and streaming tables from a notebook with serverless general compute.

Serverless general compute

BETA

Creating and refreshing standalone materialized views and streaming tables from a notebook on serverless general compute is in Beta. This feature is available in select regions only. See Regional availability.

You can create and refresh standalone materialized views and streaming tables from a notebook attached to serverless general compute. This option is useful when you want to define and run materialized views or streaming tables alongside other notebook-based workflows without provisioning a SQL warehouse.

Serverless general compute requirements

- A notebook attached to serverless general compute.
- Databricks Runtime 18.1 or above. Interactive notebooks meet this requirement automatically; jobs pinned to an older version do not.
- Your workspace must be in a supported region.

Limitations

- Only the table owner can refresh the table. To allow another user to refresh, change the owner. See Change the owner of a streaming table and Change the owner of a materialized view.
- Asynchronous refreshes are not supported. Use a synchronous refresh instead.
- The preview channel is not supported. Tables created on serverless general compute use the `current` channel.
- A table can only be refreshed using the compute type it was created with. A table created on a SQL warehouse must be refreshed on a SQL warehouse, and a table created on serverless general compute must be refreshed on serverless general compute. To check the compute type, view the table in Catalog Explorer.
- Cost attribution and control are not available. Use a SQL warehouse if you need per-table cost attribution.
- Vertical autoscaling on out-of-memory errors is not available.
- Retries for schema upgrades are not available.
- Performance mode selection on refresh is not available. See Select a performance mode for scheduled refreshes.

> **NOTE**
>
> `spark.sql` is supported when running a refresh in a notebook on serverless general compute.

:::

Query requirements

To query a standalone materialized view or streaming table, you must be the owner, or you must have `SELECT` on the table along with `USE CATALOG` and `USE SCHEMA` on its parents.

You must use one of the following compute resources:

- SQL warehouse
- Lakeflow Spark Declarative Pipelines interfaces
- Standard access mode compute (formerly shared access mode)
- Dedicated access mode compute (formerly single user access mode) on Databricks Runtime 15.4 or above, if the workspace is enabled for serverless compute. See Fine-grained access control on dedicated compute. If you are the owner, you can use dedicated access mode compute running Databricks Runtime 14.3 or above.

For streaming tables on Databricks Runtime 15.3 and below, you can use dedicated compute to query an streaming table only if you own it. Databricks Runtime 15.4 LTS and above support querying pipeline-generated tables on dedicated compute even if you aren't the owner. You might be charged for serverless compute resources when you use dedicated compute to run data filtering operations. See Fine-grained access control on dedicated compute.

Regional availability

Tables created and refreshed using a Databricks SQL warehouse are available in all regions that support serverless Databricks SQL warehouses.

Creating and refreshing standalone materialized views and streaming tables on serverless general compute is available in select regions only.

For the list of supported regions for both compute options, see Serverless availability.

*Last updated on **May 8, 2026***

Use standalone streaming tables

A standalone *streaming table* is a table registered to Unity Catalog with extra support for streaming or incremental data processing, defined outside of Lakeflow Spark Declarative Pipelines. A pipeline is automatically created for each streaming table. You can use streaming tables for incremental data loading from Kafka and cloud object storage.

You can create and refresh standalone streaming tables from a Databricks SQL warehouse, or from a notebook running on serverless general compute. For details on the differences between the two compute options, see Requirements for standalone pipelines.

> **NOTE**
>
> To learn how to use Delta Lake tables as streaming sources and sinks, see Delta table streaming reads and writes.

Requirements

For compute options, permissions, and other requirements for creating, refreshing, and querying standalone streaming tables, see Requirements for standalone pipelines.

Create streaming tables

A streaming table is defined by a SQL query in Databricks SQL. When you create a streaming table, the data currently in the source tables is used to build t

table. After that, you refresh the table, usually on a schedule, to pull in any added data in the source tables to append to the streaming table.

When you create a streaming table, you are considered the owner of the table.

To create a streaming table from an existing table, use the CREATE STREAMING TABLE statement, as in the following example:

SQL

```
CREATE OR REFRESH STREAMING TABLE sales
  SCHEDULE EVERY 1 hour
  AS SELECT product, price FROM STREAM raw_data;
```

In this case, the streaming table `sales` is created from specific columns of the `raw_data` table, with a schedule to refresh every hour. The query used must be a *streaming* query. Use the `STREAM` keyword to use streaming semantics to read from the source.

Compute used for refresh

When you create a streaming table using the `CREATE OR REFRESH STREAMING TABLE` statement, the initial data refresh and population begin immediately. These operations do not consume Databricks SQL warehouse compute. Instead, streaming tables rely on serverless pipelines for both creation and refresh. A dedicated serverless pipeline is automatically created and managed by the system for each streaming table.

Load files with Auto Loader

To create a streaming table from files in a volume, you use Auto Loader. Use Auto Loader for most data ingestion tasks from cloud object storage. Auto Loader and pipelines are designed to incrementally and idempotently load ever-growing data as it arrives in cloud storage.

To use Auto Loader in Databricks SQL, use the `read_files` function. The following examples shows using Auto Loader to read a volume of JSON files into a streaming table:

SQL

```
CREATE OR REFRESH STREAMING TABLE sales
  SCHEDULE EVERY 1 hour
  AS SELECT * FROM STREAM read_files(
    "/Volumes/my_catalog/my_schema/my_volume/path/to/data",
    format => "json"
  );
```

To read data from cloud storage, you can also use Auto Loader:

SQL

```
CREATE OR REFRESH STREAMING TABLE sales
  SCHEDULE EVERY 1 hour
  AS SELECT *
  FROM STREAM read_files(
  's3://mybucket/analysis/*/*/*.json',
    format => "json"
  );
```

To learn about Auto Loader, see What is Auto Loader?. To learn more about using Auto Loader in SQL, with examples, see Load data from object storage.

Streaming ingestion from other sources

For example of ingestion from other sources, including Kafka, see Load data in pipelines.

Apply change data capture (CDC) with Auto CDC flows

> **BETA**
>
> This feature is in Beta. Available in Databricks SQL using the `Current` channel.

Use the `FLOW AUTO CDC` clause to process change data capture (CDC) records from a source into a streaming table. Previously, the `MERGE INTO` statement was commonly used for processing CDC records on Databricks. However, `MERGE INTO` can produce

incorrect results because of out-of-sequence records or requires complex logic to re-order records. See Change data capture and snapshots.

`AUTO CDC` simplifies CDC by automatically handling out-of-order records. You specify keys to identify records, a sequence column for ordering, and whether to store results as SCD type 1 (direct updates) or SCD type 2 (history tracking).

The following example creates a streaming table that applies CDC changes using SCD type 1:

SQL

```
CREATE OR REFRESH STREAMING TABLE target
  FLOW AUTO CDC
  FROM stream(cdc_data.users)
  KEYS (userId)
  SEQUENCE BY sequenceNum
  STORED AS SCD TYPE 1;
```

The following example uses SCD type 2 to retain a history of changes:

SQL

```
CREATE OR REFRESH STREAMING TABLE target
  FLOW AUTO CDC
  FROM stream(cdc_data.users)
  KEYS (userId)
  APPLY AS DELETE WHEN operation = "DELETE"
  SEQUENCE BY sequenceNum
  COLUMNS * EXCEPT (operation, sequenceNum)
  STORED AS SCD TYPE 2;
```

For full details on Auto CDC options and behavior, see The AUTO CDC APIs: Simplify change data capture with pipelines. For the complete syntax reference, see CREATE STREAMING TABLE.

Ingest new data only

By default, the `read_files` function reads all existing data in the source folder during table creation, and then processes newly arriving records with each refresh.

To avoid ingesting data that already exists in the source folder at the time of table creation, set the `includeExistingFiles` option to `false`. This means that only data that arrives in the folder after table creation is processed. For example:

SQL

```
CREATE OR REFRESH STREAMING TABLE sales
  SCHEDULE EVERY 1 hour
  AS SELECT *
  FROM STREAM read_files(
    '/path/to/files',
    includeExistingFiles => false
  );
```

Set the runtime channel

Streaming tables created using SQL warehouses are automatically refreshed using a pipeline. Pipelines use the runtime in the `current` channel by default. See Lakeflow Spark Declarative Pipelines release notes and the release upgrade process to learn about the release process.

Databricks recommends using the `current` channel for production workloads. New features are first released to the `preview` channel. You can set a pipeline to the preview channel to test new features by specifying `preview` as a table property using a `CREATE OR REFRESH STREAMING TABLE` statement. To update the channel for an existing streaming table, you must run `CREATE OR REFRESH STREAMING TABLE` with the updated `TBLPROPERTIES`.

The following code example shows how to set the channel to preview:

SQL

```
CREATE OR REFRESH STREAMING TABLE sales
  TBLPROPERTIES ('pipelines.channel' = 'preview')
  SCHEDULE EVERY 1 hour
  AS SELECT *
  FROM STREAM raw_data;
```


Hide sensitive data

You can use streaming tables to hide sensitive data from users accessing the table. One approach is to define the query so that it excludes sensitive columns or rows entirely. Alternatively, you can apply column masks or row filters based on the permissions of the querying user. For example, you might hide the `tax_id` column for users who are not in the group `HumanResourcesDept`. To do this, use the `ROW FILTER` and `MASK` syntax during the creation of the streaming table. For more information, see Row filters and column masks.

Refresh a streaming table

Streaming tables automatically create and use serverless pipelines to process refresh operations. The refresh is managed by the pipeline and the update is monitored by the Databricks SQL warehouse used to create the streaming table. Streaming tables can be updated using a pipeline that runs on a schedule.

Even if you have a scheduled refresh, you can call a manual refresh at any time. Refreshes are handled by the same pipeline that was automatically created along with the streaming table.

To refresh a streaming table:

SQL

```
REFRESH STREAMING TABLE sales;
```

You can check the status of the latest refresh with DESCRIBE TABLE EXTENDED.

> **NOTE**
>
> You might need to refresh your streaming table before using time travel queries.

To learn how to schedule a refresh, see Schedule refreshes. Scheduled refreshes can have update notifications, and you can set the performance mode for the refresh.

How refresh works

A streaming table refresh only evaluates new rows that have arrived after the last update, and appends only the new data.

Each refresh uses the current definition of the streaming table to process this new data. Modifying a streaming table definition does not automatically recalculate existing data. If a modification is incompatible with existing data (for example, changing a data type), the next refresh fails with an error.

The following examples explain how changes to a streaming table definition affect refresh behavior:

- Removing a filter does not reprocess previously filtered rows.
- Changing column projections won't affect how existing data was processed.
- Joins with static snapshots use the snapshot state at the time of the initial processing. Late-arriving data that would have matched with the updated snapshot is ignored. This can lead to facts being dropped if dimensions are late.
- Modifying the CAST of an existing column results in an error.

If your data changes in a way that cannot be supported in the existing streaming table, you can perform a full refresh.

Fully refresh a streaming table

Full refreshes re-process all data available in the source with the latest definition. It is not recommended to call full refreshes on sources that don't keep the entire history of the data or have short retention periods, such as Kafka, because the full refresh truncates the existing data. You might not be able to recover old data if the data is no longer available in the source.

For example:

SQL

```
REFRESH STREAMING TABLE sales FULL;
```

Change the schedule for a streaming table

You can configure a Databricks SQL streaming table to refresh automatically based on a defined schedule, or to trigger when upstream data is changed. The following table shows the different options for scheduling refreshes.

Method	Description	Example use case
Manual	On-demand refresh using a SQL `REFRESH` statement, or through the workspace UI.	Development, testing, ad-hoc updates.
`TRIGGER ON UPDATE`	Define the streaming table to automatically refresh when the upstream data changes.	Production workloads with data freshness SLAs or unpredictable refresh periods.
`SCHEDULE`	Define the streaming table to refresh at defined time intervals.	Predictable, time-based refresh requirements.
SQL task in a job	Refresh is orchestrated through Lakeflow Jobs.	Complex pipelines with cross-system dependencies.

Manual refresh

To manually refresh a streaming table, you can call a refresh from Databricks SQL, or use the workspace UI.

REFRESH statement | **Workspace UI**

To refresh a pipeline using Databricks SQL:

1. In the **SQL Editor**, run the following statement:

```sql
REFRESH MATERIALIZED VIEW <table-name>;
```

Ask Genie

For more information, see REFRESH (MATERIALIZED VIEW or STREAMING TABLE).

Trigger on update

The `TRIGGER ON UPDATE` clause automatically refreshes a streaming table when upstream source data changes. This eliminates the need to coordinate schedules across pipelines. The streaming table stays fresh without requiring the user to know when upstream jobs finish or maintain complex scheduling logic.

This is the recommended approach for production workloads, especially when upstream dependencies don't run on predictable schedules. Once configured, the streaming table monitors its source tables and refreshes automatically when changes in any of the upstream sources are detected.

Limitations

- Upstream dependency limits: A streaming table can monitor a maximum of 10 upstream tables and 30 upstream views. For more dependencies, split the logic across multiple streaming tables.
- Workspace limits: A maximum of 1,000 streaming tables with `TRIGGER ON UPDATE` can exist per workspace. Please contact Databricks support if more than 1,000 streaming tables are needed.
- Minimum interval: The minimum trigger interval is 1 minute.

The following examples show how to set a trigger on update when defining a streaming table.

Create a streaming table with trigger on update

To create a streaming table that refreshes automatically when source data changes, include the `TRIGGER ON UPDATE` clause in the `CREATE STREAMING TABLE` statement.

The following example creates a streaming table that reads customer orders and refreshes whenever the source `orders` table is updated:

SQL

```
CREATE OR REFRESH STREAMING TABLE catalog.schema.customer_orders
  TRIGGER ON UPDATE
```

```
AS SELECT
    o.customer_id,
    o.name,
    o.order_id
FROM STREAM catalog.schema.orders o;
```

Throttle refresh frequency

If upstream data refreshes frequently, use `AT MOST EVERY` to cap how often the view refreshes and limit compute costs. This is useful when source tables update frequently but downstream consumers don't need real-time data. The `INTERVAL` keyword is required before the time value.

The following example limits the streaming table to refresh at most every 5 minutes, even if source data changes more frequently:

SQL

```
CREATE OR REFRESH STREAMING TABLE catalog.schema.customer_orders
  TRIGGER ON UPDATE AT MOST EVERY INTERVAL 5 MINUTES
AS SELECT
    o.customer_id,
    o.name,
    o.order_id
FROM STREAM catalog.schema.orders o;
```

Scheduled refresh

Refresh schedules can be defined directly in the streaming table definition to refresh the view at fixed time intervals. This approach is useful when the data update cadence is known and predictable refresh timing is desired.

When there is a refresh schedule, you can still run a manual refresh at any time if you need updated data.

Databricks supports two scheduling syntaxes: `SCHEDULE EVERY` for simple intervals and `SCHEDULE CRON` for precise scheduling. The `SCHEDULE` and `SCHEDULE REFRESH` keywords are semantically equivalent.

For details about the syntax and use of the `SCHEDULE` clause, see CREATE STREAMING TABLE SCHEDULE clause.

When a schedule is created, a new Databricks job is automatically configured to process the update.

To view the schedule, do one of the following:

- Run the `DESCRIBE EXTENDED` statement from the SQL editor in the Databricks UI. See DESCRIBE TABLE.
- Use Catalog Explorer to view the streaming table. The schedule is listed on the **Overview** tab, under **Refresh status**. See What is Catalog Explorer?.

The following examples show how to create a streaming table with a schedule:

Schedule every time interval

This example schedules a refresh once every 5 minutes:

SQL

```
CREATE OR REFRESH STREAMING TABLE catalog.schema.hourly_metrics
  SCHEDULE EVERY 5 MINUTES
AS SELECT
    event_id,
    event_time,
    event_type
FROM catalog.schema.raw_events;
```

Schedule using cron

This example schedules a refresh every 15 minutes, at the quarter hour of the UTC time zone:

SQL

```
CREATE OR REFRESH STREAMING TABLE catalog.schema.hourly_metrics
  SCHEDULE CRON '0 */15 * * * ?' AT TIME ZONE 'UTC'
AS SELECT
    event_id,
    event_time,
    event_type
FROM catalog.schema.raw_events;
```

SQL task in a job

Streaming table refreshes can be orchestrated through Databricks Jobs by creating SQL tasks that include `REFRESH STREAMING TABLE` commands. This approach integrates streaming table refreshes into existing job orchestration workflows.

There are two ways to create a job for refreshing streaming tables:

- **From the SQL Editor**: Write the `REFRESH STREAMING TABLE` command and click the **Schedule** button to create a job directly from the query.
- **From the Jobs UI**: Create a new job, add a SQL task type, and attach a SQL Query or Notebook with the `REFRESH STREAMING TABLE` command.

The following example shows the SQL statement within a SQL task that refreshes a streaming table:

SQL

```
REFRESH STREAMING TABLE catalog.schema.sales;
```

This approach is appropriate when:

- Complex multi-step pipelines have dependencies across systems.
- Integration with existing job orchestration is required.
- Job-level alerting and monitoring is needed.

SQL tasks use both the SQL warehouse attached to the job and the serverless compute that executes the refresh. If using streaming table definition-based scheduling meets the requirements, switching to `TRIGGER ON UPDATE` or `SCHEDULE` can simplify the workflow.

Add a schedule to an existing streaming table

To set the schedule after creation, use the `ALTER STREAMING TABLE` statement:

SQL

```
-- Alters the schedule to refresh the streaming table when its upstream
-- data gets updated.
ALTER STREAMING TABLE sales
  ADD TRIGGER ON UPDATE;
```

Modify an existing schedule or trigger

If a streaming table already has a schedule or trigger associated, use `ALTER SCHEDULE` or `ALTER TRIGGER ON UPDATE` to change the refresh configuration. This applies whether changing from one schedule to another, one trigger to another, or switching between a schedule and a trigger.

The following example changes an existing schedule to refresh every 5 minutes:

SQL

```
ALTER STREAMING TABLE catalog.schema.my_table
  ALTER SCHEDULE EVERY 5 MINUTES;
```

Drop a schedule or trigger

To remove a schedule, use `ALTER ... DROP`:

SQL

```
ALTER STREAMING TABLE catalog.schema.my_table
  DROP SCHEDULE;
```

Track the status of a refresh

You can view the status of a streaming table refresh by viewing the pipeline that manages the streaming table in the Pipelines UI or by viewing the **Refresh Information** returned by the `DESCRIBE EXTENDED` command for the streaming table.

SQL

```
DESCRIBE TABLE EXTENDED <table-name>;
```

Alternately, you can view the streaming table in Catalog Explorer and see the refresh status there:

1. Click **Catalog** in the sidebar.
2. In the Catalog Explorer tree at the left, open the catalog and select the schema where your streaming table is located.
3. Open the **Tables** item under the schema you selected, and click the streaming table.

From here, you can use the tabs under the streaming table name to view and edit information about the streaming table, including:

- Refresh status and history
- The table schema
- Sample data (requires an active compute)
- Permissions
- Lineage, including tables and pipelines that this streaming table depends on
- Insights into usage
- Monitors that you have created for this streaming table

Timeouts for refreshes

Streaming table refreshes are run with a timeout that limits how long they can run. For streaming tables created or updated on or after August 14, 2025, the timeout is captured when you update by running `CREATE OR REFRESH`:

- If a `STATEMENT_TIMEOUT` is set, that value is used. See STATEMENT_TIMEOUT.
- Otherwise, the timeout from the SQL warehouse used to run the command is used.
- If the warehouse does not have a timeout configured, a default of 2 days applies.

The timeout is used on the initial create, but also on scheduled refreshes that follow.

For streaming tables that were last updated prior to August 14, 2025, the timeout is set to 2 days.

Example: Set a timeout for a streaming table refresh You can explicitly control how long a streaming table refresh is allowed to run by setting a statement-level timeout when creating or updating the table:

SQL

```
SET STATEMENT_TIMEOUT = '6h';

CREATE OR REFRESH STREAMING TABLE my_catalog.my_schema.my_st
  SCHEDULE EVERY 12 HOURS
AS SELECT * FROM large_source_table;
```

This sets up the streaming table to be refreshed every 12 hours, and if a refresh takes more than 6 hours, it times out and waits for the next scheduled refresh.

How scheduled refreshes handle timeouts

Timeouts are synchronized only when you explicitly run `CREATE OR REFRESH`.

- Scheduled refreshes continue using the timeout captured during the most recent `CREATE OR REFRESH`.
- Changing the warehouse timeout alone does not affect existing scheduled refreshes.

> **IMPORTANT**
> After changing a warehouse timeout, run `CREATE OR REFRESH` again to apply the new timeout to future scheduled refreshes.

Control access to streaming tables

Streaming tables support rich access controls to support data-sharing while avoiding exposing potentially private data. A streaming table owner or a user with the `MANAGE` privilege can grant `SELECT` privileges to other users. Users with `SELECT` access to the streaming table do not require `SELECT` access to the tables referenced by the

streaming table. This access control enables data sharing while controlling access to the underlying data.

You can also modify the owner of a streaming table.

Grant privileges to a streaming table

To grant access to a streaming table, use the GRANT statement:

```
SQL

GRANT <privilege_type> ON <st_name> TO <principal>;
```

The `privilege_type` can be:

- `SELECT` - the user can `SELECT` the streaming table.
- `REFRESH` - the user can `REFRESH` the streaming table. Refreshes are run using the owner's permissions.

The following example creates a streaming table and grants select and refresh privileges to users:

```
SQL

CREATE OR REFRESH STREAMING TABLE st_name AS SELECT * FROM source_table;

-- Grant read-only access:
GRANT SELECT ON st_name TO read_only_user;

-- Grant read and refresh access:
GRANT SELECT ON st_name TO refresh_user;
GRANT REFRESH ON st_name TO refresh_user;
```

For more information about granting privileges on Unity Catalog securable objects, see Unity Catalog privileges reference.

Revoke privileges from a streaming table

To revoke access from a streaming table, use the REVOKE statement:

SQL

```
REVOKE privilege_type ON <st_name> FROM principal;
```

When `SELECT` privileges on a source table are revoked from the streaming table owner or any other user who has been granted `MANAGE` or `SELECT` privileges on the streaming table, or the source table is dropped, the streaming table owner or user granted access is still able to query the streaming table. However, the following behavior occurs:

- The streaming table owner or others who have lost access to a streaming table can no longer `REFRESH` that streaming table, and the streaming table becomes stale over time.
- If automated with a schedule, the next scheduled `REFRESH` fails or is not run.

The following example revokes the `SELECT` privilege from `read_only_user`:

SQL

```
REVOKE SELECT ON st_name FROM read_only_user;
```

Change the owner of a streaming table

A user with `MANAGE` permissions on a streaming table defined in Databricks SQL can set a new owner through the Catalog Explorer. The new owner can be themselves or a service principal on which they have the **Service Principal User** role.

1. From your Databricks workspace, click **Catalog** to open the Catalog Explorer.
2. Select the streaming table that you want to update.
3. In the right sidebar, under **About this streaming table**, find the **Owner**, and click edit.

> **NOTE**
>
> If you get a message that tells you to update the owner by changing the **Run as** user in pipeline settings, then the streaming table is defined in Lakeflow

> Spark Declarative Pipelines, not Databricks SQL. The message includes a link to the pipeline settings, where you can change the **Run as** user.

4. Select a new owner for the streaming table.

   Owners automatically have `MANAGE` and `SELECT` privileges on streaming tables that they own. If you are setting a service principal as the owner for a streaming table that you own, and you do not explicitly have `SELECT` or `MANAGE` privileges on the streaming table, then this change would cause you to lose all access to the streaming table. In this case, you are prompted to explicitly provide those privileges.

   Select both **Grant MANAGE** and **Grant SELECT** privileges to provide those on **Save**.

5. Click **Save** to change the owner.

The owner of the streaming table is updated. All future updates are run using the new owner's identity.

When the owner loses privileges to source tables

If you change the owner, and the new owner does not have access to the source tables (or `SELECT` privileges are revoked on the underlying source tables), users can still query the streaming table. However:

- They cannot `REFRESH` the streaming table.
- The next scheduled refresh of the streaming table fails.

Losing access to the source data prevents updates, but doesn't immediately invalidate the existing streaming table from being read.

Permanently delete records from a streaming table

PREVIEW

Support for the `REORG` statement with streaming tables is in Public Preview.

NOTE

- Using a `REORG` statement with a streaming table requires Databricks Runtime 15.4 and above.
- Although you can use the `REORG` statement with any streaming table, it's only required when deleting records from a streaming table with deletion vectors enabled. The command has no effect when used with a streaming table without deletion vectors enabled.

To physically delete records from the underlying storage for a streaming table with deletion vectors enabled, such as for GDPR compliance, additional steps must be taken to ensure that a VACUUM operation runs on the streaming table's data.

To physically delete records from underlying storage:

1. Update records or delete records from the streaming table.
2. Run a `REORG` statement against the streaming table, specifying the `APPLY (PURGE)` parameter. For example `REORG TABLE <streaming-table-name> APPLY (PURGE);`.
3. Wait for the streaming table's data retention period to pass. The default data retention period is seven days, but it can be configured with the `delta.deletedFileRetentionDuration` table property. See Configure data retention for time travel queries.
4. `REFRESH` the streaming table. See Refresh a streaming table. Within 24 hours of the `REFRESH` operation, pipeline maintenance tasks, including the `VACUUM` operation which is required to ensure records are permanently deleted, are run automatically.

Monitor runs using query history

You can use the query history page to access query details and query profiles that can help you identify poorly performing queries and bottlenecks in the pipeline used to run your streaming table updates. For an overview of the kind of information available in query histories and query profiles, see Query history and Query profile.

> **PREVIEW**
>
> This feature is in Public Preview. Workspace admins can control access to this feature from the **Previews** page. See Manage Databricks previews.

All statements related to streaming tables appear in the query history. You can use the **Statement** drop-down filter to select any command and inspect the related queries. All `CREATE` statements are followed by a `REFRESH` statement that executes asynchronously on a pipeline. The `REFRESH` statements typically include detailed query plans that provide insights into optimizing performance.

To access `REFRESH` statements in the query history UI, use the following steps:

1. Click in the left sidebar to open the **Query History** UI.
2. Select the **REFRESH** checkbox from the **Statement** drop-down filter.
3. Click the name of the query statement to view summary details like the duration of the query and aggregated metrics.
4. Click **See query profile** to open the query profile. See Query profile for details about navigating the query profile.
5. Optionally, you can use the links in the **Query Source** section to open the related query or pipeline.

You can also access query details using links in the SQL editor or from a notebook attached to a SQL warehouse.

Access streaming tables from external clients

To access streaming tables from external Delta Lake or Iceberg clients that don't support open APIs, you can use Compatibility Mode. Compatibility Mode creates a read-only version of your streaming table that can be accessed by any Delta Lake or Iceberg client.

Additional resources

- Lakeflow Spark Declarative Pipelines
- `read_files` table-valued function
- `read_kafka` table-valued function
- CREATE STREAMING TABLE
- ALTER STREAMING TABLE
- Use standalone materialized views
- The AUTO CDC APIs: Simplify change data capture with pipelines

*Last updated on **May 8, 2026***

Use standalone materialized views

This page describes how to create and refresh standalone materialized views to improve performance and reduce the cost of your data processing and analysis workloads.

You can create and refresh standalone materialized views from a Databricks SQL warehouse, or from a notebook running on serverless general compute. For details on the differences between the two compute options, see Requirements for standalone pipelines.

What are standalone materialized views?

A standalone materialized view is a Unity Catalog managed table that physically stores the results of a query, defined outside of Lakeflow Spark Declarative Pipelines. Unlike standard views, which compute results on demand, materialized views cache the results and update them as the underlying source tables change, either on a schedule or automatically.

Materialized views are well-suited for data processing workloads such as extract, transform, and load (ETL) processing. Materialized views provide a simple, declarative way to process data for compliance, corrections, aggregations, or general change data capture (CDC). Materialized views also enable easy-to-use transformations by cleaning, enriching, and denormalizing base tables. By pre-computing expensive or frequently used queries, Materialized views lower query latency and resource consumption. In many cases, they can incrementally compute changes from source tables, further improving efficiency and end-user experience.

The following are common use cases for materialized views:

- Keeping a BI dashboard up to date with minimal end-user query latency.
- Reducing complex ETL orchestration with simple SQL logic.
- Building complex, layered transformations.
- Any use cases that demand consistent performance with up-to-date insights.

When you create a materialized view in a Databricks SQL warehouse, a serverless pipeline is created to process the create and refreshes to the materialized view. You can monitor the status of refresh operations in Catalog Explorer. See View details with `DESCRIBE EXTENDED`.

Requirements

For compute options, permissions, and other requirements for creating, refreshing, and querying standalone materialized views, see Requirements for standalone pipelines.

To learn about other restrictions on using materialized views, see Limitations.

Create a materialized view

Databricks SQL materialized view `CREATE` operations use a Databricks SQL warehouse to create and load data in the materialized view. Creating a materialized view is a synchronous operation, which means that the `CREATE MATERIALIZED VIEW` command blocks until the materialized view is created and the initial data load finishes. A serverless pipeline is automatically created for every Databricks SQL materialized view. When the materialized view is refreshed, the pipeline processes the refresh.

To create a materialized view, use the `CREATE MATERIALIZED VIEW` statement. To submit a create statement, use the SQL editor in the Databricks UI, the Databricks SQL CLI, or the Databricks SQL API.

The user who creates a materialized view is the materialized view owner.

Ad-hoc materialized view

The following example creates the materialized view `mv1` from the base table `base_table1`:

SQL

```
-- This query defines the materialized view:
CREATE OR REPLACE MATERIALIZED VIEW mv1
AS SELECT
  date,
  sum(sales) AS sum_of_sales
FROM
  base_table1
GROUP BY
  date;
```

On-trigger materialized view

The following example creates a materialized view that automatically refreshes whenever the upstream source data changes using `TRIGGER ON UPDATE`. Use this approach for production workloads, especially when upstream dependencies don't run on predictable schedules.

SQL

```
-- Refresh automatically when the source table is updated.
CREATE OR REPLACE MATERIALIZED VIEW mv_trigger
  TRIGGER ON UPDATE
AS SELECT
  date,
  sum(sales) AS sum_of_sales
FROM
  base_table1
GROUP BY
  date;
```

Scheduled materialized view

The following example creates a materialized view with a daily CRON refresh schedule at midnight UTC. Expressions and aggregates in the `SELECT` clause must use aliases. `GROUP BY` column references don't require aliases.

SQL

```
-- Refresh nightly at midnight UTC.
-- The cron expression uses six space-separated fields: seconds minutes
hours day-of-month month day-of-week
-- Use '?' for either day-of-month or day-of-week to leave it
unspecified.
CREATE OR REPLACE MATERIALIZED VIEW daily_revenue_by_region
  SCHEDULE CRON '0 0 0 * * ?' AT TIME ZONE 'UTC'
AS SELECT
  date_trunc('day', order_time) AS sales_date,
  region,
  sum(revenue) AS total_revenue,
  count(*) AS order_count
FROM
  orders
GROUP BY sales_date, region;
```

For more scheduling options, including `SCHEDULE EVERY` syntax and additional CRON examples, see Schedule refreshes.

When you create a materialized view using the `CREATE OR REPLACE MATERIALIZED VIEW` statement, the initial data refresh and population begins immediately. This does not consume SQL warehouse compute. Instead, a serverless pipeline is used for creation and subsequent refreshes. See How are Databricks SQL materialized views refreshed?.

Column comments on a base table are automatically propagated to the new materialized view on create only. To add a schedule, table constraints, or other properties, modify the materialized view definition (the SQL query).

The same SQL statement refreshes a materialized view if called a subsequent time, or on a schedule. A refresh done in this way acts as any other refresh. For details, see Refresh a materialized view.

To learn more about configuring a materialized view, see Configure standalone materialized views. To learn about the complete syntax for creating a materialized view, see CREATE MATERIALIZED VIEW. To learn about loading data in different formats and from different places, see Load data in pipelines.

Load data from external systems

Materialized views can be created on external data using Lakehouse Federation for supported data sources. For information on loading data from sources not supported by Lakehouse Federation, see Data format options. For general information about loading data, including examples, see Load data in pipelines.

Hide sensitive data

You can use materialized views to hide sensitive data from users accessing the table. One way to do this is to create the query so it doesn't include that data in the first place. But you can also mask columns or filter rows based on the permissions of the querying user. For example, you could hide the `tax_id` column for users that are not in the group `HumanResourcesDept`. To do this, use the `ROW FILTER` and `MASK` syntax during the creation of the materialized view. For more information, see Row filters and column masks.

Refresh a materialized view

Refreshing a materialized view updates the view to reflect the latest changes to the base table at the time of the refresh.

When you define a materialized view, the `CREATE OR REPLACE MATERIALIZED VIEW` statement is used both to create the view, and to refresh it for any scheduled refreshes. You can also use the `REFRESH MATERIALIZED VIEW` statement to refresh the materialized view without needing to supply the query again. See REFRESH (MATERIALIZED VIEW or STREAMING TABLE) for details on the SQL syntax and parameters for this command. To learn more about the types of materialized views that can be incrementally refreshed, see Incremental refresh for materialized views.

To submit a refresh statement, use the SQL editor in the Databricks UI, a notebook attached to a SQL warehouse, the Databricks SQL CLI, or the Databricks SQL API.

The owner, and any user who has been granted the `REFRESH` privilege on the table, can refresh the materialized view.

The following example refreshes the `mv1` materialized view:

SQL Ask Genie

```
REFRESH MATERIALIZED VIEW mv1;
```

The operation is synchronous by default, meaning that the command blocks until the refresh operation is complete. To refresh asynchronously, you can add the `ASYNC` keyword:

```
SQL

REFRESH MATERIALIZED VIEW mv1 ASYNC;
```

To learn how to schedule a refresh, see Schedule refreshes.

How are Databricks SQL materialized views refreshed?

Materialized views automatically create and use serverless pipelines to process refresh operations. The refresh is managed by the pipeline and the update is monitored by the Databricks SQL warehouse used to create the materialized view. Materialized views can be updated using a pipeline that runs on a schedule. Databricks SQL created materialized views always run in triggered mode. See Triggered vs. continuous pipeline mode.

Scheduled refreshes can have update notifications, and you can set the performance mode for the refresh.

Incremental refresh

Materialized views are refreshed using one of two methods.

- **Incremental refresh** - The system evaluates the view's query to identify changes that happened after the last update and merges only the new or modified data.
- **Full refresh** - If an incremental refresh can't be performed or is not cost-effective, the system runs the entire query and replaces the existing data in the materialized view with the new results.

The structure of the query and the type of source data determine whether Ask Genie incremental refresh is supported. To support incremental refresh, source data should

be stored in Delta tables, with row tracking and change data feed enabled. To see if a query is incrementalizable, use the Databricks SQL EXPLAIN CREATE MATERIALIZED VIEW statement. After you create a materialized view, you can monitor its refresh behavior to verify whether it is updated incrementally or through a full refresh.

By default, Databricks uses a cost model to choose the more cost-effective option between full and incremental refresh. You can override this behavior to prefer either incremental or full refreshes by setting a `REFRESH POLICY` in your SQL definition of the materialized view.

For details on refresh types, and how to optimize for incremental refreshes, see Incremental refresh for materialized views.

Asynchronous refreshes

By default, refresh operations are performed synchronously. You can also set a refresh operation to occur asynchronously. This can be set using the refresh command with the `ASYNC` keyword. See REFRESH (MATERIALIZED VIEW or STREAMING TABLE). The behavior associated with each approach is as follows:

- **Synchronous**: A synchronous refresh prevents other operations from proceeding until the refresh is complete. If the result is needed for the next step, such as when sequencing refresh operations in orchestration tools like Lakeflow Jobs, use a synchronous refresh. To orchestrate materialized views with a job, use the **SQL** task type. See Lakeflow Jobs.
- **Asynchronous**: An asynchronous refresh starts a background job on serverless compute when a materialized view refresh begins, allowing the command to return before the data load completes. This refresh type can save on cost because the operation doesn't necessarily hold compute capacity in the warehouse where the command is initiated. If the refresh becomes idle and no other tasks are running, the warehouse can shut down while the refresh uses other available compute. Additionally, asynchronous refreshes support starting multiple operations in parallel.

Permanently delete records from a materialized view with deletion vectors

enabled

> **PREVIEW**
>
> Support for the `REORG` statement with materialized views is in Public Preview.

> **NOTE**
>
> - Using a `REORG` statement with a materialized view requires Databricks Runtime 15.4 and above.
> - Although you can use the `REORG` statement with any materialized view, it's only required when deleting records from a materialized view with deletion vectors enabled. The command has no effect when used with a materialized view without deletion vectors enabled.

To physically delete records from the underlying storage for a materialized view with deletion vectors enabled, such as for GDPR compliance, additional steps must be taken to ensure that a VACUUM operation runs on the materialized view's data.

To physically delete records:

1. Run a `REORG` statement against the materialized view, specifying the `APPLY (PURGE)` parameter. For example `REORG TABLE <materialized-view-name> APPLY (PURGE);`. See REORG TABLE.
2. Wait for the materialized view's data retention period to pass. The default data retention period is seven days, but it can be configured with the `delta.deletedFileRetentionDuration` table property. See Configure data retention for time travel queries.
3. `REFRESH` the materialized view. See Refresh a materialized view. Within 24 hours of the `REFRESH` operation, pipeline maintenance tasks, including the `VACUUM` operation which is required to ensure records are permanently deleted, are run automatically.

Drop a materialized view

> **NOTE**
>
> Ask Genie

> To submit the command to drop a materialized view, you must be the owner of that materialized view or have the `MANAGE` privilege on the materialized view.

To drop a materialized view, use the DROP VIEW statement. To submit a `DROP` statement, you can use the SQL editor in the Databricks UI, the Databricks SQL CLI, or the Databricks SQL API. The following example drops the `mv1` materialized view:

SQL

```
DROP MATERIALIZED VIEW mv1;
```

You can also use the Catalog Explorer to drop a materialized view.

1. Click **Catalog** in the sidebar.
2. In the Catalog Explorer tree at the left, open the catalog and select the schema where your materialized view is located.
3. Open the **Tables** item under the schema you selected, and click the materialized view.
4. On the kebab menu , select **Delete**.

Understand the costs of a materialized view

When you run `CREATE MATERIALIZED VIEW` or `REFRESH MATERIALIZED VIEW`, Databricks automatically creates and runs a serverless pipeline to process the operation. This pipeline is independent of the Databricks SQL warehouse or compute resource from which you submitted the command. The cluster size of your warehouse does not limit the compute or cost used by the refresh.

- The refresh pipeline runs on serverless compute, billed as serverless Lakeflow Spark Declarative Pipelines DBUs.
- The serverless pipeline is separate from your warehouse. Compute from your warehouse is only used to coordinate the operation, not to perform the data processing.

- Cost scales with the volume of data processed, not the size of your SQL warehouse.
- To monitor Materialized view refresh costs, use system tables. See What is the DBU consumption of a materialized view or streaming table?.
- To view the underlying pipeline that manages your materialized view:
  i. Click **Jobs & Pipelines** in the left sidebar of your Databricks workspace.
  ii. Click **Pipeline type**. Then, select **MV/ST Pipeline** to see materialized views created in Databricks SQL.

> **NOTE**
> You might incur serverless compute charges even when the originating warehouse uses dedicated compute.

Enabling row tracking

To support incremental refreshes from Delta tables, row tracking must be enabled for those source tables. If you recreate a source table, you must re-enable row tracking.

The following example shows how to enable row tracking on a table:

SQL

```
ALTER TABLE source_table SET TBLPROPERTIES (delta.enableRowTracking =
true);
```

For more details, see Row tracking in Databricks

Limitations

- For compute options and workspace requirements, see Requirements for standalone pipelines.
- For incremental refresh requirements, see Incremental refresh for materialized views.
- Materialized views do not support identity columns or surrogate keys.

- If a materialized view uses a sum aggregate over a `NULL`-able column and only `NULL` values remain in that column, the materialized views resultant aggregate value is zero instead of `NULL`.
- You cannot read a change data feed from a materialized view.
- Time travel queries are not supported on materialized views.
- The underlying files supporting materialized views might include data from upstream tables (including possible personally identifiable information) that do not appear in the materialized view definition. This data is automatically added to the underlying storage to support incremental refreshing of materialized views. Because the underlying files of a materialized view might risk exposing data from upstream tables not part of the materialized view schema, Databricks recommends not sharing the underlying storage with untrusted downstream consumers. For example, suppose the definition of a materialized view includes a `COUNT(DISTINCT field_a)` clause. Even though the materialized view definition only includes the aggregate `COUNT DISTINCT` clause, the underlying files contain a list of the actual values of `field_a`.
- You might incur some serverless compute charges, even when using these features on dedicated compute.
- If you need to use an AWS PrivateLink connection with your materialized view, contact your Databricks representative.

Access materialized views from external clients

To access materialized views from external Delta Lake or Iceberg clients that don't support open APIs, you can use Compatibility Mode. Compatibility Mode creates a read-only version of your materialized view that can be accessed by any Delta Lake or Iceberg client.

Related articles

- Configure standalone materialized views
- Monitor standalone materialized views
- Use standalone streaming tables

this page

as this page helpful?

Yes

No

Send feedback

Ask Genie

*Last updated on **May 8, 2026***

Schedule refreshes

You can manually refresh a standalone materialized view or streaming table when you know that the source tables have been updated. However, you can also set a schedule for refreshes, either by time, when source tables update, or through orchestration. This page describes how to create the schedules for your standalone tables.

You can also create notifications and set the performance mode for your scheduled refreshes.

Create a schedule

You can configure a Databricks SQL pipeline to refresh automatically based on a defined schedule, or to trigger when upstream data is changed. The following table shows the different options for scheduling refreshes.

Method	Description	Example use case
Manual	On-demand refresh using a SQL `REFRESH` statement, or through the workspace UI.	Development, testing, ad-hoc updates.
`TRIGGER ON UPDATE`	Schedule the pipeline to automatically refresh when the upstream data changes.	Production workloads with data freshness SLAs or unpredictable refresh periods.
`SCHEDULE`	Schedule the pipeline to refresh at defined time intervals.	Predictable, time-based refresh requirements.

Method	Description	Example use case
SQL task in a job	Refresh is orchestrated through Lakeflow Jobs.	Complex pipelines with cross-system dependencies.

Even when you schedule refreshes, you can run a manual refresh at any time if you need updated data.

Manual refresh

To manually refresh a pipeline, you can call a refresh from Databricks SQL, or use the workspace UI.

REFRESH statement | **Workspace UI**

To refresh a pipeline using Databricks SQL:

1. In the **SQL Editor**, run the following statement:

   ```
   SQL

   REFRESH MATERIALIZED VIEW <table-name>;
   ```

   For streaming tables, use `REFRESH STREAMING TABLE`.

For more information, see REFRESH (MATERIALIZED VIEW or STREAMING TABLE).

Trigger on update

The `TRIGGER ON UPDATE` clause automatically refreshes a pipeline when upstream source data changes. This eliminates the need to coordinate schedules across pipelines. The dataset stays fresh without requiring the user to know when upstream jobs finish or maintain complex scheduling logic.

This is the recommended approach for production workloads, especially when upstream dependencies don't run on predictable schedules. After configuring trigger

on update, the pipeline monitors its source tables and refreshes automatically when changes in any of the upstream sources are detected.

Limitations

- Upstream dependency limits: A pipeline can monitor a maximum of 10 upstream tables and 30 upstream views. For more dependencies, split the logic across multiple pipelines.
- Workspace limits: A maximum of 1,000 pipelines with `TRIGGER ON UPDATE` can exist per workspace. Contact Databricks support if more than 1,000 are needed.
- Minimum interval: The minimum trigger interval is 1 minute.

The following examples show how to set a trigger on update when defining a pipeline.

Create a pipeline with trigger on update

To create a pipeline that refreshes automatically when source data changes, include the `TRIGGER ON UPDATE` clause in the `CREATE` statement.

The following example creates a streaming table that reads customer orders and refreshes whenever the source `orders` table is updated:

SQL

```
CREATE OR REFRESH STREAMING TABLE catalog.schema.customer_orders
  TRIGGER ON UPDATE
AS SELECT
    o.customer_id,
    o.name,
    o.order_id
FROM catalog.schema.orders o;
```

Throttle refresh frequency

If upstream data refreshes frequently, use `AT MOST EVERY` to cap how often the view refreshes and limit compute costs. This is useful when source tables update frequently but downstream consumers don't need real-time data. The `INTERVAL` keyword is required before the time value.

The following example limits the streaming table to refresh at most every 5 minutes, even if source data changes more frequently:

SQL

```
CREATE OR REFRESH STREAMING TABLE catalog.schema.customer_orders
  TRIGGER ON UPDATE AT MOST EVERY INTERVAL 5 MINUTES
AS SELECT
    o.customer_id,
    o.name,
    o.order_id
FROM catalog.schema.orders o;
```

Scheduled refresh

Refresh schedules can be defined directly in the pipeline definition to refresh the view at fixed time intervals. This approach is useful when the data update cadence is known and predictable refresh timing is desired.

When there is a refresh schedule, you can still run a manual refresh at any time if you want updated data.

Databricks supports two scheduling syntaxes: `SCHEDULE EVERY` for simple intervals and `SCHEDULE CRON` for precise scheduling. The `SCHEDULE` and `SCHEDULE REFRESH` keywords are semantically equivalent.

For details about the syntax and use of the `SCHEDULE` clause, see CREATE STREAMING TABLE SCHEDULE clause, or CREATE MATERIALIZED VIEW SCHEDULE clause.

When a schedule is created, a new Databricks job is automatically configured to process the update.

To view the schedule, do one of the following:

- Run the `DESCRIBE EXTENDED` statement from the SQL editor in the Databricks UI. See DESCRIBE TABLE.
- Use Catalog Explorer to view the dataset. The schedule is listed on the **Overview** tab, under **Refresh status**. See What is Catalog Explorer?.

The following examples show how to create a materialized view with a schedule:

Schedule every time interval

This example schedules a refresh every 5 minutes:

```sql
CREATE OR REPLACE MATERIALIZED VIEW catalog.schema.hourly_metrics
  SCHEDULE EVERY 1 HOUR
AS SELECT
    date_trunc('hour', event_time) AS hour,
    count(*) AS events
FROM catalog.schema.raw_events
GROUP BY 1;
```

Schedule using cron

This example schedules a refresh every 15 minutes, at the quarter hour of the UTC time zone:

```sql
CREATE OR REPLACE MATERIALIZED VIEW catalog.schema.regular_metrics
  SCHEDULE CRON '0 */15 * * * ?' AT TIME ZONE 'UTC'
AS SELECT
    date_trunc('minute', event_time) AS minute,
    count(*) AS events
FROM catalog.schema.raw_events
WHERE event_time > current_timestamp() - INTERVAL 1 HOUR
GROUP BY 1;
```

SQL task in a job

Pipeline refreshes can be orchestrated through Lakeflow Jobs by creating SQL tasks that include `REFRESH` commands. This approach integrates pipeline refreshes into existing orchestration using jobs.

There are two ways to create a job for refreshing streaming tables:

- **From the SQL Editor**: Write the `REFRESH` command and click the **Schedule** button to create a job directly from the query.

- **From the Jobs UI**: Create a new job, add a SQL task type, and attach a SQL Query or notebook with the `REFRESHBLE` command.

The following example shows the SQL statement within a SQL task that refreshes a streaming table:

SQL

```
REFRESH STREAMING TABLE catalog.schema.sales;
```

This approach is appropriate when:

- Complex multi-step pipelines have dependencies across systems.
- Integration with existing job orchestration is required.
- Job-level alerting and monitoring is needed.

SQL tasks use both the SQL warehouse attached to the job and the serverless compute that executes the refresh. If using streaming table definition-based scheduling meets the requirements, switching to `TRIGGER ON UPDATE` or `SCHEDULE` can simplify the workflow.

Add a schedule to an existing pipeline

To set the schedule after creation, use the `ALTER STREAMING TABLE` or `ALTER MATERIALIZED VIEW` statement. For example:

SQL

```
-- Alters the schedule to refresh the streaming table when its upstream
-- data gets updated.
ALTER STREAMING TABLE sales
  ADD TRIGGER ON UPDATE;
```

Modify an existing schedule or trigger

If a pipeline already has a schedule or trigger associated, use `ALTER SCHEDULE` or `ALTER TRIGGER ON UPDATE` to change the refresh configuration. This applies whether changing from one schedule to another, one trigger to another, or switching between a schedule and a trigger.

The following example changes an existing schedule to refresh every 5 minutes:

SQL

```
ALTER STREAMING TABLE catalog.schema.my_table
  ALTER SCHEDULE EVERY 5 MINUTES;
```

Drop a schedule or trigger

To remove a schedule, use `ALTER ... DROP`:

SQL

```
ALTER STREAMING TABLE catalog.schema.my_table
  DROP SCHEDULE;
```

Track the status of a refresh

You can view the status of a refresh by viewing the pipeline in the Pipelines UI or by viewing the **Refresh Information** returned by the `DESCRIBE EXTENDED` command for the dataset.

SQL

```
DESCRIBE TABLE EXTENDED <table-name>;
```

Alternately, you can view the dataset in Catalog Explorer and see the refresh status there:

1. Click **Catalog** in the sidebar.

2. In the Catalog Explorer tree at the left, open the catalog and select the schema where your dataset is located.
3. Open the **Tables** item under the schema you selected, and click the streaming table or materialized view.

From here, you can use the tabs under the dataset name to view and edit information about the dataset, including:

- Refresh status and history
- The table schema
- Sample data (requires an active compute)
- Permissions
- Lineage, including tables and other pipelines that this dataset depends on
- Insights into usage
- Monitors that you have created for this dataset

Stop an active refresh

To stop an active refresh in the Databricks UI, in the **Pipeline details** page click **Stop** to stop the pipeline update. You can also stop the refresh with the Databricks CLI or the POST /api/2.0/pipelines/{pipeline_id}/stop operation in the Pipelines REST API.

View the history of runs for a scheduled refresh

If you open your dataset in Catalog Explorer, the details pane on the right side of the workspace shows the **Refresh schedule**. Clicking the schedule (for example, the **Every 1 hour**) link takes you to the job page for the (system managed) job that runs the schedule. You can see the history of runs, including a graph of the last 48 hours of runs, including success or failure and time taken. You can click a specifc run to get more details.

You can't edit this system managed job. To make changes to the schedule, edit the definition of the pipeline with `CREATE OR REFRESH` or with `ALTER`. See Modify an existing schedule or trigger.

Timeouts for refreshes

Pipeline refreshes are run with a timeout that limits how long they can run. For Databricks SQL pipelines created or updated on or after August 14, 2025, the timeout is captured when you update by running `CREATE OR REFRESH`:

- If a `STATEMENT_TIMEOUT` is set, that value is used. See STATEMENT_TIMEOUT.
- Otherwise, the timeout from the SQL warehouse used to run the command is used.
- If the warehouse does not have a timeout configured, a default of 2 days applies.

The timeout is used on the initial create, but also on scheduled refreshes that follow.

For streaming tables that were last updated prior to August 14, 2025, the timeout is set to 2 days.

Example: Set a timeout for a refresh You can explicitly control how long a refresh is allowed to run by setting a statement-level timeout when creating or updating the dataset:

SQL

```sql
SET STATEMENT_TIMEOUT = '6h';

CREATE OR REFRESH MATERIALIZED VIEW my_catalog.my_schema.my_mv
  SCHEDULE EVERY 12 HOURS
AS SELECT * FROM large_source_table;
```

This sets up the materialized view to be refreshed every 12 hours, and if a refresh takes more than 6 hours, it times out and waits for the next scheduled refresh.

How scheduled refreshes handle timeouts

Timeouts are synchronized only when you explicitly run `CREATE OR REFRESH`.

- Scheduled refreshes continue using the timeout captured during the most recent `CREATE OR REFRESH`.

- Changing the warehouse timeout alone does not affect existing scheduled refreshes.

> **IMPORTANT**
>
> After changing a warehouse timeout, run `CREATE OR REFRESH` again to apply the new timeout to future scheduled refreshes.

Get notifications for scheduled refreshes

> **BETA**
>
> The notifications for DDL scheduled refreshes feature is in Beta. Workspace admins can control access to this feature from the **Previews** page by opting into the *System-Managed Job for Materialized Views & Streaming Tables* preview. See Manage Databricks previews.

When you create a schedule for your pipeline, you can edit it to get notifications. There are multiple ways to schedule pipelines, and getting notifications depends on which of these methods you choose:

- **Scheduled with a job**: To get notifications from a SQL task in Lakeflow Jobs, edit the task and add notifications. See SQL task for jobs.

  You have a wide range of options for the notifications to received, and how to receive them. See Add notifications on a job

- **Scheduled with a `SCHEDULE` clause**: To get notifications from a pipeline that is scheduled by a `SCHEDULE` clause in the SQL definition, edit it in the Catalog Explorer:

  i. Open the dataset in Catalog Explorer.

  ii. On the **Overview** tab, under **Refresh schedule**, click ✎ to edit the schedule you want to receive notifications for.

  iii. Under **More options**, add or modify notifications.

You have the option to get notified via email on start, success, or failure of the scheduled refresh. By default, the owner is notified on failure only.

The email includes a link that takes you the run history for the system managed job that orchestrates your schedule. See View the history of runs for a scheduled refresh.

Select a performance mode for scheduled refreshes

The serverless compute used by the pipeline runs in **Performance-optimized mode** when run through the UI.

For pipelines scheduled in the SQL definition, you can select the serverless compute performance mode using the **Performance optimized** setting in Catalog Explorer. When this setting is disabled (the default), the pipeline uses standard performance mode. Standard performance mode is designed to reduce costs for workloads where a slightly higher launch latency is acceptable. Serverless workloads using standard performance mode typically start within four to six minutes after being triggered, depending on compute availability and optimized scheduling.

When Performance optimized is enabled, your pipeline is optimized for performance, resulting in faster startup and execution for time-sensitive workloads.

Both modes use the same SKU, but standard performance mode consumes fewer DBUs, reflecting lower compute usage.

> **BETA**
>
> Modifying the performance mode for scheduled refreshes is in Beta. Workspace admins can control access to this feature from the **Previews** page by opting into the *System-Managed Job for Materialized Views & Streaming Tables* preview. See Manage Databricks previews.

By default, pipelines use the performance-optimized mode when run interactively in the UI or when scheduled with a SQL task, and use the standard mode when

scheduled. To set the compute mode for pipelines scheduled by a `SCHEDULE` clause in the definition, edit the schedule in the Catalog Explorer:

1. Open the dataset in Catalog Explorer.
2. On the **Overview** tab, under **Refresh schedule**, click ✎ to edit the schedule you want to modify.
3. Check **Performance optimized** to use the performance-optimized mode on future scheduled refreshes.

*Last updated on **May 12, 2026***

Best practices for Lakeflow Spark Declarative Pipelines

This page describes recommended patterns for designing, building, and operating pipelines with Lakeflow Spark Declarative Pipelines. Apply these guidelines when starting a new pipeline or improving an existing one.

Choose the right dataset type

Lakeflow Spark Declarative Pipelines offers three dataset types: streaming tables, materialized views, and temporary views. Choosing the right type for each layer of your pipeline avoids unnecessary compute costs and keeps your code easy to reason about.

Streaming tables are the right choice for data ingestion and low-latency streaming transformations. Each input row is read and processed only once, which makes them ideal for append-only workloads, high-volume data, and event-driven processing from cloud storage or message buses.

Materialized views are the right choice for complex transformations and analytical queries. Their results are pre-computed and kept up to date using incremental refresh, so queries against them are fast. You can't directly modify the data in a materialized view — the query definition controls the output.

Temporary views are pipeline-scoped views that organize your transformation logic without materializing any data to storage. Use them for intermediate steps that don't need their own table.

The following table summarizes when to use each type:

Use case	Recommended type	Reason
Ingestion from cloud storage or a message bus	Streaming table	Processes each record once; handles high volume and append-only workloads.
CDC streams (inserts, updates, deletes)	Streaming table	Used as the target of `APPLY CHANGES INTO` for ordered, deduplicated CDC ingestion.
Complex aggregations and joins	Materialized view	Incrementally refreshed; avoids full recomputation on each update.
Dashboard query acceleration	Materialized view	Pre-computed results make queries faster than against raw tables.
Intermediate transformations (no downstream readers)	Temporary view	Organizes pipeline logic without incurring storage cost.

For more information, see Streaming tables, Materialized views, and Lakeflow Spark Declarative Pipelines concepts.

Use declarative CDC instead of imperative MERGE

Implementing change data capture (CDC) with imperative SQL `MERGE` statements requires significant custom code to handle event ordering, deduplication, partial updates, and schema evolution correctly. Each of these concerns must be solved independently, and the resulting code is difficult to maintain and test.

Lakeflow Spark Declarative Pipelines provides the `APPLY CHANGES INTO` statement (SQL) and `apply_changes()` function (Python), which handle ordering, deduplication,

out-of-order events, and schema evolution declaratively. You describe the shape of the change feed and the target table — the pipeline handles the rest. `APPLY CHANGES INTO` supports both SCD Type 1 (overwrite) and SCD Type 2 (history preservation).

For more information, see Change data capture and snapshots and The AUTO CDC APIs: Simplify change data capture with pipelines.

Enforce data quality with expectations

Expectations are true/false SQL expressions applied to every row passing through a dataset. When a row fails the condition, the pipeline responds according to the violation policy you've configured. Expectations emit metrics to the pipeline event log regardless of the policy, so you can track data quality trends over time.

Choose a violation policy

Three violation policies are available. Pick the one that matches your tolerance for bad data:

- **warn** (default): Records that are not valid are written to the target table and flagged in metrics. Use this policy when you need to capture all data but want visibility into quality issues.
- **drop**: Records that are not valid are discarded before writing. Use this when bad rows are expected and shouldn't propagate downstream.
- **fail**: The pipeline update stops on the first invalid record. Use this for critical data where any bad record indicates a serious upstream problem.

The following examples show each policy applied to a streaming table:

SQL Python

SQL

```
-- Warn: write invalid records but track them in metrics
CREATE OR REFRESH STREAMING TABLE orders_raw (
  CONSTRAINT valid_order_id EXPECT (order_id IS NOT NULL)
) AS SELECT * FROM STREAM read_files("/volumes/raw/orders", format =>
```

```
"json");

-- Drop: discard invalid records before writing
CREATE OR REFRESH STREAMING TABLE orders_clean (
  CONSTRAINT non_negative_amount EXPECT (amount >= 0) ON VIOLATION DROP
ROW
) AS SELECT * FROM STREAM(orders_raw);

-- Fail: stop the pipeline on any invalid record
CREATE OR REFRESH STREAMING TABLE orders_critical (
  CONSTRAINT required_customer_id EXPECT (customer_id IS NOT NULL) ON
VIOLATION FAIL UPDATE
) AS SELECT * FROM STREAM(orders_clean);
```

Quarantine invalid records

When you want to preserve dropped records for investigation rather than silently discarding them, use a quarantine pattern. Route rows that fail validation to a separate streaming table by using two flows: one that drops invalid rows from the main table and a second that writes only the invalid rows to a quarantine table. This lets you investigate, fix, and reprocess bad data without contaminating your clean dataset.

For a detailed example of the quarantine pattern, see Expectation recommendations and advanced patterns.

For more information about expectations, see Manage data quality with pipeline expectations.

Parameterize your pipelines

Pipelines have default catalog and schema settings, so code that reads and writes within the same catalog and schema works across environments without any parameters. However, if your pipeline needs to reference a second catalog or schema — for example, reading from a shared source catalog that differs between development and production — avoid hardcoding those names directly in your source code. Instead, define them as pipeline configuration parameters (key-value pairs set in the pipeline settings) and reference them in your code. This lets a single codebase run correctly across environments by swapping the parameter values.

SQL Python

SQL

```
CREATE OR REFRESH MATERIALIZED VIEW transaction_summary AS
SELECT account_id, COUNT(txn_id) AS txn_count, SUM(amount) AS
total_amount
FROM ${source_catalog}.sales.transactions
GROUP BY account_id;
```

For more information, see Use parameters with pipelines.

Choose between triggered and continuous pipeline mode

Triggered mode processes all available data and then stops. It's the right choice for the vast majority of pipelines: those that run on a schedule (hourly, daily, or on demand) and don't require sub-minute data freshness.

Continuous mode keeps the cluster running and processes new data as it arrives. It's appropriate only when your use case requires latency in the seconds-to-minutes range. Because continuous mode requires an always-on cluster, it's significantly more expensive than triggered mode.

For more information, see Triggered vs. continuous pipeline mode and Configure Pipelines.

Use liquid clustering for data layout

Liquid clustering replaces static partitioning and `ZORDER` for optimizing data layout in Delta tables. Static partitioning requires you to select partition columns and reorganize data up front, which can cause data skew for unevenly distributed values. Liquid clustering is self-tuning, skew-resistant, and incremental, rewriting only the data that needs reorganization on each run.

Change clustering columns at any time without rewriting the full table as query patterns evolve.

Define clustering columns in your streaming table definition:

SQL Python

SQL

```
CREATE OR REFRESH STREAMING TABLE events
CLUSTER BY (event_date, region)
AS SELECT * FROM STREAM read_files("/volumes/raw/events", format =>
"parquet");
```

If you're not sure which columns to cluster by, use `CLUSTER BY AUTO` to let Databricks select the optimal clustering columns based on your query workload automatically.

For more information, see Streaming tables and Use liquid clustering for tables.

Manage pipelines with CI/CD and Declarative Automation Bundles

Version-control your pipeline source code and use Declarative Automation Bundles to manage deployments across environments.

For more information, see Create a source-controlled pipeline, Convert a pipeline into a bundle project, and Use parameters with pipelines.

Store pipeline code in version control

Store all pipeline source files (Python and SQL) alongside your bundle configuration in a Git repository. Version-controlling the full project gives you a complete history of changes, makes collaboration easier, and lets you validate changes in a development environment before promoting them to production.

Databricks recommends Declarative Automation Bundles for managing this workflow. A bundle defines your pipeline configuration in YAML alongside your source code, and the `databricks bundle` CLI lets you validate, deploy, and run pipelines from your terminal or a CI/CD system.

Use bundle targets for environment isolation

Bundles enable multiple *targets* (for example, `dev`, `staging`, `prod`), each with its own set of overrides for catalog names, cluster policies, notification addresses, and other settings. Combine bundle targets with pipeline parameters to inject the correct environment-specific values at deploy time, keeping your source code free of environment constants.

A typical workflow looks like this:

1. A developer works on a feature branch, deploying to a personal development pipeline in a dev catalog.
2. On merge to the main branch, a CI system runs `databricks bundle validate` and `databricks bundle deploy --target staging` to validate and deploy the pipeline to a staging environment.
3. After testing passes, the CI system deploys to production with `databricks bundle deploy --target prod`.

Streaming best practices

Use these patterns to manage state, control late data, and keep streaming pipelines reliable.

For more information, see Optimize stateful processing with watermarks, Recover a pipeline from streaming checkpoint failure, and Backfilling historical data with pipelines.

Use watermarks for stateful operations

Watermarks bound the state that the pipeline keeps in memory during stateful streaming operations such as windowed aggregations and deduplication. Without a

watermark, state grows unbounded as the pipeline accumulates data for every possible key, eventually causing out-of-memory errors on long-running pipelines.

A watermark specifies a timestamp column and a tolerance threshold for late data. Records that arrive after the threshold has passed are dropped. Choose a threshold that balances your tolerance for late data against the memory cost of keeping that state open.

The following example computes a one-minute tumbling window aggregation with a three-minute watermark:

SQL Python

SQL

```
CREATE OR REFRESH STREAMING TABLE event_counts AS
SELECT window(event_time, '1 minute') AS time_window, region, COUNT(*) AS
cnt
FROM STREAM(events_raw)
  WATERMARK event_time DELAY OF INTERVAL 3 MINUTES
GROUP BY time_window, region;
```

NOTE

To ensure that aggregations are processed incrementally rather than fully recomputed on each update, you must define a watermark.

Understand streaming state and full refresh

Streaming state is incremental: the pipeline builds and maintains state across updates rather than recomputing from scratch each time. This is what makes stateful streaming efficient, but it also means that if you change the logic of a stateful query (for example, modifying a watermark threshold or changing aggregation columns), the existing state is no longer compatible with the new logic. In this case, you must perform a full refresh to reprocess all historical data with the new logic and rebuild the state from scratch.

A full refresh can also lead to data loss if the source does not retain historical data. For example, a Kafka source with a short retention period may only have the last few

minutes of data available at the time of the refresh, resulting in a table that contains far less data than before. Plan stateful query logic changes carefully, especially for high-volume streams where a full refresh is expensive or where the source has limited data retention. Using the medallion architecture helps by creating bronze tables with minimal transformation, and allows silver or gold tables to recompute from the bronze tables with full history.

Stream-stream joins

Stream-stream joins require a watermark on **both** sides of the join and a time-bounded join condition. The time interval in the join condition tells the streaming engine when no further matches are possible, allowing it to evict state that can no longer be matched. If you omit either the watermarks or the time-bound condition, state grows without bound.

The following example joins ad impression events with click events, requiring the click to occur within three minutes of the impression:

SQL Python

SQL

```sql
CREATE OR REFRESH STREAMING TABLE impression_clicks AS
SELECT imp.ad_id, imp.impression_time, clk.click_time
FROM STREAM(ad_impressions)
    WATERMARK impression_time DELAY OF INTERVAL 3 MINUTES AS imp
JOIN STREAM(user_clicks)
    WATERMARK click_time DELAY OF INTERVAL 3 MINUTES AS clk
ON imp.ad_id = clk.ad_id
  AND clk.click_time BETWEEN imp.impression_time
    AND imp.impression_time + INTERVAL 3 MINUTES;
```

When you join a stream against a static table (a snapshot join), the static table snapshot is refreshed at the start of each microbatch. This means late-arriving dimension records are not retroactively applied to facts that were already processed. If retroactive application is required, use a materialized view or restructure the pipeline.

Optimize pipeline performance

Apply these techniques to reduce compute costs and speed up pipeline updates.

For more information, see Materialized views and Optimize stateful processing with watermarks.

Avoid small files

Triggering a pipeline too frequently on a low-volume source writes a large number of small files to cloud storage. Small files degrade read performance because each file requires a separate metadata lookup and I/O round trip, and cloud storage APIs throttle listing operations at scale. To avoid this, choose a trigger interval that matches your data volume: run triggered pipelines on a schedule that allows a meaningful amount of data to accumulate between updates, rather than continuously.

Handle data skew

Data skew occurs when values in a join or groupBy key are unevenly distributed across partitions, causing a small number of tasks to process the majority of the data. This creates hotspots that increase end-to-end update time. Use liquid clustering to address skew in stored tables. For skew that occurs during in-flight computation, salt highly-skewed keys by appending a random bucket suffix before grouping and aggregating in two stages.

For more information, see Use liquid clustering for data layout.

Use incremental refresh for materialized views

When you use a materialized view for a large aggregation, Lakeflow Spark Declarative Pipelines attempts to incrementally refresh it — processing only the upstream changes since the last update rather than recomputing the full result set. Incremental refresh is significantly cheaper than rerunning the query from scratch on each pipeline trigger. To maximize the chance that a materialized view can be refreshed incrementally, write simple, deterministic aggregation queries and avoid constructs that prevent incremental processing, such as non-deterministic functions.

See Incremental refresh for materialized views.

Optimize joins

For joins where one side is a small dimension table, add a broadcast hint to instruct Spark to broadcast the smaller table to all executors instead of performing a shuffle join:

SQL Python

SQL

```sql
CREATE OR REFRESH MATERIALIZED VIEW enriched_orders AS
SELECT o.*, /*+ BROADCAST(p) */ p.product_name, p.category
FROM orders o
JOIN products p ON o.product_id = p.product_id;
```

For time-series proximity joins (for example, finding the nearest event within a time range), use a range join condition and ensure both sides have a watermark if joining streams, or consider pre-binning events into time buckets before joining.

Monitor your pipelines

The pipeline event log is the primary observability primitive in Lakeflow Spark Declarative Pipelines. Every pipeline run writes structured records to the event log covering execution progress, data quality expectation results, data lineage, and error details. The event log is a Delta table that you can query directly.

To query the event log without knowing the underlying storage path, use the `event_log()` table-valued function on a shared cluster or SQL warehouse:

SQL

```sql
SELECT * FROM event_log('<pipeline-id>')
WHERE event_type = 'flow_progress'
```

Ask Genie

```
ORDER BY timestamp DESC
LIMIT 100;
```

Build data quality dashboards by querying the event log for the expectation metrics. The `details` column contains a nested JSON structure with pass/fail counts for each constraint, which you can use to track quality trends over time and alert on regressions.

For event-driven alerting, use event hooks to trigger custom webhooks or notification services (such as Slack or PagerDuty) when a pipeline fails or when a data quality threshold is breached. Event hooks are Python functions that run in response to pipeline events.

For more information, see Monitor pipelines, Pipeline event log, and Define custom monitoring of pipelines with event hooks.

Use serverless compute

Databricks recommends serverless compute for new pipelines. With serverless, there's no manual cluster configuration — Databricks manages the infrastructure automatically. Serverless pipelines use enhanced autoscaling that can scale both horizontally (more executors) and vertically (larger executor size) in response to workload demands. Serverless pipelines always use Unity Catalog, so governance and lineage tracking are built in by default.

For more information, see Configure a serverless pipeline.

Organize pipelines with the medallion architecture

The medallion architecture organizes data into three logical layers — bronze, silver, and gold — each with a distinct purpose. Mapping Lakeflow Spark Declarative Pipelines dataset types to the right layer keeps each layer's responsibilities clear and makes pipelines easier to maintain.

- **Bronze**: Use streaming tables to ingest raw data from cloud storage, message buses, or CDC sources. Bronze tables preserve the raw source data with minimal transformation, making it possible for silver or gold layers to reprocess from the source in the bronze layer if requirements change.
- **Silver**: Use streaming tables for incremental row-level transformations (filtering, cleaning, and parsing). Use materialized views when silver-layer logic involves enrichment joins against dimension tables or complex aggregations that benefit from incremental refresh.
- **Gold**: Use materialized views to pre-compute aggregations, metrics, and summaries served to dashboards, reporting tools, and downstream consumers.

Separate ingestion (bronze) and transformation (silver and gold) into distinct pipelines whenever possible. Decoupling the layers lets you schedule, monitor, and troubleshoot each layer independently, and a failure in a transformation pipeline doesn't block new data from landing in bronze.

For more information, see Streaming tables and Materialized views.

*Last updated on **Jan 23, 2026***

What happened to Delta Live Tables (DLT)?

The product formerly known as Delta Live Tables (DLT) has been updated to Lakeflow Spark Declarative Pipelines (SDP). If you have previously used DLT, there is no migration required to use Lakeflow Spark Declarative Pipelines: your code will still work in SDP. There are changes that you can make to better take advantage of Lakeflow Spark Declarative Pipelines, both now and in the future, as well as to introduce compatibility with the Apache Spark™ Declarative Pipelines (beginning in Apache Spark 4.1).

In Python code, references to `import dlt` can be replaced with `from pyspark import pipelines as dp`, which also requires the following changes:

- `@dlt` is replaced with `@dp`.
- The `@table` decorator is now used to create streaming tables, and the new `@materialized_view` decorator is used to create materialized views.
- `@view` is now `@temporary_view`.

For more details on the Python API name changes, and differences between Lakeflow SDP and Apache Spark Declarative Pipelines, see What happened to `@dlt`? in the pipelines Python reference.

> **NOTE**
>
> There are still some references to the DLT name in Databricks. The classic SKUs for Lakeflow Spark Declarative Pipelines still begin with `DLT`, and event log schemas with `dlt` in the name have not changed. Python APIs that used `dlt` in the name can still be used, but Databricks recommends moving to the new names.

Learn more

- Lakeflow Spark Declarative Pipelines
- Lakeflow Spark Declarative Pipelines product page

Last updated on ***Apr 21, 2026***

Pipeline Limitations

The following are limitations of Lakeflow Spark Declarative Pipelines that are important to know as you develop your pipelines:

- A Databricks workspace is limited to 1000 concurrent pipeline updates. The number of datasets that a single pipeline can contain is determined by the pipeline configuration and workload complexity.
- The configuration of a pipeline includes references to source files and folders.
  - If the configuration references *only* individual notebooks or files, the limit per pipeline is 100 source files.
  - If the configuration includes folders, you can include up to 50 source entries made up of files or folders.

    Referencing a folder indirectly references the files within that folder. In this case, the limit on the number of files referenced (directly or indirectly) is 1000.

  If you need more than 100 source files, organize them into folders. To learn how to use folders to contain source files, see Pipeline asset browser in the Lakeflow pipeline editor.
- Pipeline datasets can be defined only once. Because of this, they can be the target of only a single operation across all pipelines. The exception is streaming tables with append flow processing, which allows you to write to the streaming table from multiple streaming sources. See Using multiple flows to write to a single target.
- Identity columns have the following limitations. To learn more about identity columns in Delta tables, see Use identity columns in Delta Lake.

    - Identity columns are not supported with tables that are the target of AUTO CDC processing.
    - Identity columns might be recomputed during updates to a materialized views. Because of this, Databricks recommends using identity columns in pipelines only with streaming tables.

- Materialized views and streaming tables published from pipelines, including those created by Databricks SQL, can be accessed only by Databricks clients and applications. However, to make your materialized views and streaming tables accessible externally, you can use the `sink` API to write to tables in an external Delta instance. See Sinks in Lakeflow Spark Declarative Pipelines.

- There are limitations for the Databricks compute required to run and query Unity Catalog pipelines. See the Requirements for pipelines that publish to Unity Catalog.

- Delta Lake time travel queries are supported only with streaming tables, and are *not* supported with materialized views. See Work with table history.

- You cannot enable Iceberg reads on materialized views and streaming tables.

- The `pivot()` function is not supported. The `pivot` operation in Spark requires the eager loading of input data to compute the output schema. This capability is not supported in pipelines.

For Lakeflow Spark Declarative Pipelines resource quotas, see Resource limits.